From Intent to Motion

A Pedagogical Textbook on CAM Compilers, Geometry, Semantics, and Certificates

Dropcut / Makera Z1 project edition

August 2026

Contents

List of Figures

Preface

A computer-aided manufacturing system can look deceptively simple from a distance. The user describes a pocket, the software draws some lines, and a postprocessor writes G-code. That description omits nearly everything that makes the problem intellectually interesting and physically dangerous. A real system must connect a human intention to an actual machine whose axes have limits, whose controller retains modal state, whose tool has a three-dimensional shape, whose setup is uncertain, and whose mistakes can break a cutter or damage a workpiece.

This book develops a way to reason about that entire chain. Its central proposal is that a CAM application should be built as a **refinement compiler for a cyber-physical process**. The source language describes acceptable manufacturing outcomes. Intermediate representations progressively choose geometry, order, timing, machine capabilities, and controller syntax. Every important transformation states what it preserves, what it approximates, and what evidence supports that statement. The final product is not merely a text file. It is an exact job bundle accompanied by explicit assumptions and checkable claims.

A **cyber-physical system** is one in which software decisions interact with continuous physical state through sensors and actuators. A CNC mill is cyber-physical because controller bytes become motor currents, moving axes, cutting forces, heat, vibration, and removed material. A **refinement** is a step from an abstract description to a more concrete one that removes choices or adds detail without introducing behavior forbidden by the abstract description. When numerical approximation is unavoidable, refinement is qualified by a named metric and error bound. These definitions will be made formal in Chapter 1, but they explain the book's title-level claim: the compiler must preserve physical meaning, not merely syntax.

The running case study is a JavaScript-based CAM system for a Makera Z1-class desktop mill. The supplied Dropcut Studio implementation already contains many of the right ingredients: unit brands, frame-tagged points, path objects, high-level operations, cutter-location fields, contour extraction, planning strategies, a canonical command layer, machine profiles, G-code backends, simulation, and a controller client. It also contains instructive gaps. Some are ordinary software defects. Others are deeper category errors: treating a sampled simulation as a proof, treating approximate coordinates as topological identity, or treating a command prefix as the meaning of a compound payload. These examples let us study theory without detaching it from code.

The book has exactly four large chapters.

1. **Meaning before syntax** develops the physical and semantic model. It explains units, frames, paths, trajectories, swept volumes, denotational meaning, operational transitions, contracts, and temporal behavior.
2. **Languages, IRs, and passes** turns that model into a compiler architecture. It introduces staged JavaScript, multi-level IRs, categorical path composition, effectful commands, typestate, SSA-style state tokens, pass contracts, provenance, and translation validation.

3. **Geometry, planning, and optimization** develops the computational side of CAM: drop-cutter geometry, cutter-location fields, contours, offsets, entries, linking, stock models, robust numerics, scheduling, and feed planning.
4. **Certificates and runtime assurance** explains assertions, invariants, abstract interpretation, geometric evidence, proof-carrying artifacts, protocol state machines, hash-bound authorization, and a practical migration path for the Z1 controller and Dropcut compiler.

The goal is not to make every reader a specialist in formal semantics, computational geometry, operations research, and machine control at once. The goal is to build a connected mental model. Each technical idea is introduced because a concrete machining problem demands it. Definitions are followed by worked examples. Counterexamples show why tempting shortcuts fail. Exercises ask the reader to reconstruct the reasoning rather than merely repeat terminology.

How to use this book

A reader with software experience and basic algebra can begin at Chapter 1. Calculus and linear algebra are introduced only where needed. Familiarity with CNC terminology helps, but the relevant concepts are defined. Readers who already know CAM may move quickly through the physical preliminaries, but they should not skip the distinction among a geometric path, a timed trajectory, and a material-removal process; much of the later architecture depends on it.

Code examples use TypeScript-like notation because that matches the project. They are specifications first and implementation sketches second. Branded scalar types and phantom frame parameters are useful, but TypeScript erases them at runtime. Every static guarantee must therefore be paired with validation at serialization, deserialization, plugin, and execution boundaries.

Mathematical notation is used when it removes ambiguity. A formula is always explained in prose. When a model is idealized, the text states what has been omitted. A proof about a nominal mesh is not automatically a proof about the actual clamped part. A guarantee about controller bytes is not automatically a guarantee about the mechanics. The difference between a theorem and an assumption is one of the book's recurring themes.

The running job

We will repeatedly return to one small job. A rectangular stock blank measures 60 mm by 40 mm by 8 mm. The work coordinate origin is at the lower-left corner of the stock top. We want to machine a 30 mm by 20 mm pocket centered in the blank, 4 mm deep, leaving a finished wall and floor tolerance of 0.05 mm. A flat end mill roughs the pocket; a second pass finishes the boundary and floor. Two clamps lie near the upper and lower stock edges.

This job is deliberately ordinary. It still forces us to answer difficult questions:

- Does "make a pocket" denote one path or a set of acceptable outcomes?
- In which coordinate frame are the dimensions expressed?
- What tool geometry is assumed?
- How are entry and linking moves distinguished from cutting moves?
- What happens if the controller is still in incremental mode?
- How does the compiler know that a low traverse does not cross remaining stock?

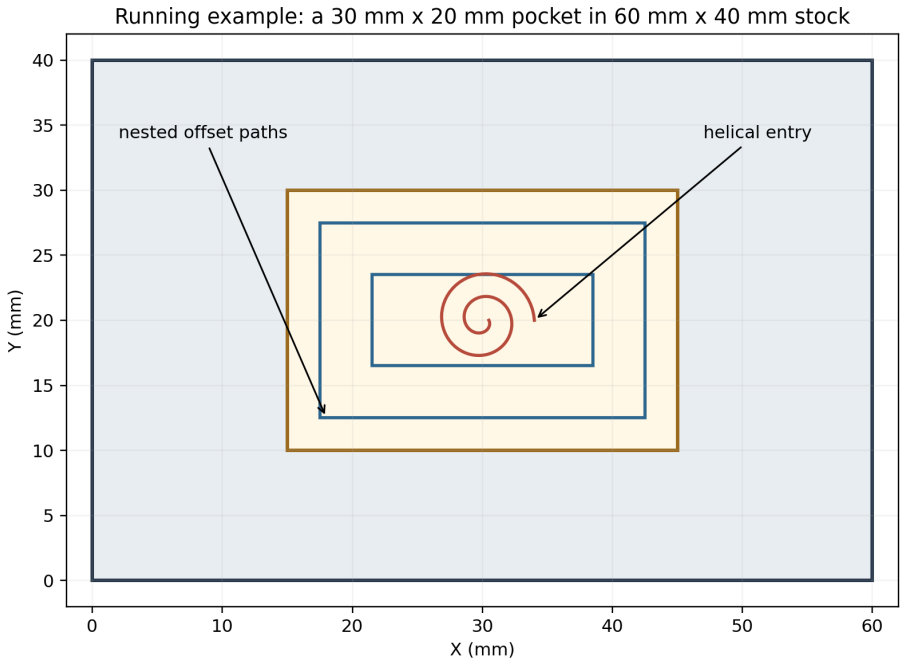

Figure 1: The running pocket example.

- What does a claim such as "no gouge" actually quantify over?
- Which facts can be checked before execution, and which require live machine state?

By the end of Chapter 4, the job will have become a complete chain of artifacts, claims, and runtime checks.

Conventions

Lengths are expressed internally in millimetres. A point is written $p = (x, y, z)^F$ when the superscript names its coordinate frame. The symbol S denotes current stock, P the target or protected part, O fixtures and other obstacles, and T a tool or tool assembly. A geometric path is usually written γ, a timed trajectory $x(t)$, and a machine or process state σ.

A statement labeled **Definition** introduces terminology used later. A **Worked example** carries out a calculation or design step. A **Counterexample** demonstrates a failure of an appealing but invalid rule. A **Fundamental idea** callout expands a prerequisite that readers from another discipline may not know. A **Design consequence** turns theory into an engineering rule.

Chapter 1

Meaning Before Syntax

Chapter orientation

A novice CAM implementation often begins with a function named `generateGCode`. That starting point feels productive because G-code is visible and the machine accepts it. It is also a trap. The text `G1 X30 Y20 F400` has no safe, context-free meaning. Its interpretation depends on units, distance mode, active plane, work offset, feed mode, tool, machine state, firmware dialect, and prior blocks. More importantly, a list of controller blocks does not state what workpiece result the user requested.

This chapter therefore begins one level above code and one level below user interface. We will define what exists physically, what a manufacturing operation means, and how execution changes state. Only after those foundations are clear will later chapters design a language and compiler.

1.0.1 Learning objectives

After this chapter, you should be able to:

- distinguish manufacturing intent from one implementation;
- model units, frames, tools, stock, target, fixtures, and controller state explicitly;
- distinguish a path from a trajectory and a trajectory from a material-removal action;
- explain denotational, operational, axiomatic, and temporal semantics;
- state safety properties as predicates over continuous executions;
- derive a simple preflight condition using weakest preconditions;
- identify assumptions that no compiler can establish from source code alone.

1.1 Why a pocket is not a polyline

1.1.1 Motivation

Suppose a user asks for the running pocket. One planner generates nested rectangular offsets. Another uses parallel raster lines. A third uses adaptive clearing and a final contour. These paths look different, take different times, and load the tool differently, yet all may produce an acceptable pocket.

If the source meaning were “execute this exact point list,” only one of them could be correct. That would make optimization and strategy choice impossible by definition. The user’s intention is instead a constraint on the

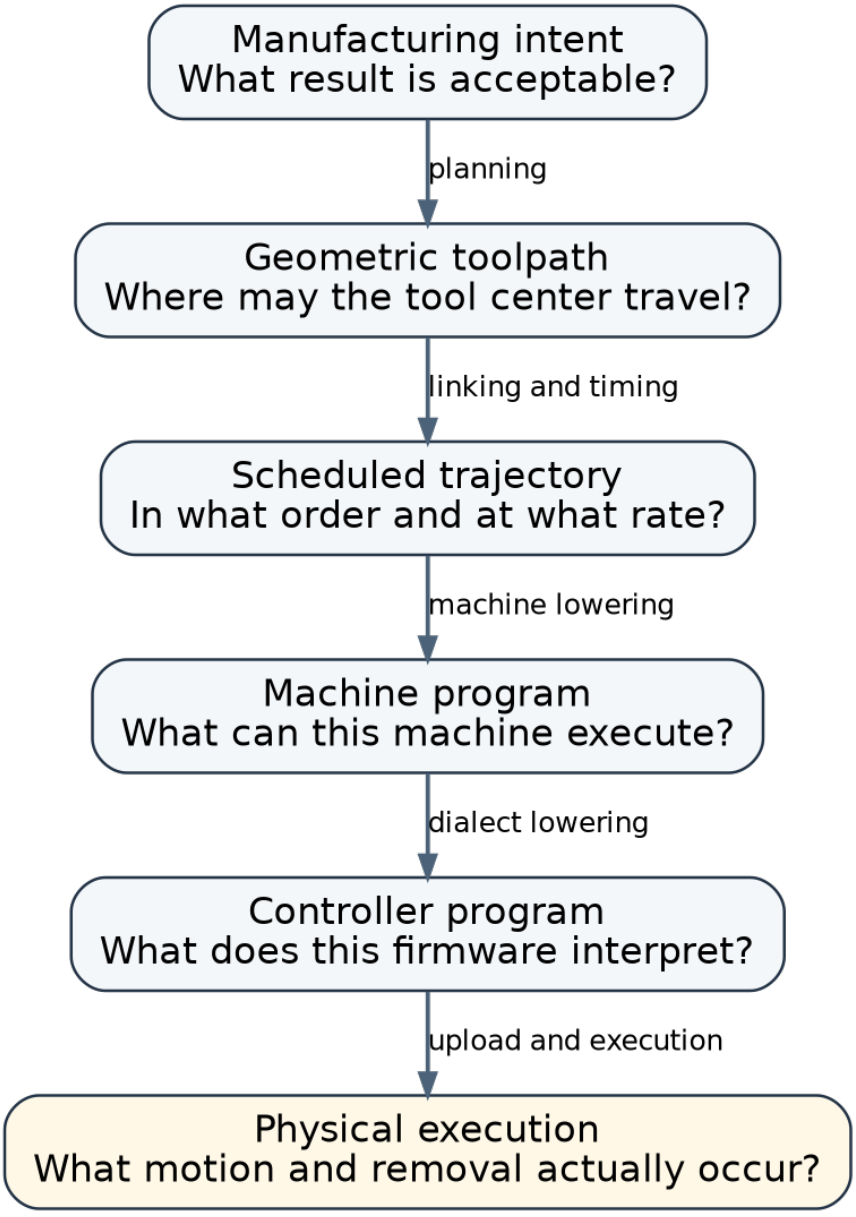

Figure 1.1: The semantic stack from intent to physical execution.

resulting material.

1.1.2 Definition: manufacturing intent

A **manufacturing intent** is a specification of acceptable outcomes and process constraints, not a commitment to one sequence of axis motions. It may name:

- a region that must be removed;
- material that must remain;
- a target surface or tolerance zone;
- roughing allowance;
- required tools or prohibited tools;
- precedence constraints;
- surface finish or scallop limits;
- setup and inspection requirements.

Mathematically, an intent I can be interpreted as a set of acceptable final states:

$$[\![I]\!] \subseteq \Sigma,$$

where Σ is the set of possible complete machining states. For the pocket, $[\![I]\!]$ contains every final state whose stock has the required cavity within tolerance, whose protected material remains, and whose process constraints have been respected.

A toolpath is then a **witness candidate**: a proposed implementation that should lead to an element of $[\![I]\!]$.

A **behavior** is an observable execution or outcome of a program. At a high level, behavior may mean the final workpiece and terminal machine state. At a low level, it may include every motion, probe result, alarm, and acknowledgement. We say that a concrete program **refines** an intent when all of its relevant behaviors are allowed by that intent, after accounting for the declared approximation and abstraction. Refinement is one-way: an intent may permit many strategies, while one compiled job chooses only one of them.

> **Fundamental idea — sets as specifications**. A **predicate** is a true-or-false condition on an object. Writing $[\![I]\!]$ as a set does not imply that the compiler enumerates every acceptable workpiece. A predicate such as `meetsPocketTolerance(stock)` can define the set implicitly. Set notation gives us a precise way to say that a planner's output belongs to the allowed family.

1.1.3 Worked example: two valid implementations

Let R be the 30 mm by 20 mm rectangular pocket footprint and let the requested floor depth be $z = -4$ mm. Ignoring corner-radius details for the moment, an intent can say:

1. Material inside R above $z = -4$ must be removed, except for at most 0.05 mm residual allowance.
2. Material below $z = -4.05$ is protected.
3. Material outside the wall tolerance band is protected.
4. The tool assembly must not intersect either clamp.

A raster planner may cut horizontal lines with a stepover of 1.5 mm. An offset planner may cut shrinking rectangles. If both satisfy the four conditions, both refine the same intent.

This distinction gives an optimizer room to choose. The planner can minimize cycle time, tool changes, or rapid distance while treating the intent as a hard constraint.

1.1.4 Counterexample: using emitted G-code as the source meaning

Imagine that a user edits the operation tolerance from 0.10 mm to 0.03 mm, but the UI preview and certificate still reference an old cached `.nc` file. If the G-code text is treated as the semantic source, the system has no principled way to say that the file no longer implements the current intent. The missing relation is between the high-level operation and the emitted artifact.

A second failure is subtler. Suppose two G-code files trace the same nominal points, but one begins in `G90` absolute mode and the other inherits `G91` incremental mode from the controller. Textual similarity does not imply behavioral equality. Meaning belongs to an interpreter plus an initial state.

> **Design consequence — keep intent alive.** Preserve feature identity, target geometry, tolerances, tools, and operation provenance through the compiler. Do not reduce the program to anonymous points before the checks that require manufacturing meaning.

1.2 The physical state a command can change

1.2.1 Motivation

A path alone cannot tell us whether a move is legal. A line through space may be a safe rapid when the stock is already cleared, a crash when stock remains, or a valid cut when the spindle is running with the correct tool. Legality depends on state.

1.2.2 Definition: machine/process state

A **machine/process state** is the collection of information needed to interpret a command and predict its relevant effects. One useful model is:

$$\sigma = (q, \dot{q}, F, W, T_a, C, S, P, O, M, t, U).$$

The components are:

- q: axis positions or machine configuration;
- $\dot{q}$: velocity, when dynamics matter;
- F: frame graph and transforms;
- W: active work coordinate system;
- T_a: active tool assembly, including holder;
- C: process state such as spindle, coolant, feed, and overrides;
- S: current stock;
- P: target and protected material;
- O: fixtures and machine obstacles;
- M: controller mode and modal state;
- t: time;
- U: uncertainty and assumptions associated with the other components.

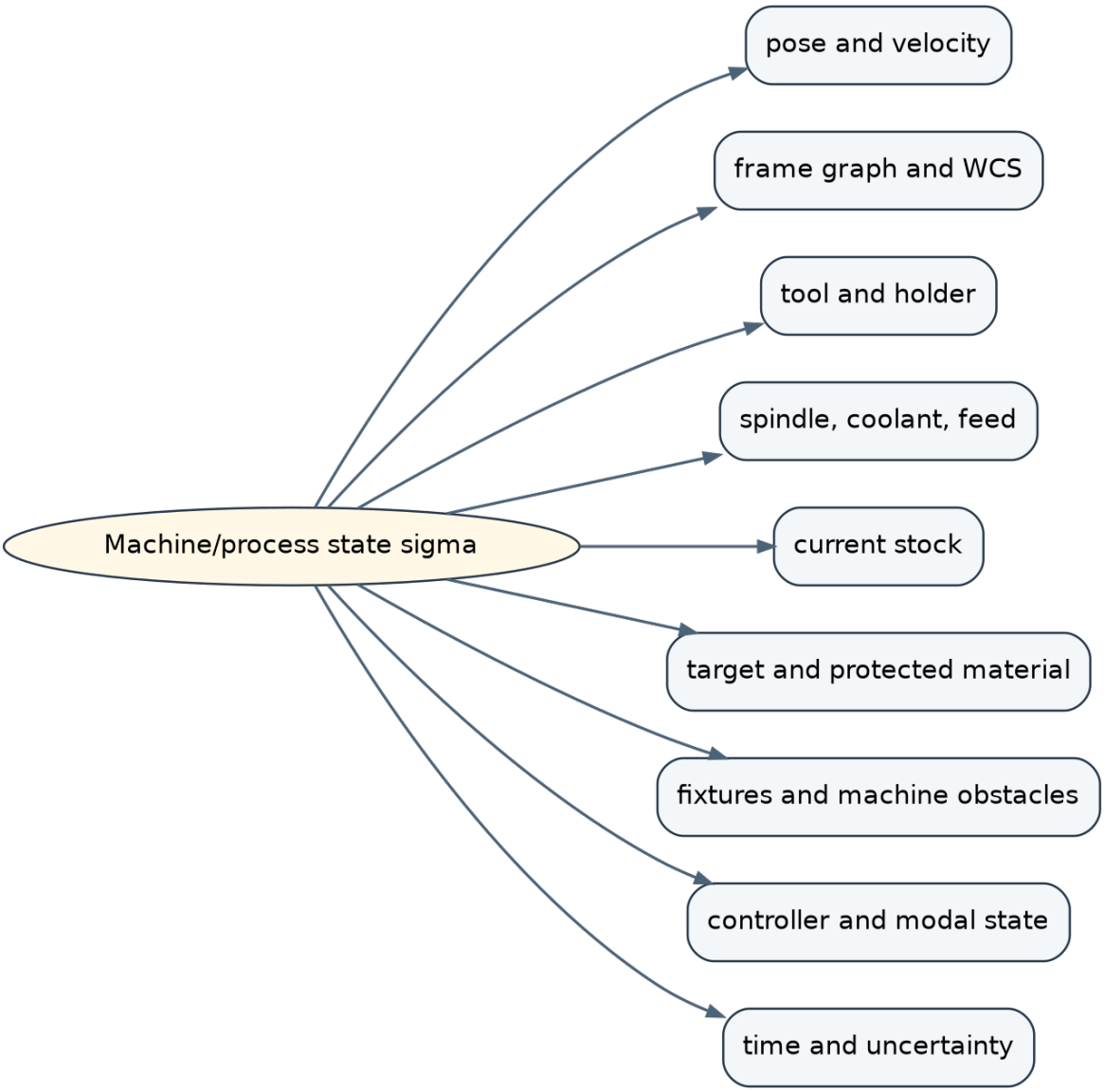

Figure 1.2: Components of a machine and process state.

Not every pass needs every component. A unit checker does not need stock. A rapid-clearance checker does. An important architectural rule follows: a checker should receive every artifact needed to state its proposition. If a "gouge checker" receives no target geometry, it cannot establish a target-gouge claim.

The state model may look intimidating because it lists more information than one algorithm can hold in memory. Its purpose is to prevent accidental omission. Each analysis works over a **projection** of the state: the components relevant to its proposition. The projection must be conservative. A holder-clearance checker that projects away the holder has projected away the fact it claims to establish.

1.2.3 Worked example: one cutting move

Assume the machine is homed, tool T1 is selected, the spindle is running, and the current tool-tip pose is $(15, 10, 2)^{work}$. A helical entry follows a path γ down to the first roughing level at $z = -1.5$ mm.

Before execution, the state includes the original stock S_0. After successful cutting, an idealized state update is:

$$S_1 = S_0 \setminus \text{Sweep}(T_c, \gamma),$$

where T_c is the cutting geometry of the tool. The pose becomes the endpoint of γ. The spindle remains on. The target P and fixtures O do not change.

This one command has several distinct postconditions:

- endpoint pose updated;
- stock decreased;
- target not excessively penetrated;
- fixtures unchanged;
- spindle state preserved;
- trace and provenance recorded.

A validator that checks only endpoint coordinates observes very little of this meaning.

1.2.4 Fundamental idea: state does not mean one mutable object

The mathematical state σ is a semantic model. An implementation does not need one giant mutable JavaScript object containing a mesh, controller socket, and every UI value. Different representations can project the state they need. The requirement is that the semantics make dependencies explicit enough to prevent unsound assumptions.

For example, a scheduler may use a symbolic stock-state identifier rather than storing a full voxel volume in every node. A certificate checker can resolve that identifier to the exact content-addressed stock artifact.

1.3 Units are part of meaning

1.3.1 Motivation

Numbers such as `3`, `400`, and `12000` are meaningless without dimensions. A tool diameter of 3 mm, a feed of 400 mm/min, and a spindle speed of 12,000 rpm are all represented by JavaScript numbers. Accidentally

exchanging them may still produce finite values and valid JSON.

1.3.2 Definition: physical dimension and unit

A **physical dimension** identifies the kind of quantity: length, time, angle, speed, rotational rate, and so forth. A **unit** selects a scale within a dimension, such as millimetres or inches for length.

A useful core policy is to choose one internal unit for each dimension and convert at the boundary. For length:

```ts
type Mm = number & { readonly __brand: "mm" };
type MmPerMin = number & { readonly __brand: "mm/min" };
type Rpm = number & { readonly __brand: "rpm" };

const mm = (x: number): Mm => x as Mm;
const inch = (x: number): Mm => mm(25.4 * x);
```

The brand makes incompatible quantities distinct to TypeScript while erasing to a number at runtime.

1.3.3 Worked example: a dimensionally correct pocket call

```ts
job.rectPocket({
  x: mm(15),
  y: mm(10),
  w: mm(30),
  h: mm(20),
  depth: mm(4),
  stepdown: mm(1.5),
  stepover: ratio(0.40),
  feed: mmPerMin(450),
});
```

The `depth` and `feed` fields cannot be interchanged without a type error in checked TypeScript. The `stepover` is deliberately a dimensionless ratio of tool diameter, not a bare length.

1.3.4 Counterexample: branded values without runtime validation

A brand is a compile-time fiction. This code defeats it:

```ts
const feed = JSON.parse(input).feed as MmPerMin;
```

If the JSON contains `"fast"`, `NaN`, a negative number, or a value in inches, the cast proves nothing. Constructors and deserializers must validate finite values, ranges, and units. Plugins and script boundaries are runtime trust boundaries.

1.3.5 Quantization is also a unit-level effect

When G-code emits coordinates with three decimal places, each coordinate is rounded with an error of at most:

$$\varepsilon_{round} = \tfrac{1}{2}\,10^{-3}\ \text{mm} = 0.0005\ \text{mm}.$$

This contribution is small but real. It belongs in a quantitative error argument and must be associated with the metric it affects. We will return to typed error budgets in Chapter 3.

> **Design consequence — convert once, validate twice.** Convert external units into canonical units at the API boundary. Validate dimensions statically where possible and values dynamically at every untrusted boundary.

1.4 Coordinate frames prevent a high-cost class of errors

1.4.1 Motivation

The running pocket is described in a work frame whose origin lies on the stock. The machine axes use a machine frame. A mesh may arrive centered around its own origin. A clamp model may be expressed in a fixture frame. The tuple `(15, 10, -4)` has no operational meaning until its frame is known.

1.4.2 Definition: frame and rigid transform

A **coordinate frame** is a named coordinate system in which points and vectors are expressed. A **rigid transform** maps coordinates between frames using rotation and translation while preserving distances.

A three-dimensional rigid transform belongs to the group $SE(3)$. It can be represented by a rotation matrix R and translation vector t:

$$p^B = R_{A\to B}p^A + t_{A\to B}.$$

In TypeScript:

```ts
interface Point3<F extends FrameId> {
  readonly x: Mm;
  readonly y: Mm;
  readonly z: Mm;
  readonly frame: F;
}

interface Transform<A extends FrameId, B extends FrameId> {
  readonly from: A;
  readonly to: B;
  readonly r: Matrix3;
  readonly t: Vec3;
}
```

Composition is defined only when frames meet:

$$T_{A\to C} = T_{B\to C} \circ T_{A\to B}.$$

A **category** consists of objects, composable arrows, identity arrows, and associative composition. Every rigid transform is invertible. Frames as objects and invertible transforms as arrows therefore form a **groupoid**: a category in which every arrow has an inverse.

> **Side route — why "groupoid" is useful.** The term is not required for ordinary API use. It summarizes three laws worth property-testing: identity transforms change nothing; compatible transforms compose associatively; and composing a transform with its inverse returns the original point. Naming the algebra helps us derive tests instead of inventing examples ad hoc.

1.4.3 Worked example: placing the part on the machine

Suppose the work origin is 80 mm to the right and 45 mm forward of the machine origin, with the stock top 12 mm below the machine's Z reference. A pure translation gives:

$$T_{work \to machine}(x, y, z) = (x + 80, y + 45, z - 12).$$

The pocket-floor point $(15, 10, -4)^{work}$ becomes:

$$(95, 55, -16)^{machine}.$$

A travel checker must operate in a frame compatible with the machine envelope. A target-gouge checker may remain in the work or part frame if all compared geometry shares that frame.

1.4.4 Counterexample: changing a label without applying a transform

This cast is not a transform:

```
const machinePoint = workPoint as Point3<"machine">;
```

It changes only the compiler's belief. The numeric coordinates remain in the work frame. A safe API permits such a cast only inside a boundary function whose caller explicitly assumes responsibility, and runtime frame identifiers should still be checked.

1.4.5 Frame uncertainty

A probed work offset is not one exact transform. It is better modeled as a set or interval of possible transforms. A point at radius r from the origin experiences positional uncertainty from an angular error $\delta\theta$ of approximately:

$$\delta p \leq r\,\delta\theta.$$

Thus a small rotational uncertainty can dominate far from the probed datum. A certificate must either include this propagation or state that the frame is assumed exact.

1.5 Path, time law, trajectory, and swept volume

1.5.1 Motivation

CAM discussions often use “toolpath” to mean several different objects. This ambiguity causes incorrect algorithms. A polyline says where to go but not how fast. A feed schedule says how progress changes over time. Material removal depends on the three-dimensional tool shape at every pose, not only on the centerline.

1.5.2 Definition: geometric path

A **geometric path** is a continuous map from normalized progress to configuration or pose:

$$\gamma : [0, 1] \to Q.$$

For a fixed-orientation three-axis mill, Q may be approximated by $\mathbb{R}^3$. For multi-axis machining, Q includes orientation and often axis configuration.

1.5.3 Definition: time law and trajectory

A **time law** is a monotone map:

$$s : [0, T] \to [0, 1].$$

The resulting **trajectory** is:

$$x(t) = \gamma(s(t)).$$

The same path may have several time laws. One respects only a feed limit; another also respects axis velocity, acceleration, jerk, and controller tracking constraints.

1.5.4 Definition: swept volume

For a tool solid T and pose trajectory $x(t)$, the **swept volume** is:

$$\text{Sweep}(T, x) = \bigcup_{t \in [0,T]} x(t)T.$$

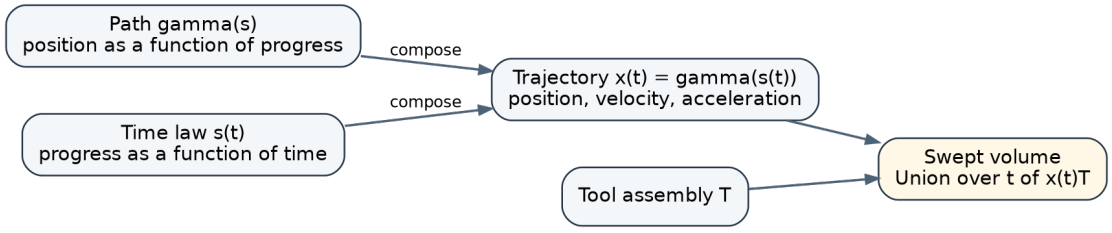

Figure 1.3: A path and a time law combine to form a trajectory; the moving tool forms a swept volume.

1.5.5 Worked example: a linear cut

Let a 6 mm diameter flat end mill move from $(15, 10, -1.5)$ to $(45, 10, -1.5)$ at 450 mm/min. The geometric path is:

$$\gamma(s) = (15 + 30s, 10, -1.5).$$

At constant feed, the move length is 30 mm and the duration is:

$$T = \frac{30 \text{ mm}}{450 \text{ mm/min}} = 0.066\bar{6} \text{ min} = 4 \text{ s}.$$

The time law is $s(t) = t/4$ for $0 \le t \le 4$ seconds. The cutting sweep is a horizontal capsule-like prism formed by translating the cutter disc along the segment and extending through the flute engagement depth.

1.5.6 Counterexample: endpoint-only collision checking

Suppose both endpoints are clear of a narrow clamp that lies halfway between them. An endpoint checker reports success. The continuous swept volume intersects the clamp. Sampling at ten equally spaced times can still miss an obstacle thinner than the sample spacing or an event between samples.

The property to prove is quantified over all time:

$$\forall t \in [0, T], \quad x(t)T_a \cap O = \emptyset,$$

where T_a is the complete tool assembly. A finite sampling method needs an additional theorem connecting samples to the continuous interval. Without that theorem it is simulation, not proof.

1.6 Motion classes carry different process meanings

1.6.1 Motivation

The same geometric line may be used to cut, traverse, probe, or inspect. Treating these actions as one generic "move" loses the state changes and safety rules that distinguish them.

1.6.2 Definition: canonical machining action

A **canonical machining action** is a controller-independent operation with explicit physical intent. NIST's canonical machining functions use the same semantic-waist idea when interpreting RS274/NGC into machine-independent actions [R1, R2]. A compact action algebra might contain:

```
type CanonicalCommand =
  | ToolChange
  | SpindleAction
  | CoolantAction
  | Traverse
  | Cut
  | Probe
```

```
  | Dwell
  | Pause;
```

A traverse promises no material removal. A cut updates stock. A probe produces a measurement and stops under a contact condition. A pause changes the controller lifecycle without prescribing a path.

1.6.3 Worked example: safe traverse versus `G0`

The running job finishes one offset loop at low Z and must move to the start of the next loop. The manufacturing request is:

> Reach point B without cutting and without intersecting current stock, target, fixture, or holder constraints.

A **conservative choice** is deliberately biased in the direction that cannot hide the failure being checked. Raising to a known-clear plane may be slower than a direct move, but it reduces the set of possible collisions. Later chapters will distinguish conservative inner and outer geometric approximations precisely. A conservative postprocessor may lower this one traverse into three motions:

```
raise Z to a certified clearance plane
move X and Y to the next entry point
lower Z to the entry height
```

This is not equivalent to blindly emitting one three-axis `G0 X... Y... Z...` block if the controller can perform a dogleg rapid. The canonical action expresses the desired property; target lowering chooses an implementation compatible with the controller semantics.

1.6.4 Counterexample: inserting a connector into two cut paths

Suppose path A ends at one side of a fixture and path B starts at the other. A generic path utility notices discontinuity and inserts a straight line. Geometry becomes continuous, but the new segment has no motion class. If interpreted as cut, it gouges stock. If interpreted as traverse, it collides with the fixture. Continuity repair is a semantic operation and must be checked as such.

1.7 Denotational semantics: what outcome does an operation permit?

1.7.1 Motivation

We need a mathematical account of intent and process that does not depend on execution order or controller syntax. Denotational semantics supplies this view by assigning each construct a mathematical meaning.

1.7.2 Definition: denotational semantics

A **denotational semantics** maps a language construct to a mathematical object in a compositional way. For a machining command, a relation is more realistic than a deterministic function:

$$[\![c]\!] : \Sigma \to \mathcal{P}(\Sigma \times Trace \times Outcome).$$

The powerset $\mathcal{P}$ represents possible outcomes caused by measurement uncertainty, controller faults, following error, or deliberately underspecified choices.

A **relation** between sets A and B is a set of allowed pairs in $A \times B$. A deterministic function selects exactly one output for each input; a relation may allow several or none. The **powerset** $\mathcal{P}(X)$ is the set of all subsets of X. Thus a semantics returning $\mathcal{P}(\Sigma \times Trace \times Outcome)$ returns a set of possible successors. This is not abstract decoration: a probe may contact within an interval, a command may fault, and a timeout may leave several protocol states possible.

For an ideal deterministic cut along trajectory x with cutter T_c:

$$[\![\text{Cut}(x, T_c)]\!](\sigma) = \sigma[S := S \setminus \text{Sweep}(T_c, x),\ q := x(T)].$$

For a traverse:

$$S' = S.$$

1.7.3 Manufacturing intent as a predicate

A pocket intent can be defined as a predicate on final stock S_f. Let V_{req} be material required to be absent and $P_{protected}$ material required to remain. Then an idealized specification is:

$$V_{req} \cap S_f = \emptyset$$

and:

$$P_{protected} \subseteq S_f.$$

Tolerance replaces exact emptiness and inclusion with metric bounds. Roughing allowance changes the required and protected regions.

1.7.4 Worked example: roughing and finishing as refinement

A roughing operation leaves 0.2 mm radial and axial allowance. It is correct relative to a roughing intent whose protected region includes that allowance. A finishing operation then refines the residual stock toward the final intent.

The two operations compose at the stock level:

$$S_2 = (S_0 \setminus R_{rough}) \setminus R_{finish}.$$

Set difference is associative in the useful sense:

$$(S \setminus R_1) \setminus R_2 = S \setminus (R_1 \cup R_2).$$

However, this algebraic equality does **not** imply the operations may be reordered physically. Intermediate stock affects entry, support, engagement, and safe linking. Chapter 3 will distinguish geometric commutativity from process independence.

Two operations **commute** when executing them in either order produces equivalent relevant behavior. Set subtraction by two fixed volumes commutes at the final-stock level, but real machining operations have additional reads and effects. We must prove commutativity under the complete process model before reordering.

1.7.5 Fundamental idea: specification versus implementation

The denotation of a high-level operation is usually a set of acceptable outcomes. A planner chooses one implementation and should produce a witness showing why it belongs to that set. This is the key move that turns strategy code from a trusted oracle into an untrusted producer whose output can be checked.

1.8 Operational semantics: how execution proceeds

1.8.1 Motivation

Denotational semantics tells us the allowed result, but controller faults, modal state, probing, pause, and resume require a step-by-step account. Operational semantics describes transitions between configurations.

1.8.2 Definition: small-step operational semantics

Structural operational semantics provides the standard rule-based vocabulary used here [R5]. A **small-step operational semantics** uses rules of the form:

$$\langle c, \sigma \rangle \to \langle c', \sigma' \rangle.$$

Each rule performs one conceptual transition. A completed atomic command may instead use:

$$\langle c, \sigma \rangle \Downarrow (\sigma', e),$$

where e is an emitted event or trace fragment.

1.8.3 Worked rule: cutting

A simplified rule is:

$$\frac{\text{Homed}(\sigma) \quad \text{WCSKnown}(\sigma) \quad \text{Tool}(\sigma) = T \quad \text{SpindleValid}(\sigma) \quad \text{PathSafe}(\gamma, \sigma)}{\langle \text{Cut}(\gamma, T, f), \sigma \rangle \Downarrow (\sigma', \text{CutTrace}(\gamma, f))}$$

with:

$$\sigma'.S = \sigma.S \setminus \text{Sweep}(T_c, \gamma), \qquad \sigma'.q = \gamma(1).$$

If a premise fails, a different rule produces a diagnostic or fault outcome. The rule makes the preconditions explicit instead of hiding them in an informal comment.

1.8.4 Worked rule: probing

A probe move introduces nondeterminism because contact may occur at an uncertain point or not occur at all:

$$\langle \text{Probe}(\gamma), \sigma \rangle \Downarrow \begin{cases} (\sigma_c, \text{Contact}(p, U_p)), \\ (\sigma_n, \text{NoContact}), \\ (\sigma_a, \text{Alarm}). \end{cases}$$

The measured point and its uncertainty become a value used by later frame construction. A command sequence that executes `probe` and then `set offset` without binding the result is semantically incomplete.

1.8.5 Why a reference interpreter matters

A pure interpreter for canonical commands provides one executable definition of meaning. It can drive:

- tests;
- abstract analysis;
- preview traces;
- time estimation;
- pass validation;
- comparison with parsed G-code.

The production simulator may use faster approximations. The reference interpreter should favor clarity and explicitness.

1.9 Axiomatic semantics: contracts around commands

1.9.1 Motivation

When designing an API or validator, we often want to reason locally: what must be true before this command, and what can callers rely on afterward? Axiomatic semantics expresses this with logical contracts.

1.9.2 Definition: Hoare triple

Hoare logic introduced this style of local contract reasoning [R3]. A **Hoare triple** has the form:

$$\{P\}\ c\ \{Q\}.$$

It means: if precondition P holds and command c terminates normally, then postcondition Q holds.

For a traverse along γ from a to b:

$$\{q = a \wedge \text{Homed} \wedge \text{Clear}(\gamma, T_a, S, O, P)\}$$

$$\text{Traverse}(\gamma)$$

$$\{q = b \land S' = S \land T' = T \land C' = C\}.$$

The stock-equality postcondition is an important semantic distinction from a cut.

1.9.3 Definition: weakest precondition

Dijkstra's predicate-transformer view makes this backward reasoning systematic [R4]. The **weakest precondition** $wp(c, Q)$ is the least restrictive condition that guarantees postcondition Q after command c. For sequencing:

$$wp(c_1; c_2, Q) = wp(c_1, wp(c_2, Q)).$$

This lets us derive preflight requirements backward from the desired final condition.

1.9.4 Worked example: deriving a pocket-job preflight

Consider the simplified program:

```
select tool T1
start spindle at 12,000 rpm
traverse to entry
cut roughing paths
retract to safe pose
stop spindle
```

Desired final condition:

$$Q = \text{Pose} = p_{safe} \land \text{SpindleOff}\,.$$

Work backward.

1. `stop spindle` requires a live controller session and establishes `SpindleOff`.
2. `retract` requires a continuous collision-free path from the final cut pose to p_{safe}.
3. `cut roughing paths` requires homing, known WCS, selected tool, running spindle, valid feed, safe sweep, and target allowance.
4. `traverse to entry` requires free-space clearance under the current stock state.
5. `start spindle` requires a selected tool and supported RPM.
6. `select tool T1` requires that T1 is available and compatible with the setup.

The resulting wp is a structured preflight specification. Some clauses can be checked at compile time, some at runtime, and some remain physical assumptions.

1.9.5 Counterexample: a checklist detached from program semantics

A UI may display fixed checkboxes for "homed," "cover closed," and "tool loaded." If the actual program contains no motion but writes a configuration file, homing is irrelevant. If it resumes a held job, current

queued motion and job identity are crucial. A weakest-precondition approach derives requirements from the actual action class and program structure.

1.10 Invariants: properties that survive every step

1.10.1 Motivation

A postcondition describes one operation. A long-running controller and a multi-pass compiler need properties that remain true across many transitions.

1.10.2 Definition: invariant

An **invariant** is a predicate I satisfying:

1. Initialization: $I(\sigma_0)$.
2. Preservation:

$$I(\sigma) \wedge \sigma \to \sigma' \Rightarrow I(\sigma').$$

Typical CAM invariants include:

- stock monotonicity: $S_{i+1} \subseteq S_i$;
- every coordinate has one known frame;
- a cutting action has a selected tool and valid spindle state;
- a controller can execute only the content hash it has authorized;
- a stop-class command is never blocked by a failed motion preflight;
- no unsupported operation remains after machine lowering.

1.10.3 Worked proof: stock monotonicity

Assume the command language contains cuts, traverses, spindle actions, tool changes, dwells, and probes. Define:

$$I(\sigma) \equiv S \subseteq S_0.$$

Initialization is immediate because $S = S_0$. For preservation:

- a cut updates $S' = S \setminus R$, so $S' \subseteq S \subseteq S_0$;
- every other command preserves S, so $S' = S \subseteq S_0$.

Therefore the invariant holds for every finite execution.

This proof does not establish **correct** removal. A broken cut may remove protected material while stock still decreases. Invariants must be chosen to match the desired property.

1.10.4 Representation invariants versus physical invariants

A `PathBuilder` can guarantee that a path's declared `end` equals the terminal point derived from its segment list. This is a representation invariant. It does not prove that the arc radii are coherent, that a polysegment

begins at the cursor, or that the path avoids fixtures. Clear naming prevents a local data-structure guarantee from being advertised as machining safety.

1.11 Temporal semantics: behavior over a whole controller trace

1.11.1 Motivation

Upload, start, hold, resume, abort, disconnect, and alarm are not well modeled as isolated functions. They are concurrent protocol transitions. Safety may depend on what is always true; liveness may depend on what eventually happens.

1.11.2 Definition: trace, safety, and liveness

A **trace** is a finite or infinite sequence of states and events:

$$\tau = \sigma_0, e_0, \sigma_1, e_1, \ldots$$

A **safety property** says that a bad event never happens. A finite prefix can demonstrate a violation. A **liveness property** says that a desired event eventually happens under stated fairness and environment assumptions.

Temporal logic and TLA-style state-transition specifications make these trace properties explicit [R6]. Temporal logic uses operators such as:

- $\Box P$: always P;
- $\Diamond P$: eventually P.

Examples:

$$\Box(\text{Running}(h) \Rightarrow \text{Authorized}(h))$$

and:

$$\Box(\text{AbortRequested} \Rightarrow \Diamond(\text{Stopped} \vee \text{Faulted})).$$

1.11.3 Worked example: resume is not read-only

A resume command may move no axis at the instant it is parsed. It nonetheless enables queued motion. Its effect class is therefore **state-enabling**, not read-only. A correct protocol requires fresh preflight and authorization at the transition from held to running.

This example illustrates why command classification should be based on operational effect rather than spelling. A bare ~ byte and a textual `resume` verb can share the same semantic class.

1.11.4 Counterexample: timeout means failure

The host sends a start command and times out waiting for the reply. There are at least two possible states:

1. The controller never received the command.

2. The controller started the job and the acknowledgement was lost.

Blindly retrying can issue a second start or corrupt protocol state. The honest successor is an **ambiguous** set of states. The session should be quarantined until a trusted status query re-establishes the controller boundary.

1.12 The end-to-end correctness statement

We can now state what this compiler is trying to achieve.

Let I be manufacturing intent, B the exact deployed job bundle, and A the set of assumptions about tool geometry, frames, fixtures, firmware, and physical machine behavior. Let $\mathrm{Exec}(B, A)$ be the possible physical traces when B executes under assumptions A. Let α abstract a low-level trace to relevant manufacturing observations.

A bounded refinement statement is:

$$\mathrm{verify}(B) = \mathrm{true} \wedge \mathrm{AssumptionsHold}(A)$$

$$\Longrightarrow \forall \tau \in \mathrm{Exec}(B, A), \quad \alpha(\tau) \in N_{\varepsilon}([\![I]\!]).$$

Here N_{ε} is an allowed neighborhood under named metrics. The theorem says that every permitted execution produces an outcome within the declared tolerances of the intent.

A **metric** is a rule $d(x, y)$ for measuring distance that obeys non-negativity, identity, symmetry, and the triangle inequality. An ε-**neighborhood** of a set contains every object within distance ε of some allowed object. Different machining claims require different metrics: point-position error, Hausdorff distance between sets, normal-direction surface error, maximum gouge depth, or timing error. The symbol α is an **abstraction function**: it forgets low-level details that the high-level intent does not observe, such as packet boundaries, while retaining material removal and safety-relevant events.

This is stronger than "the preview looked right" and more honest than "safe: true." It also makes incompleteness visible. If fixture geometry is absent, the fixture-clearance proposition cannot be proved. If the controller semantics are unknown, final-byte equivalence is conditional. If the actual tool is unmeasured, its diameter remains an assumption.

> **Fundamental idea — proofs are conditional.** Formal reasoning does not remove assumptions. It makes them explicit and prevents a claim from silently depending on facts the system never checked.

1.13 Chapter synthesis: specify the running pocket

We can now give the running job a semantic specification.

1.13.1 Artifacts

- stock solid S_0;
- target/protected part P;

- clamp obstacles O;
- roughing tool assembly T_1;
- optional finishing tool assembly T_2;
- work-to-machine transform set $\mathcal{T}$;
- machine and controller profile M.

1.13.2 Required outcome

Let V_{pocket} be the ideal cavity and let $\delta = 0.05$ mm. The final stock must satisfy a target-deviation predicate, for example:

$$d_H(\partial S_f \cap R, \partial P \cap R) \leq \delta,$$

with additional directional conditions if normal error is the intended metric. Protected material outside the tolerance zone must remain.

1.13.3 Process requirements

- all cutting paths use a compatible selected tool;
- spindle speed and feed remain within supported ranges;
- every traverse is disjoint from current stock and obstacles using the complete tool assembly;
- machine configurations remain inside the admissible envelope;
- the final spindle state is off;
- the controller executes exactly the certified bytes;
- runtime assumptions about machine identity, WCS, tool, setup, and interlocks hold.

This specification is not yet an algorithm. Chapter 2 designs the language and compiler that can carry it. Chapter 3 constructs candidate paths and schedules. Chapter 4 develops the evidence that lets a small checker accept or reject the final bundle.

1.13.4 Check your understanding

Before continuing, try to explain the running job without using the words `G0`, `G1`, or "point list." You should be able to name the acceptable final stock, the protected material, the tool and holder, the relevant frames, the distinction between cut and traverse, and the runtime assumptions. If that description is difficult, the semantic model is not yet explicit enough.

1.14 Exercises

1.14.1 Concept checks

1. Give two geometrically different toolpaths that implement the same pocket intent. State which properties must be equal and which may differ.
2. Classify each object as intent, path, trajectory, or physical trace: a 0.4 tool-diameter stepover; a list of XYZ points; a velocity-versus-time curve; encoder samples.
3. Explain why a point requires a frame even when all current jobs use only G54.
4. Distinguish a representation invariant from a safety invariant using a path example.

5. Give one safety property and one liveness property for an upload protocol.

1.14.2 Worked derivations

6. A 40 mm line is traversed at 1,200 mm/min. Compute the constant-feed duration. Then explain why this does not establish actual duration on a machine with acceleration limits.
7. A frame has angular uncertainty 0.0003 rad. Bound the resulting positional uncertainty 80 mm from the datum.
8. Write a Hoare triple for `startSpindle(12000)` that mentions tool selection and maximum RPM.
9. Derive the weakest precondition of `traverse; cut; stopSpindle` for the postcondition "spindle off and stock conforms to roughing intent."
10. Prove stock monotonicity for a language that also contains an additive-manufacturing command. What changes?

1.14.3 Counterexample construction

11. Construct three points a, b, c for which $d(a, b) < \varepsilon$ and $d(b, c) < \varepsilon$ but $d(a, c) \geq \varepsilon$.
12. Design a path whose endpoints are clear of an obstacle but whose swept volume collides.
13. Give a controller payload whose first token is read-only but whose later content can move the machine. What must a safe parser do?
14. Describe a state in which `resume` is more dangerous than a new single jog command.

1.14.4 Design exercise

15. Write a structured semantic specification for drilling four holes. Include required removal, protected material, probe or setup assumptions, tool state, and terminal controller state. Do not write G-code.

Chapter 2

Languages, Intermediate Representations, and Compiler Passes

Chapter orientation

Chapter 1 described the meaning we want to preserve. We now need software structures that can carry that meaning from a user-friendly program to controller bytes. This is the compiler-design problem.

The most common architectural mistake is to search for one universal representation. A single list of `{x, y, z, feed}` records seems attractive because every component can manipulate it. In practice it either loses information early or grows hundreds of optional fields whose valid combinations are undocumented. A trustworthy compiler uses several intermediate representations, each designed around the questions that can still be asked at that stage.

This chapter proceeds from the authoring language inward. We will use JavaScript as a staged macro language, define a ladder of IRs, explain why paths form a useful category, model machine actions as effects, and give every compiler pass an explicit semantic contract. The running pocket will travel through the complete ladder.

2.0.1 Learning objectives

After this chapter, you should be able to:

- explain why user JavaScript should construct an inert AST rather than define the core semantics;
- distinguish elaboration, normalization, planning, scheduling, lowering, and serialization;
- use a multi-level IR without duplicating meaning arbitrarily;
- explain the categorical structure of path composition and the limits of approximate equality;
- explain monads and Kleisli composition through machine-state sequencing;
- compare typestate with an SSA-style state token;
- specify pass correctness as exact preservation, refinement, bounded approximation, witness satisfaction, or feasible optimization;
- design provenance, hashing, diagnostics, and deterministic compilation records.

2.1 Why an authoring language and a compiler language are different

2.1.1 Motivation

Users want loops, parameters, helper functions, and reusable modules. They may want to generate a family of fixtures or repeat a hole pattern. JavaScript provides these conveniences immediately. The compiler, however, wants immutable, finite, typed, serializable data. It cannot soundly analyze an arbitrary closure that may inspect time, network state, mutable globals, or a future callback.

Trying to use the same language object for both jobs creates a conflict. The user-facing language wants expressive computation; the trusted pipeline wants stable data.

2.1.2 Definition: staged authoring

Staged authoring divides execution into phases. In the first phase, a macro program runs and constructs an inert program representation. In later phases, the compiler analyzes and transforms that representation without invoking user code.

```
JavaScript source
      |
      | isolated evaluation
      v
immutable authoring AST
      |
      | elaboration
      v
explicit Plan IR
```

The boundary is complete when no arbitrary closure, promise, prototype-dependent object, or ambient host reference remains in the AST.

This is an instance of **multi-stage programming**: code in one stage constructs data or code for a later stage [R16]. The important engineering property is not metaprogramming cleverness; it is that the stage boundary creates a finite artifact that can be validated, hashed, cached, and interpreted by another implementation.

2.1.3 Worked example: a parametric hole row

The user writes:

```
const drillTool = tools.flatEndMill({
  diameter: mm(3),
  fluteLength: mm(10),
});

job.withTool(drillTool, () => {
  for (let i = 0; i < 5; i++) {
    job.drill({
      points: [{ x: mm(10 + 8 * i), y: mm(8) }],
      depth: mm(5),
      feed: mmPerMin(120),
```

```
    });
  }
});
```

After evaluation, the loop no longer exists. The authoring AST contains five explicit drill operations with source provenance. A later planner does not know or care whether the operations came from a loop, a generated file, or direct calls.

2.1.4 Definition: capability object

A **capability object** is an explicit collection of operations granted to the script. Instead of ambient APIs, the authoring environment provides only constructors such as:

```
interface CamCapabilities {
  units: UnitConstructors;
  tools: ToolConstructors;
  geometry: GeometryConstructors;
  strategy: StrategyConstructors;
  job: PlanBuilder;
  diagnostics: DiagnosticSink;
}
```

This improves API clarity and reduces accidental authority. It is not by itself a secure sandbox.

2.1.5 Counterexample: same-realm `new Function`

A host may shadow `fetch`, `process`, and `globalThis` as function parameters and then run:

```
new Function(...names, source)(...values);
```

This closes obvious accidental routes but leaves the script in the same language realm. Standard constructors and prototype chains can lead back to powerful globals. An infinite loop also prevents a timeout handler in the same thread from running. The correct security boundary is a separately terminable worker, process, isolate, or interpreter with resource limits enforced externally.

2.1.6 Determinism

A reproducible authoring run should be modeled as:

$$AST = \text{evaluate}(source, apiVersion, inputs, seed).$$

Every file, mesh, material table, tool library, project parameter, and random seed must be declared. Time and randomness should be fixed or unavailable. A compilation record can be:

```
interface ScriptEvaluationRecord {
  sourceHash: Hash;
  languageVersion: string;
  apiVersion: string;
  inputs: readonly ArtifactRef[];
```

```
  seed?: bigint;
  resultAstHash: Hash;
  diagnostics: readonly Diagnostic[];
}
```

> **Design consequence — JavaScript ends at the AST.** The trusted compiler should consume only immutable, schema-checked, content-addressed data. User code must not execute during planning, validation, preview, or postprocessing.

2.2 Elaboration turns convenient syntax into explicit meaning

2.2.1 Motivation

Authoring APIs often permit defaults and scopes: a current tool, current spindle speed, default work offset, or inherited tolerance. These are useful to writers but dangerous if they remain ambient. Compiler passes should not ask, "What was current when this object was built?"

2.2.2 Definition: elaboration

Elaboration resolves implicit or syntactic information into an explicit typed representation. It typically performs:

- name and reference resolution;
- default insertion;
- unit conversion;
- frame attachment;
- tool lookup;
- scope expansion;
- finite-value and domain checks;
- source-location attachment;
- schema version normalization.

The result is an **Elaborated Plan IR** that can be saved and compiled without re-running JavaScript.

2.2.3 Worked example: expanding scopes

Authoring code:

```
job.withTool(T1, () => {
  job.withSpindle({ speed: rpm(12000) }, () => {
    job.rectPocket({ /* ... */ });
  });
});
```

Elaborated node:

```
interface PocketOperation {
  id: OperationId;
  frame: "work:G54";
```

```
  tool: ToolRef;              // T1 resolved
  spindleSpeed: Rpm;          // 12000 explicit
  feed: MmPerMin;
  region: Rectangle2;
  depth: Mm;
  stepdown: Mm;
  stepover: Ratio;
  provenance: Provenance;
}
```

No downstream pass needs a stack of dynamic scopes.

2.2.4 Counterexample: asynchronous scope combinator

A synchronous helper saves the old tool, invokes a callback, and restores the tool in `finally`:

```
withTool(tool, body) {
  const previous = activeTool;
  activeTool = tool;
  try { body(); }
  finally { activeTool = previous; }
}
```

If `body` returns a promise and performs work after `await`, the tool is restored before the later calls execute. The surface syntax suggests lexical scope, but the runtime behavior is different.

Sound choices are:

1. forbid promises and detect them;
2. make the helper async and await the body;
3. prefer immutable context passing in the elaborated builder.

For a deterministic macro language, the third is easiest to reason about.

2.2.5 Definition: legality predicate

Every IR should have a **legality predicate**: a precise condition describing which values are valid at that level. The Elaborated Plan IR may require:

```
all references resolve
all quantities are finite and dimensionally valid
all points carry known frames
all operation IDs are unique
all tools have positive usable geometry
all defaults are explicit
no executable values remain
```

A conversion is partial. If it cannot produce legal output, it returns structured diagnostics rather than inventing meaning.

2.3 The IR ladder

Multi-level compiler infrastructures such as MLIR make legality and conversion between abstraction-specific dialects explicit [R12]. CAM needs the same discipline because manufacturing intent, geometric paths, machine trajectories, and controller syntax have different semantics.

2.3.1 Motivation

The running pocket begins as a high-level request and ends as bytes. Several decisions happen in between. Combining them into one pass makes correctness claims too broad and failures too hard to locate. Splitting them arbitrarily creates needless conversions. The right levels correspond to distinct semantic questions.

2.3.2 Definition: intermediate representation

An **intermediate representation**, or **IR**, is a language used between compiler stages. It has syntax, well-formedness rules, and semantics appropriate to one abstraction level. An IR is not merely an internal TypeScript interface. It is a contract among passes.

An **abstraction level** determines which distinctions are visible. Intent IR distinguishes a pocket from a freeform surface but does not commit to a raster. Controller IR distinguishes modal operations and exact interpolation but may no longer retain the original feature boundary except through provenance. Lowering should remove only distinctions that later stages no longer need.

2.3.3 Level 1: Authoring AST

The Authoring AST preserves source-oriented concepts:

- source spans and comments;
- names and declarations;
- user-level operation constructors;
- explicit data after macro execution.

It excludes live closures and ambient references.

2.3.4 Level 2: Elaborated Plan IR

This level resolves units, frames, references, defaults, tools, and scoped settings. It is the stable project representation.

2.3.5 Level 3: Manufacturing Intent IR

Intent IR states what should be manufactured:

- features and target regions;
- roughing and finishing requirements;
- tolerance metrics;
- allowances;
- precedence and setup constraints;
- protected and permitted material.

It does not contain a particular tool-center curve.

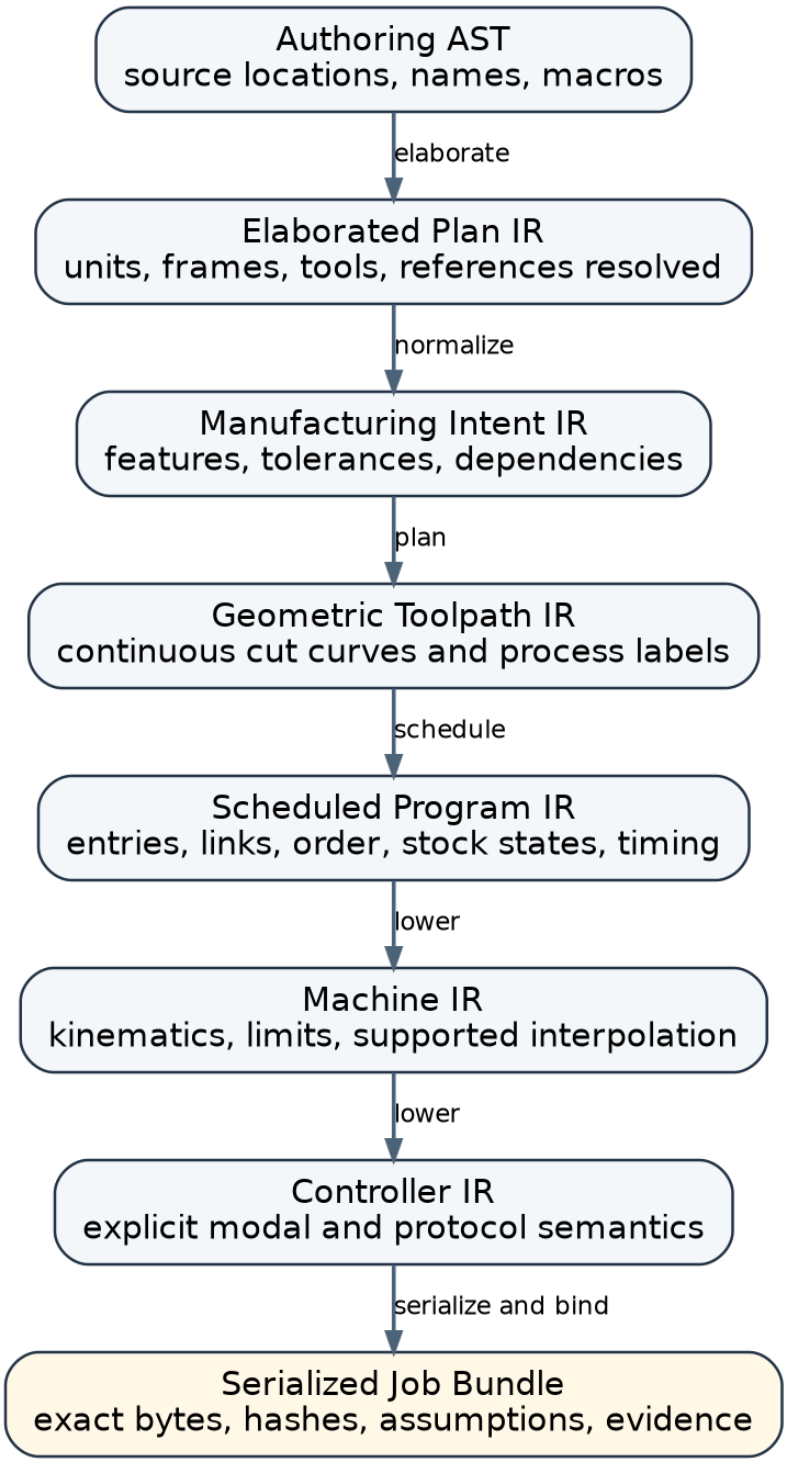

Figure 2.1: A multi-level IR ladder for CAM.

2.3.6 Level 4: Geometric Toolpath IR

Toolpath IR contains proposed continuous paths with process meaning:

```
interface PlannedCut<F extends FrameId> {
  path: Path<F>;
  operation: OperationRef;
  tool: ToolRef;
  phase: "entry" | "rough" | "finish" | "exit";
  directionality: "reversible" | "forward-only";
  feedPolicy: FeedPolicy;
  approximation: readonly ErrorBound[];
}
```

It still excludes machine-specific controller syntax.

2.3.7 Level 5: Scheduled Program IR

Scheduling introduces:

- a legal order;
- path orientation choices;
- entries, exits, links, and retracts;
- tool changes;
- stock-state versions;
- probe dependencies;
- time laws or feed schedules.

This level is stateful because link safety depends on when material has been removed.

2.3.8 Level 6: Machine IR

Machine lowering resolves:

- axis configuration and kinematics;
- travel limits;
- supported interpolation;
- machine frames;
- feed, spindle, acceleration, and jerk capabilities;
- tool-change and probing support;
- safe implementation of abstract traverses.

Every operation remaining after full machine lowering must be supported or rejected.

2.3.9 Level 7: Controller IR

Controller IR represents the target dialect explicitly but structurally:

```
type ControllerOp =
  | SetUnits
  | SetDistanceMode
```

```
  | SelectPlane
  | SelectWorkOffset
  | LinearMove
  | ArcMove
  | ProbeMove
  | SetSpindle
  | ProgramEnd;
```

It has an interpreter for modal semantics.

2.3.10 Level 8: Serialized job bundle

The final artifact includes:

- exact controller bytes;
- canonical byte hash;
- machine and firmware profile hashes;
- setup, tool, stock, target, and fixture references;
- certificate graph;
- runtime assumption manifest;
- optional preview artifacts linked to the same hashes.

2.3.11 Worked example: the pocket across the ladder

At Intent IR:

```
PocketIntent {
  region: rectangle(15, 10, 30, 20),
  floorZ: mm(-4),
  wallTolerance: mm(0.05),
  floorTolerance: mm(0.05),
  roughingAllowance: mm(0.2),
}
```

At Geometric Toolpath IR:

```
rough level -1.5: three nested loops + helical entry
rough level -3.0: three nested loops + linked descent
rough level -3.8: three nested loops
finish wall: one boundary loop at final dimension
finish floor: raster or spiral at z = -4.0
```

At Scheduled IR, every loop has an orientation, entry, link, tool, and referenced stock state. At Machine IR, unsupported arcs may be linearized and feeds constrained. At Controller IR, every modal prerequisite is explicit. The final bundle binds exact lines such as `G1 X...` to the original pocket operation through provenance.

2.3.12 Counterexample: `ValidatedProgram` as a language level

A type brand called `ValidatedProgram` is a useful API gate: it prevents accidental postprocessing before a validation function runs. Semantically, however, validation is evidence about an artifact, not a new programming language. Different claims attach at different stages. Path continuity applies to Toolpath IR. Travel applies after machine lowering. Modal equivalence applies after serialization. Runtime identity applies only to an execution instance.

Prefer:

```
interface Certified<T> {
  artifact: T;
  artifactHash: Hash;
  claims: readonly Claim[];
  evidence: readonly EvidenceRef[];
}
```

while retaining local brands as convenience guards.

2.4 Paths form a useful category

2.4.1 Motivation

Toolpaths must compose. A roughing strategy produces loops; an entry planner produces a descent; a linker connects operations. Without a structural composition rule, implementations concatenate arrays and hope endpoints agree.

2.4.2 Definition: category

A **category** consists of:

- objects;
- arrows between objects;
- an identity arrow for each object;
- associative arrow composition when endpoints match.

For paths:

- objects are poses or exact endpoint identities;
- an arrow $p : A \to B$ is a path from A to B;
- the stationary path id_A is the identity;
- concatenation composes $p : A \to B$ with $q : B \to C$.

$$q \circ p : A \to C.$$

Associativity says:

$$(r \circ q) \circ p = r \circ (q \circ p).$$

2.4.3 Worked example: hierarchical job assembly

```
const entry: Path<A, B> = planEntry(...);
const loop1: Path<B, C> = roughLoop1(...);
const link: Path<C, D> = planLink(...);
const loop2: Path<D, E> = roughLoop2(...);

const roughing = concat(entry, loop1, link, loop2);
```

Because composition is associative, the compiler can group operations into reusable subprograms without changing the segment sequence:

```
const firstLevel = concat(entry, loop1);
const rest = concat(link, loop2);
const roughing = concat(firstLevel, rest);
```

Property tests can check identity and associativity.

> **Side route — algebra becomes a test generator.** Instead of writing only hand-selected examples, generate random compatible paths and test `concat(id, p) = p`, `concat(p, id) = p`, and the associativity of segment sequences. For floating-point geometry, the test must use the same explicit equality or bounded relation claimed by the API.

2.4.4 Definition: path equality

A subtlety appears immediately: what counts as the same path? Literal array equality is too strict. Curves may trace the same geometric image under different parameterizations. Geometric equality is often taken modulo monotone reparameterization.

For approximate curves, a metric such as Hausdorff distance can define bounded equivalence:

$$d_H(\gamma_1, \gamma_2) \le \varepsilon.$$

This is a claim with a bound, not exact identity.

The **Hausdorff distance** between two compact point sets A and B is the greatest distance from a point in either set to its nearest point in the other:

$$d_H(A, B) = \max\left\{ \sup_{a \in A} d(a, B),\ \sup_{b \in B} d(b, A) \right\}.$$

For a circular arc and a chordal polyline, this metric captures the maximum geometric deviation of their images. It does not capture direction, feed, **topology**—connectivity, components, boundaries, and holes—or timing. Two curves can have small Hausdorff distance while being traversed in opposite directions, so the complete refinement relation needs more than one scalar.

2.4.5 Counterexample: tolerance is not equality

Suppose endpoint matching uses:

$$a \sim b \iff d(a,b) < \varepsilon.$$

This relation is not transitive. In one dimension, with $\varepsilon = 1$:

$$a = 0, \quad b = 0.75, \quad c = 1.5.$$

Then $a \sim b$ and $b \sim c$, but $a \nsim c$. Therefore tolerance-based endpoint matching does not create an ordinary category with exact object identity. Repeated composition may accumulate drift.

2.4.6 Better endpoint designs

1. **Exact symbolic identities.** Adjacent segments reference the same endpoint object or node ID.
2. **Canonical snapping.** Coordinates are snapped once to a proven grid and the displacement is recorded.
3. **Join witnesses.** Composition returns the measured mismatch and selected repair.
4. **Metric-enriched paths.** Composition accumulates an explicit uncertainty bound.

```
interface JoinWitness<F extends FrameId> {
  leftEnd: Point3<F>;
  rightStart: Point3<F>;
  displacement: Mm;
  repair: "none" | "snap" | "insert-checked-link";
}
```

An inserted line is not a harmless numerical patch. It is a new motion with cut or traverse semantics.

2.4.7 Representation invariant versus full validity

A path builder that computes `end` from the last segment prevents a stale endpoint field. It does not prove:

- that an arc endpoint lies on the declared circle;
- that an arc axis is unit length;
- that a bulk polyline begins at the current cursor;
- that curvature is feasible;
- that the path avoids protected geometry.

The category structure is valuable because it narrows one class of errors. It is not a universal safety proof.

2.5 Machine commands are effectful computations

2.5.1 Motivation

Paths compose geometrically. Commands do more. They modify machine state, may fail, emit diagnostics and traces, produce measurements, consume time, and update stock. Ordinary function composition does not directly sequence this context.

2.5.2 Definition: effectful command

A simple command that returns a value of type A can be modeled as:

$$M(A) = \Sigma \to Result(A \times \Sigma, E).$$

In TypeScript:

```
type Command<A> =
  (state: MachineState) =>
    Result<{ value: A; state: MachineState }, CommandError>;
```

A cut returns no interesting value but changes state. A probe returns a measurement and changes state.

2.5.3 Why ordinary composition fails

Suppose:

$$f : A \to M(B), \qquad g : B \to M(C).$$

Ordinary composition $g \circ f$ is ill typed because $f(a)$ is an effectful result $M(B)$, not a plain B.

2.5.4 Definition: bind and monad

A **bind** operation sequences an effectful value with a function that produces the next effect:

$$bind : M(A) \times (A \to M(B)) \to M(B).$$

Moggi and Wadler developed the connection between computational effects, monads, and compositional programming [R7, R8]. A **monad** supplies `pure` and `bind` satisfying identity and associativity laws. The laws ensure that sequencing is stable under regrouping.

The laws are:

- left identity: `pure(a).flatMap(f)` behaves like `f(a)`;
- right identity: `m.flatMap(pure)` behaves like `m`;
- associativity: regrouping nested `flatMap` calls does not change behavior.

For machine commands, associativity lets a library package `setup`, `roughing`, `finishing`, and `shutdown` as subprograms and regroup them without changing failure propagation or state threading. The laws do not say that commands commute; order remains significant.

```
function bind<A, B>(ma: Command<A>, f: (a: A) => Command<B>): Command<B> {
  return state0 => {
    const ra = ma(state0);
    if (!ra.ok) return ra;
    return f(ra.value.value)(ra.value.state);
  };
}
```

2.5.5 Definition: Kleisli composition

Kleisli composition composes effectful arrows:

$$(g \star f)(a) = bind(f(a), g).$$

The Kleisli category has ordinary types as objects and functions $A \to M(B)$ as arrows. Effects become part of the composition rule.

2.5.6 Worked example: probe then set offset

```
const establishTop: Command<void> =
  probeZ(maxTravel)
    .flatMap(measurement =>
      setWorkOrigin({ z: measurement.contactZ }));
```

If probing fails, `setWorkOrigin` does not run. If it succeeds, its measured value determines the next command. Bind handles both state and failure propagation.

2.5.7 Fundamental idea: monads are not decorative here

The useful statement is not "CAM is a monad." The useful statement is:

> Stateful, fallible commands can be given one lawful sequencing operator, so larger programs inherit predictable composition instead of duplicating error and state plumbing.

The API need not expose category-theory vocabulary. The semantics should still obey the laws.

2.6 Indexed commands, typestate, and SSA state tokens

2.6.1 Motivation

The simple command type allows `cut` to be called in any state and fail at runtime. Some illegal sequences can be prevented earlier by encoding state transitions in types or IR dependencies.

2.6.2 Definition: typestate

Typestate was introduced to make legal protocol operations depend on program state [R9]. **Typestate** represents protocol state in the type system. Operations are available only for compatible states. An indexed command has the conceptual type:

$$Cmd\langle S_{before}, S_{after}, A\rangle.$$

Composition requires the post-state of one command to match the pre-state of the next.

```
type StartSpindle<S extends Homed & HasTool> =
  Cmd<S, S & SpindleRunning, void>;
```

```
type Cut<S extends Homed & HasTool & SpindleRunning & KnownWcs> =
  Cmd<S, S & AtPathEnd, void>;
```

2.6.3 Limit of typestate

TypeScript types are erased. A forged value, stale controller report, or physical tool mismatch defeats the compile-time model. Typestate constrains program construction; runtime preflight re-establishes live facts.

2.6.4 Definition: SSA-style state token

Parameterized effects generalize state-indexed computations whose pre-state and post-state types differ [R10]. A practical IR alternative threads a single-assignment **state token** through effectful commands. Each operation consumes one token and produces the next.

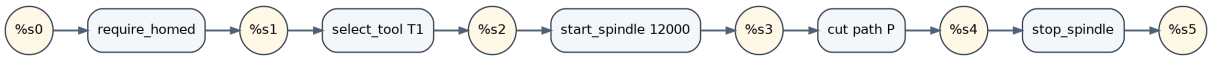

Figure 2.2: An SSA-style state token makes command order explicit.

```
%s0 = machine.initial
%s1 = machine.require_homed %s0
%s2 = tool.select %s1 @T1
%s3 = spindle.start %s2 12000rpm
%s4 = motion.cut %s3 path=@rough1 feed=450
%s5 = spindle.stop %s4
```

The token is an ordering witness, not necessarily the complete state value. It prevents silent reordering across effects and makes probe data dependencies explicit.

Static single assignment, or SSA, is an IR discipline in which each value has one definition [R11]. The token applies SSA to effects. A probe can produce both a measurement value and a successor state token:

```
%m, %s1 = probe.toward %s0 direction=-Z max=10mm
%tf     = frame.from_probe %m datum=@stock_top
%s2     = frame.install %s1 %tf as=G54
```

The frame installation depends on the measurement as data and on the probe completion as state. The two dependencies are visible to optimizers and checkers.

2.6.5 Worked example: rejecting an invalid optimization

An optimizer sees:

```
%s1 = spindle.start %s0
%s2 = cut %s1 @P
%s3 = spindle.stop %s2
```

Moving `spindle.stop` before `cut` changes the token passed to `cut`. The cut's required state no longer holds. The dependency graph reveals the invalid transformation even if the textual commands seem independently movable.

2.6.6 Linear use

A state token should not be duplicated because that forks one machine into two contradictory futures. It should normally be consumed exactly once. TypeScript may not enforce linearity, but an IR verifier can check single definition and single consumption.

2.6.7 Multiple tokens

Advanced IRs can split resources into controller, stock, probe environment, and provenance tokens. This enables more parallelism but requires precise alias and commutativity rules. A single process token is a safer starting point for a desktop CAM compiler.

2.7 Passes need semantic contracts

2.7.1 Motivation

A pipeline of well-typed functions can still be wrong. `lowerArcs(path)` may return a `Path`, but that type does not say how closely the polyline follows the arc. `compressModal(program)` may return controller blocks, but not whether they interpret to the same trace.

2.7.2 Definition: certifying pass

Verified compilers such as CompCert compose pass-level semantic preservation theorems [R13]. A **certifying pass** produces an output and a witness. An independent checker establishes a named relation between input and output.

A **transformation pass** changes an artifact. An **analysis pass** computes facts without changing its subject. Both can be proof-producing. A planner is a transformation from intent to candidate toolpath; a travel analysis produces a claim about Machine IR. Keeping this distinction explicit helps incremental compilation: changing analysis policy need not change the program artifact.

```
interface CertifyingPass<I, O, W> {
  readonly id: string;
  readonly version: string;

  transform(input: I, config: PassConfig):
    Result<{ output: O; witness: W }, Diagnostic[]>;

  check(input: I, output: O, witness: W, config: PassConfig):
    CheckResult;
}
```

The checker promises:

$$check(I, O, W) = true \Rightarrow R(I, O).$$

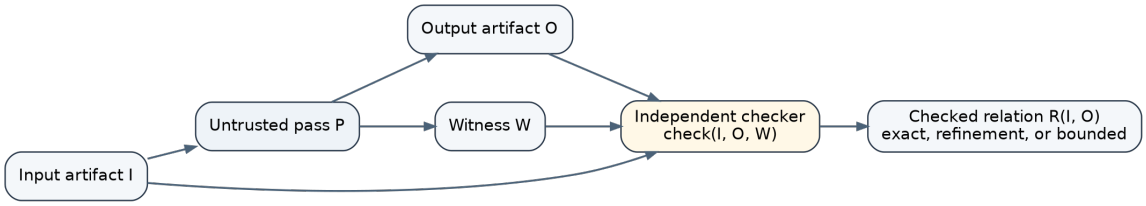

Figure 2.3: A pass proposes an output and witness; a checker establishes the relation.

2.7.3 Five useful pass relations

2.7.3.1 1. Exact semantic preservation

$$Sem(O) = Sem(I).$$

Use for canonicalization, lossless normalization, and modal compression when interpreted traces are identical.

2.7.3.2 2. Trace refinement

$$Traces(O) \subseteq Traces(I).$$

Use when a lower-level pass resolves choices left open above. Lowering an abstract safe traverse to retract–XY–descend selects one permitted route.

2.7.3.3 3. Bounded geometric refinement

$$d_H(Geom(O), Geom(I)) \leq \varepsilon.$$

Use for arc linearization, curve fitting, and surface approximation. The metric and frame must be named.

2.7.3.4 4. Witness satisfaction

$$O \in [\![I]\!].$$

Use when a planner selects one implementation of a high-level intent.

2.7.3.5 5. Feasible optimization

$$Feasible(O) \wedge Sem(O) \approx Sem(I) \wedge J(O) \leq J(I)$$

or, more commonly, 0 is feasible and has a reported objective value. Global optimality requires a lower-bound certificate.

2.7.4 Definition: translation validation

Translation validation checks individual compiler runs rather than trusting the transformer implementation [R14]. **Translation validation** checks each actual input/output pair produced by a compiler pass instead of proving the pass implementation correct for all inputs. This is often practical for geometry and optimization code: let the producer be complex, but independently validate its result.

2.7.5 Worked example: arc linearization

Input: a circular arc of radius R and sweep θ. Output: n chords. A witness contains the subdivision count and maximum angular step $\Delta\theta$.

The sagitta error of one chord is:

$$e = R\left(1 - \cos\frac{\Delta\theta}{2}\right).$$

The checker verifies:

- endpoints match;
- chord order follows the arc orientation;
- every chord endpoint lies on the arc within numeric bounds;
- $e \leq \varepsilon$;
- the output remains in the same frame;
- process metadata and provenance are preserved.

A function that merely samples according to the same formula used by the producer is less independent than a checker that recomputes a conservative bound from the final polyline.

2.7.6 Worked example: modal compression

Canonical Controller IR:

```
SetFeed 450
Linear X15 Y10 Z2
Linear X45 Y10 Z2
Linear X45 Y30 Z2
```

Compressed G-code may emit `F450` only once. The checker parses final bytes, interprets modal state from an explicit initial state, and compares the resulting canonical motion trace. String comparison is not enough; interpreted behavior is the relation.

2.7.7 Counterexample: validating before all lowering

Suppose travel is checked before a postprocessor expands one abstract traverse into three controller moves. The inserted vertical and horizontal segments were not part of the checked artifact. The final bytes need their own validation or a proven pass contract showing that the expansion preserves clearance.

> **Design consequence — no unaccounted motion after certification.** Every pass that inserts, removes, reorders, approximates, rounds, or reinterprets motion must either preserve existing claims by theorem or trigger revalidation of affected properties.

2.8 Provenance, identity, and diagnostics

2.8.1 Motivation

A checker reports that G-code line 813 may gouge the target. The user needs to know which feature, strategy, source line, tool, and pass produced it. Without provenance, validation becomes a disconnected red error

marker.

2.8.2 Definition: provenance

Provenance records the origin and transformation history of an artifact or IR node.

```
interface Provenance {
  sourceSpan?: SourceSpan;
  authoringNode?: NodeId;
  feature?: FeatureId;
  operation?: OperationId;
  strategy?: { id: string; version: string };
  parentArtifacts: readonly Hash[];
  passHistory: readonly PassRecord[];
}
```

A pass that inserts a synthetic retract should record `reason: "safety-retract"` and the operations it connects.

2.8.3 Definition: content address

A **content address** is a cryptographic digest of canonical artifact bytes. It identifies exact content rather than a mutable filename or object reference.

Canonical serialization must define:

- object-key order;
- floating-point and negative-zero representation;
- Unicode normalization;
- absent versus null fields;
- binary endianness;
- schema version;
- hashes of referenced artifacts.

2.8.4 Worked example: planning cache key

A safe cache key for the pocket toolpath includes:

```
hash(
  intentIR,
  targetGeometryHash,
  stockHash,
  toolAssemblyHash,
  strategyIdAndVersion,
  planningTolerance,
  machine-relevant constraints,
  deterministicSeed
)
```

If the tool hash is omitted, a path planned for a 6 mm cutter may be reused for a 3 mm cutter. If tolerance is omitted, evidence for a coarse request may be attached to a stricter one.

2.8.5 Definition: structured diagnostic

A **structured diagnostic** contains a stable code, severity, message, provenance, quantitative detail, and optional counterexample.

```ts
interface Diagnostic {
  code: string;
  severity: "info" | "warning" | "error";
  message: string;
  provenance?: Provenance;
  claim?: ClaimId;
  detail?: Record<string, unknown>;
  counterexample?: ArtifactRef;
  remediation?: string;
}
```

Stable codes let tests and UIs respond without parsing prose.

2.8.6 Counterexample: mutable tool object

A script registers a tool object, the plan stores the reference, and the script later mutates its diameter. A previously computed hash or validation result may now describe a different tool. Boundary objects should be deep-copied into canonical immutable records before hashing.

2.9 API design: convenience without hidden semantics

2.9.1 Motivation

A mathematically principled core can still produce an unpleasant API. Conversely, a fluent API can conceal dangerous ambient state. The goal is layered convenience over explicit semantics.

2.9.2 Three API levels

2.9.2.1 Feature API

```ts
job.roughPocket(feature, {
  tool: T1,
  strategy: strategy.offset({ stepover: ratio(0.4) }),
  allowance: mm(0.2),
});
```

This is the default level. The planner chooses paths.

2.9.2.2 Path API

```ts
job.cut(customPath, {
  tool: T1,
  feed: mmPerMin(350),
  purpose: "finish",
```

```
  target: pocketFloor,
});
```

The user provides geometry but retains process meaning and target references.

2.9.2.3 Canonical machine API

```
program.append(traverse(path, clearancePolicy));
program.append(dwell(seconds(1)));
```

The user accepts more responsibility. Lower layers demand stronger evidence and fewer automatic assumptions.

2.9.3 Strategy plugins

A strategy proposes paths and a witness:

```
interface Strategy<I extends OperationIntent, W> {
  id: string;
  version: string;
  supports(intent: I, context: PlanningContext): SupportResult;
  plan(intent: I, context: PlanningContext): Result<{
    paths: readonly PlannedPath[];
    witness: W;
  }, Diagnostic[]>;
}
```

It should not mint final safety certificates. A separate checker consumes its result.

2.9.4 Geometry-kernel independence

```
interface SurfaceOracle<F extends FrameId> {
  bounds(): Box3<F>;
  heightAtXY(x: Mm, y: Mm): Interval<Mm> | "outside";
  closestPoint(p: Point3<F>): ClosestPointBound<F>;
  raycast(ray: Ray3<F>): readonly HitBound<F>[];
}
```

The key word is **bound**. A kernel API should state whether it returns a nominal approximation, a guaranteed inner result, or a guaranteed outer result.

2.9.5 Escape hatches

Raw controller text may be necessary during reverse engineering. It must be isolated:

```
interface RawControllerBlock {
  text: string;
  dialect: DialectId;
  declaredEffects?: EffectSummary;
```

```
  provenance: Provenance;
}
```

Self-declared effects are assumptions. If an independent parser cannot establish actual effects, affected analyses become unknown and production certification fails closed.

2.10 End-to-end compiler orchestration

A clear orchestration function exposes every stage:

```
function compileProject(project: Project): CompileResult<JobBundle> {
  const authored = evaluateInIsolate(project.source, project.inputs);
  const plan = elaborate(authored.ast, project.context);
  const intent = normalizeManufacturingIntent(plan.value);

  const proposed = strategyRegistry.plan(intent, project.planningContext);
  const checkedToolpaths = checkPlanningWitness(intent, proposed.value);

  const scheduled = schedule(checkedToolpaths, project.schedulePolicy);
  const checkedSchedule = validateSchedule(intent, scheduled);

  const machine = lowerToMachine(checkedSchedule, project.machineProfile);
  const checkedMachine = validateMachineProgram(machine);

  const controller = lowerToController(checkedMachine, project.dialect);
  const bytes = serializeControllerProgram(controller);
  const finalEvidence = validateFinalBytes(controller, bytes);

  return buildJobBundle({
    source: authored.record,
    intent,
    machine,
    controller,
    bytes,
    evidence: finalEvidence,
    assumptions: project.runtimeAssumptions,
  });
}
```

The function contains no claim that one call proves everything. Each checker establishes specific relations. The job bundle composes those claims.

2.11 Chapter synthesis: the running pocket as a compiler trace

The pocket now has a traceable history:

1. JavaScript constructs an inert pocket AST.

2. Elaboration resolves G54, T1, T2, millimetres, feeds, and defaults.
3. Intent normalization creates required-removal and protected regions with tolerances.
4. A pocket strategy proposes nested offsets and an entry witness.
5. A checker validates coverage, allowance, and path structure.
6. A scheduler chooses loop order and links relative to stock states.
7. Machine lowering checks Z1 limits and supported interpolation.
8. Controller lowering creates explicit modal operations.
9. Serialization emits exact bytes.
10. Parse-back validation compares final interpreted motion with Controller IR.
11. Provenance connects every final block to its operation and pass history.
12. Certificate claims bind all artifacts by hash.

The compiler is now structurally capable of carrying meaning. Chapter 3 addresses the algorithms that produce the geometric and scheduled witnesses.

2.11.1 Check your understanding

Take one final G-code line from a job and trace it backward. Which Controller IR operation produced it? Which Machine IR trajectory produced that operation? Which scheduled step inserted the link or cut? Which path and operation intent does it implement? Which source call created the intent? If any link in that chain is unavailable, diagnostics and certificate invalidation will eventually become guesswork.

2.12 Exercises

2.12.1 Concept checks

1. Distinguish authoring AST, Elaborated Plan IR, and Manufacturing Intent IR for one drill operation.
2. Explain why validation evidence should not be represented only by a `ValidatedProgram` brand.
3. State the objects, arrows, identity, and composition of the path category.
4. Explain in one paragraph why approximate endpoint proximity is not equality.
5. Distinguish a monad from its Kleisli category.
6. Explain what an SSA state token proves and what it does not prove.

2.12.2 API and type exercises

7. Design a type-safe API for a rectangular stock and show where runtime validation is still required.
8. Write an indexed command type for `Probe` that returns a measurement and preserves a `ProbeReady` state.
9. Define a legality predicate for Machine IR.
10. Design a `JoinWitness` that handles snapping and records its error contribution.
11. Write a structured diagnostic for a helix entry that degraded to a plunge.
12. Design a raw-command policy for preview mode versus production mode.

2.12.3 Pass-contract exercises

13. State an exact semantic-preservation relation for modal compression.
14. Derive the sagitta formula used to bound arc linearization.
15. Design a witness for path reversal that proves the operation is directionally reversible.
16. Give an example of a pass that is trace-refining but not trace-equivalent.

17. Design a translation validator for coordinate rounding.
18. Explain why final-byte validation needs an explicit initial modal state.

2.12.4 Counterexample construction

19. Construct an async use of `withTool` that violates the apparent lexical scope.
20. Show how a mutable tool object can invalidate a cache entry.
21. Give two passes that are individually plausible but whose composition loses provenance.
22. Construct a postprocessor change that inserts motion after travel validation.

2.12.5 Capstone design

23. Define the complete IR ladder for drilling four holes with an initial probing cycle. Include the value dependency from probing to work-offset installation and the state-token chain through tool selection, spindle, drilling, and shutdown.

Chapter 3

Geometry, Planning, and Optimization

Chapter orientation

The compiler architecture of Chapter 2 deliberately treats planning algorithms as producers rather than unquestioned authorities. This chapter studies what those producers must compute and what a checker must later verify.

CAM geometry is difficult because several problems are intertwined. A cutter has volume, not merely a center point. A mesh is an approximation to a design surface. A finishing tolerance is a statement about continuous geometry, while most algorithms operate on samples. Contour extraction makes discrete topological decisions. Linking depends on material removed by earlier operations. Scheduling and feed selection are constrained optimization problems rather than drawing operations.

We will separate these concerns. First we construct cutter-location geometry. Then we turn fields into paths, paths into safe connected programs, and programs into schedules and time laws. Throughout the chapter, we distinguish a useful numerical approximation from a sound enclosure.

3.0.1 Learning objectives

After this chapter, you should be able to:

- formulate CAM planning as constrained witness search;
- explain cutter-location surfaces for flat and ball end mills;
- describe the role of triangle feature contact and spatial indices in a drop-cutter evaluator;
- explain why grid spacing and sample count are not automatically error bounds;
- implement and critique marching-squares contour extraction;
- derive 2.5D offset-pocket and ball-tool scallop formulas;
- compare raster, waterline, constant-scallop, and hybrid strategies;
- formulate safe linking in configuration space;
- compare height-field, dexel, and voxel stock models;
- use inner and outer approximations in the direction required by a claim;
- formulate operation ordering and feed planning as operations-research problems;
- compose typed numerical and physical error bounds.

Figure 3.1: The major stages of geometric and process planning.

3.1 Planning is constrained witness search

3.1.1 Motivation

A strategy called `pocket`, `raster`, or `constantScallop` can easily be treated as a black box that returns points. That obscures the most important fact: a planner searches for one object satisfying a specification.

3.1.2 Definition: planning problem

A **planning problem** consists of:

- an intent I;
- a context C containing geometry, tools, machine constraints, stock state, and tolerances;
- a feasible set $\mathcal{F}(I, C)$;
- optionally, an objective J.

The planner seeks:

$$x \in \mathcal{F}(I, C),$$

or, for optimization:

$$\min_{x \in \mathcal{F}(I,C)} J(x).$$

The candidate x may include paths, entries, links, orientation choices, operation order, feeds, and time laws.

3.1.3 Hard constraints versus objectives

A target-gouge limit, fixture clearance, axis travel limit, or required precedence is a **hard constraint**. Cycle time, rapid distance, and number of tool changes are **objectives**. Safety must not be encoded merely as a large penalty:

$$J(x) = time(x) + 10^9 \, collisionDepth(x).$$

A sufficiently large time improvement could still make a colliding candidate numerically "better." The correct model excludes colliding candidates from $\mathcal{F}$.

3.1.4 Worked example: planning variables for the pocket

A pocket planner may choose:

- tool T1 or T2;

- stepdown levels $z_1, \dots, z_k$;
- lateral stepover s;
- offset or raster pattern;
- direction of each loop;
- helical, ramp, or plunge entry;
- link paths;
- feed schedule.

Feasibility requires complete coverage within roughing allowance, no protected-material penetration, valid engagement, safe entry and links, and machine-supported motion. An objective may minimize estimated cycle time plus a penalty for retracts.

3.1.5 Definition: witness

A **planning witness** is data that helps an independent checker establish feasibility. For an offset pocket it might contain:

- the inward-offset regions at each depth;
- the path-to-region coverage correspondence;
- selected entry-clearance regions;
- the stock-state version used for each link;
- the declared stepover and allowance;
- provenance from paths to the operation.

The witness is not trusted merely because the strategy produced it. It reduces checker work and improves diagnostics.

3.2 Cutter-location geometry

3.2.1 Motivation

A target surface tells us where the finished material boundary should lie. The tool center or tip cannot simply follow that surface because the tool has a shape. A ball end mill touching a slope has its center displaced along the surface normal. A flat end mill must remain above the highest point under its circular bottom.

The planner therefore needs a surface of legal tool-reference positions.

3.2.2 Definition: cutter-location surface

For a fixed tool orientation and planar XY placement, the **cutter-location surface** gives the lowest legal tool-reference height at every (x, y) position without penetrating protected target geometry.

Write:

$$CL_T(x, y) = \inf\{z \mid T + (x, y, z) \text{ does not penetrate } P\}.$$

For common three-axis cutters and height-like target geometry, this becomes a maximum over possible contacts.

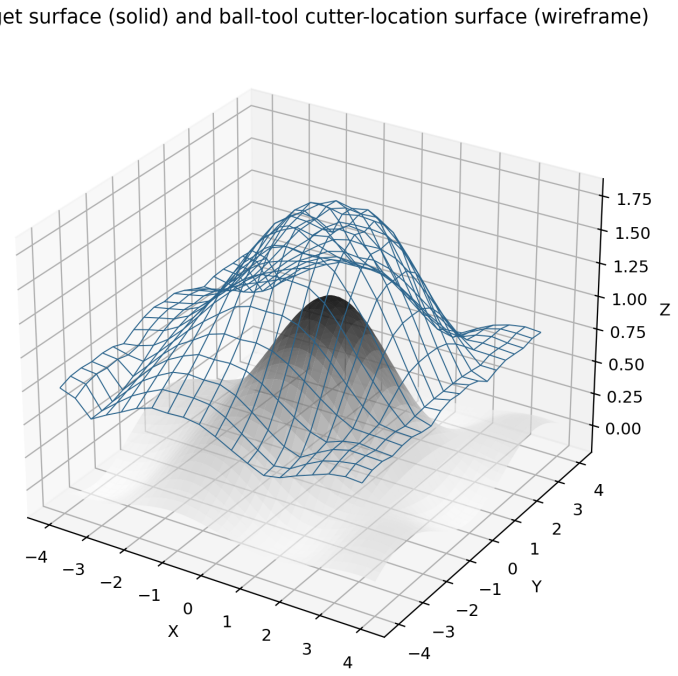

Figure 3.2: A target surface and the corresponding ball-tool cutter-location surface.

3.2.3 Flat end mill

Let a flat end mill have radius R. Its cutting bottom is a horizontal disc. If the target can be described by height $h(u, v)$, the lowest safe tip height at axis position (x, y) is:

$$CL_{flat}(x, y) = \max_{(u-x)^2+(v-y)^2 \le R^2} h(u, v).$$

The tool must clear the highest target point under its disc.

3.2.4 Ball end mill

Let the spherical tip have radius R, and let the tool reference be the lowest point of the ball. A target point $(u, v, h(u, v))$ at horizontal distance:

$$r = \sqrt{(u - x)^2 + (v - y)^2}$$

can contact the ball only when $r \le R$. The sphere center must lie at least:

$$h(u, v) + \sqrt{R^2 - r^2}.$$

The tool-tip height is one radius lower, so:

$$CL_{ball}(x, y) = \max_{r \le R} \left[h(u, v) + \sqrt{R^2 - r^2} - R \right].$$

This is a morphological dilation-like maximum. It explains why a ball tool cannot enter concave details smaller than its radius.

3.2.5 Worked example: ball over a single point

A protected target point lies at height 2 mm, 1 mm horizontally from the tool axis. For a ball radius $R = 3$ mm:

$$CL = 2 + \sqrt{3^2 - 1^2} - 3 = 2 + \sqrt{8} - 3 \approx 1.828 \text{ mm}.$$

The ball tip may lie below the target point because the side of the ball contacts it.

3.2.6 Triangle meshes and feature contact

A mesh triangle can contact a spherical cutter at:

1. a vertex;
2. an edge interior;
3. a face interior.

For the stated mesh and fixed-orientation tool model, an analytic drop-cutter evaluator computes the highest legal contact from all three feature classes and then takes the maximum across candidate triangles. Checking vertices alone misses a large face under the tool. Checking face planes alone misses edge and corner contacts near triangle boundaries.

For a ball tool over a triangle plane:

$$z = Ax + By + C,$$

the sphere center touching the infinite plane lies a normal distance R away. Its center height at axis position (x, y) is:

$$z_c = Ax + By + C + R\sqrt{1 + A^2 + B^2}.$$

The contact is valid only if the projected contact point lies inside the triangle. Edge and vertex formulas handle the remaining cases.

3.2.7 Spatial acceleration

A mesh may contain millions of triangles, but only triangles within the tool's horizontal reach can contribute at one (x, y). A **spatial index** partitions or bounds triangles so the evaluator queries a disc of radius R rather than scanning the entire mesh.

The index can also store an upper height bound. If a node cannot exceed the current best contact height, it is pruned. This is a branch-and-bound pattern: a cheap conservative bound eliminates expensive exact feature tests.

3.2.8 Counterexample: approximate V-bit as a flat disc

Treating a V-bit as a small flat disc may be useful for preview, but it changes the contact geometry. A cone's effective radius grows with depth. The approximation cannot support a claim about V-carved width or no-gouge on sloped walls unless a conservative relation to the true cone is proved. The honest result is "unsupported or approximated under these limitations," not an exact cutter-location surface.

3.2.9 Design consequence

Separate the fast evaluator from its evidence level. A specialized, allocation-free triangle loop may be appropriate in the planner. A certificate checker may use a slower independent enclosure, adaptive subdivision, or a different representation.

3.3 From an evaluator to a sampled field

3.3.1 Motivation

Calling a drop-cutter evaluator everywhere is expensive. Many strategies therefore sample it on a regular grid and interpolate. A field enables contours, distance transforms, and fast preview. The cost is approximation.

A **scalar field** assigns one scalar value to every point in a domain. Here the domain is an XY region and the value is legal tool-tip height. A stored grid is not the field itself; it is a finite representation from which an interpolant or enclosure is constructed.

3.3.2 Definition: sampled cutter-location field

Choose grid spacing g, origin (x_0, y_0), and dimensions n_x, n_y. Store:

$$H_{ij} = CL(x_0 + ig, y_0 + jg).$$

An interpolant $\widetilde{CL}(x, y)$ estimates values between samples. Bilinear interpolation is common.

3.3.3 Worked example: memory and work

A 60 mm by 40 mm stock sampled every 0.1 mm uses approximately:

$$(601)(401) \approx 241{,}000$$

samples. At eight bytes each, one scalar field uses about 1.9 MB. At 0.02 mm, the field exceeds six million samples and about 48 MB before auxiliary arrays. Resolution increases cost quadratically in the XY plane.

3.3.4 Definition: discretization error

Discretization error is the difference between the continuous mathematical object and its finite representation. Grid spacing is an input to an error analysis, not the error itself.

A function is **Lipschitz continuous** with constant L when its output cannot change faster than L times the input distance:

$$|f(p) - f(q)| \le L\|p - q\|.$$

This gives a quantitative bridge from sample spacing to unsampled values. If a function is known to be Lipschitz with constant L: If a function is known to be Lipschitz with constant L:

$$|CL(p) - CL(q)| \le L\|p - q\|,$$

then the distance to the nearest sample gives a conservative value bound. Without such regularity, a narrow spike between samples may be arbitrarily high.

3.3.5 Counterexample: "verified at 0.1 mm resolution"

A target contains a 0.03 mm-wide ridge between grid lines. No sample sees it. A planner follows the interpolated field through the ridge. Stating "verified to 0.1 mm" is unsupported unless the geometry representation or derivative bounds prove that no feature can vary that rapidly.

A sound field can instead store conservative cell bounds:

$$CL^{-}(cell) \le CL(p) \le CL^{+}(cell) \quad \forall p \text{ in cell.}$$

The planner may use nominal values; the checker uses the enclosures.

3.3.6 Adaptive refinement

A practical refinement loop is:

```
function certifyCell(cell, budget):
    bound = conservativeSurfaceBound(cell)
    approximation = localInterpolant(cell)
    error = boundDeviation(bound, approximation)

    if error <= budget:
        accept cell with evidence
    else if cell is still splittable:
        subdivide and recurse
    else:
        return inconclusive
```

A midpoint residual is useful only when a theorem relates it to the maximum interval error. For arbitrary geometry, comparing one midpoint with the endpoint interpolant is a heuristic. The certificate must say so.

3.4 Extracting contours without corrupting topology

3.4.1 Motivation

Waterline finishing, Z-level roughing, and constant-scallop methods often extract level sets from a sampled field. The result should be a collection of closed loops and open curves. Small local topology errors can create

self-intersections, missing loops, or connectors between unrelated regions.

Topology studies connectivity, adjacency, boundaries, holes, and components independently of small metric deformations. In a contour planner, topology answers questions such as "Is this one closed loop or two?" A coordinate error changes shape gradually; a topology error changes which regions the tool will visit.

3.4.2 Definition: level set and contour

For scalar field $f(x, y)$ and level c, the **level set** is:

$$L_c = \{(x, y) \mid f(x, y) = c\}.$$

A contour algorithm approximates L_c with line segments or curves.

3.4.3 Marching squares

Marching squares examines the four signs of $f - c$ at each grid cell. The four bits select one of 16 cases. Crossing points are interpolated along cell edges. Most cases produce one segment.

3.4.4 Saddle ambiguity

Cases with diagonally opposite corners on the same side of the level admit two connections.

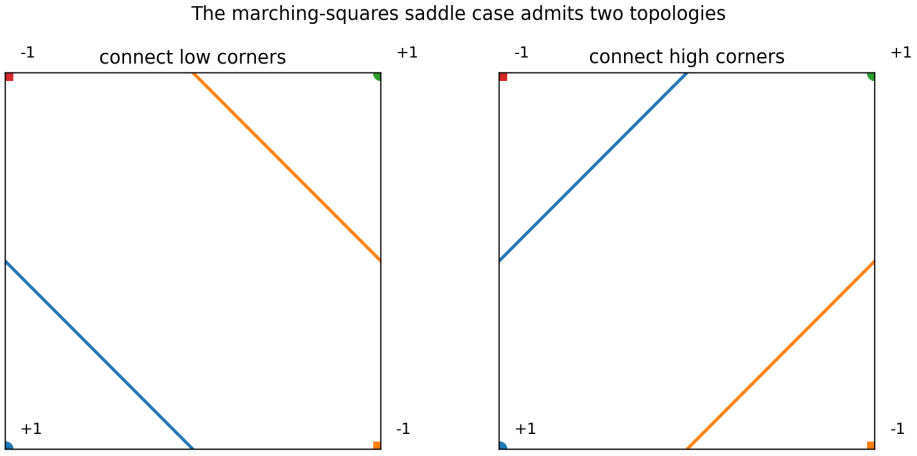

Figure 3.3: The ambiguous saddle case has two possible contour topologies.

A cell-center test is a common pragmatic decider. A more principled **asymptotic decider** uses the bilinear interpolant to determine which branches connect. The chosen rule must be consistent across cells and documented, because it changes topology rather than merely position.

3.4.5 Segment chaining

After local segments are emitted, their endpoints must be joined. A robust implementation distinguishes:

- topological identity: which two endpoints arise from the same grid edge;
- metric proximity: how close two independently constructed coordinates are.

For marching squares, endpoints on a shared grid edge can be assigned an exact symbolic identity such as `(gridEdgeId, levelId)`. This avoids guessing from floating-point coordinates.

3.4.6 Counterexample: modulo-packed endpoint keys

Consider an implementation that quantizes coordinates at 10^{-6} mm and packs them after reducing each integer coordinate modulo $2^{26} = 67{,}108{,}864$:

```
qx = round(x * 1_000_000) % 67_108_864;
qy = round(y * 1_000_000) % 67_108_864;
key = qx * 67_108_864 + qy;
```

Coordinates separated by:

$$\frac{67{,}108{,}864}{1{,}000{,}000} = 67.108864 \text{ mm}$$

produce the same quantized residue. On a machine whose work envelope exceeds that distance, unrelated endpoints can enter the same bucket and contours can merge. The code may look numerically careful because the tolerance is tiny, yet the modulo operation destroys global uniqueness.

The fix is not simply a smaller tolerance. Use a collision-free pair key, nested maps, a string or bigint tuple, or—best for grid contours—exact edge identifiers. If hashing is used, equality must still compare the full coordinates or symbolic IDs.

3.4.7 Counterexample: tolerant chaining as topology

Two separate contours pass within 0.5 micrometres. A tolerance join merges them. The resulting loop may look plausible in a preview but cross a protected island. Topology decisions need robust identities and predicates, not only nearest-neighbor thresholds.

> **Fundamental idea — topology is discontinuous.** A 1-nanometre coordinate error is small. A wrong "connected/not connected" decision can change an entire toolpath. Spend numerical rigor on predicates that control topology.

3.5 Planning a 2.5D pocket

3.5.1 Motivation

A rectangular pocket is conceptually simple, which makes it an excellent place to see how tool geometry, stepdown, stepover, coverage, direction, entry, and linking interact.

3.5.2 Definition: configuration-space offset for a pocket

Let R be the pocket region and let D_r be a disc of tool radius r. A tool center may remain inside the inward offset:

$$R_c = R \ominus D_r.$$

This is a morphological erosion: the set of center positions for which the cutter disc remains inside the pocket boundary.

For sets A and B, the **Minkowski sum** is:

$$A \oplus B = \{a + b \mid a \in A,\ b \in B\}.$$

The **erosion** of A by B is:

$$A \ominus B = \{x \mid x + B \subseteq A\}.$$

The second definition exactly expresses tool-center feasibility: translating the cutter footprint B to center x must keep the complete footprint inside pocket region A. These operations are central to configuration-space planning and mathematical morphology.

For a 30 mm by 20 mm rectangle and a 6 mm diameter cutter, $r = 3$ mm. The boundary centerline rectangle is 24 mm by 14 mm, offset 3 mm from each wall.

3.5.3 Stepover

The **stepover** is lateral distance between adjacent cutting tracks. If specified as fraction ρ of tool diameter D:

$$s = \rho D.$$

With $D = 6$ mm and $\rho = 0.4$:

$$s = 2.4 \text{ mm}.$$

Nested rectangular loops can be generated at inward offsets:

$$3.0, \quad 5.4, \quad 7.8 \text{ mm}, \ldots$$

A coverage checker must verify that the union of cutter sweeps covers the required region. Merely counting loops is insufficient near corners and the center.

3.5.4 Stepdown

The **stepdown** is axial depth per roughing layer. For a 4 mm pocket and 1.5 mm stepdown, one possible level sequence is:

$$z = -1.5, \quad -3.0, \quad -3.8$$

when 0.2 mm axial allowance is left for finishing. The finish floor then cuts to $z = -4.0$.

A planner must define whether `depth` means positive magnitude or signed Z. The Elaborated Plan IR should normalize this convention so later passes do not guess.

3.5.5 Climb and conventional direction

For a rotating cutter, reversing a loop can change climb milling to conventional milling. Therefore a geometric loop may be reversible while a machining action is direction-constrained. The toolpath IR should carry directionality and cutting convention explicitly.

3.5.6 Entry planning

A vertical plunge is often undesirable because the center of many end mills has low surface speed and limited chip evacuation. Common preference order:

1. helical entry;
2. linear or zig-zag ramp;
3. plunge when no safer entry fits and the tool supports it.

For ramp angle α and desired vertical drop d, required horizontal length is:

$$L = \frac{d}{\tan \alpha}.$$

At $d = 1.5$ mm and $\alpha = 3^\circ$:

$$L \approx \frac{1.5}{0.05241} \approx 28.6 \text{ mm}.$$

This is longer than many compact pockets. A zig-zag can distribute the descent across repeated passes.

3.5.7 Counterexample: twelve samples prove helix clearance

Checking a helical orbit at twelve angles can catch many obvious collisions. It does not prove continuous clearance. A narrow obstacle or concave boundary may intrude between samples. The result should be labeled a sampled feasibility heuristic unless a bound on boundary variation connects the angular spacing to continuous separation.

3.5.8 Worked planning pseudocode

```
function planPocket(intent, tool, context):
    centerRegion = erode(intent.region, tool.radius)
    levels = chooseDepthLevels(intent.depth, intent.axialAllowance,
                               intent.stepdown)
    paths = []

    for z in levels:
        loops = inwardOffsets(centerRegion, intent.stepover * tool.diameter)
        ordered = chooseLoopOrderAndDirection(loops, context.cutConvention)
        entry = chooseEntry(ordered.first.start, z, currentStock, tool)
```

```
        paths.append(entry)
        paths.extend(liftLoopsToZ(ordered, z))

    finish = planBoundaryAndFloorFinish(intent, tool)
    return paths + finish, coverageWitness(...)
```

Every helper has an obligation. `erode` must be topologically robust. `chooseDepthLevels` must respect allowance and maximum stepdown. `chooseEntry` must use current stock. `coverageWitness` must establish the required-removal relation.

3.6 Three-dimensional finishing strategies

3.6.1 Motivation

A sculpted surface cannot generally be covered by one planar offset family. Different surface regions favor different path families. The strategy should be understood through the error metric it controls.

3.6.2 Raster finishing

A **raster** strategy lays parallel XY lines and lifts them to the cutter-location surface. It is simple and predictable. Its disadvantages include many reversals, direction-dependent finish, and inefficient coverage of steep walls.

Important parameters are line direction, stepover, clipping region, direction reversal, and adaptive refinement along each lifted line.

3.6.3 Waterline finishing

A **waterline** or constant-Z strategy extracts contours of the cutter-location field at several Z levels. It performs well on steep walls because vertical spacing controls the cusp left between contours. It performs poorly on nearly horizontal regions, where contours become sparse or degenerate.

3.6.4 Ball-tool scallop height

On a locally planar surface machined by parallel passes with a ball radius R, the exact scallop height between two tracks separated by s is:

$$h = R - \sqrt{R^2 - \left(\frac{s}{2}\right)^2}.$$

Solving for spacing:

$$s = 2\sqrt{2Rh - h^2}.$$

For small h:

$$h \approx \frac{s^2}{8R}.$$

3.6.5 Worked example: choose stepover from scallop

A 6 mm ball mill has $R = 3$ mm. Desired planar scallop is $h = 0.01$ mm:

$$s = 2\sqrt{2(3)(0.01) - 0.01^2} \approx 0.4895 \text{ mm}.$$

A fixed 40% diameter stepover would be 2.4 mm and would leave a much larger scallop. This demonstrates why finishing stepover should derive from the surface-error requirement and tool geometry, not a roughing default.

On a curved surface, effective spacing must account for surface curvature, tool contact geometry, and direction. The planar formula is a local approximation, not a universal certificate.

3.6.6 Constant-scallop strategies

A **constant-scallop** strategy attempts to space neighboring paths by approximately equal surface distance under a metric connected to residual cusp height. One approach constructs a distance or arrival-time field and extracts iso-contours.

The Eikonal equation has the form:

$$\|\nabla u(x)\| = \frac{1}{F(x)},$$

where u is arrival time or distance and F is propagation speed. Fast marching computes a monotone numerical solution [R20], and geodesic variants connect the field to surface-distance planning [R21] on a grid. By choosing F from local tool and surface geometry, iso-values of u can approximate desired spacing.

Fast marching proves neither exact scallop height nor topology by itself. A checker still needs to relate field discretization, surface model, tool shape, extracted contours, and path refinement to the final metric.

3.6.7 Hybrid strategies

A **hybrid waterline/raster** strategy may use waterlines on steep regions and raster or constant-scallop paths on shallow regions. It requires:

- a robust slope classifier;
- overlap or blending near the boundary;
- trimming without fragmenting contours incorrectly;
- linking between path families;
- consistent error budgets.

3.6.8 Counterexample: strategy name as a guarantee

A node labeled `constant-scallop` does not establish constant scallop. It records intent or method. The guarantee comes from a proposition, evidence, and checker that quantify the maximum residual surface deviation.

3.7 Linking and free-space motion

3.7.1 Motivation

Cutting paths do not form a complete executable program. The tool must travel between entries, loops, levels, tools, and parking poses. Linking often causes more crashes than the cutting paths because a low move crosses material that a planner assumed was gone.

3.7.2 Definition: free-space link

A **free-space link** is a non-cutting path whose complete tool-assembly sweep is disjoint from current stock and obstacles under the relevant uncertainty:

$$\text{Sweep}^{+}(T_a, \gamma_{link}) \cap (S_i^{+} \cup O^{+} \cup P_{forbidden}^{+}) = \emptyset.$$

The subscript i matters. Link safety is relative to a particular stock state.

3.7.3 Configuration space

A **configuration space** is a space whose points encode complete machine configurations. For a fixed-orientation translating tool, one configuration can be represented by its reference-point position. Collision configurations become an expanded forbidden region.

Configuration-space planning turns moving-body collision into point motion among expanded obstacles [R19]. For fixed tool orientation and pure translation, expand obstacles by the reflected tool assembly: For fixed tool orientation and pure translation, expand obstacles by the reflected tool assembly:

$$O_C = O \oplus (-T_a).$$

The tool can then be represented by a point, and safe links avoid O_C.

3.7.4 Worked example: retract policy

A simple conservative linker uses a certified clearance plane z_c:

```
from current pose:
    raise vertically to z_c
    move XY at z_c
    descend vertically to destination entry height
```

This is safe only if:

- both vertical columns are clear;
- the horizontal sweep at z_c clears stock, fixtures, and holder constraints;
- z_c lies within machine travel;
- controller rapid semantics implement or refine these segments.

A single “safe Z” scalar does not prove these conditions automatically.

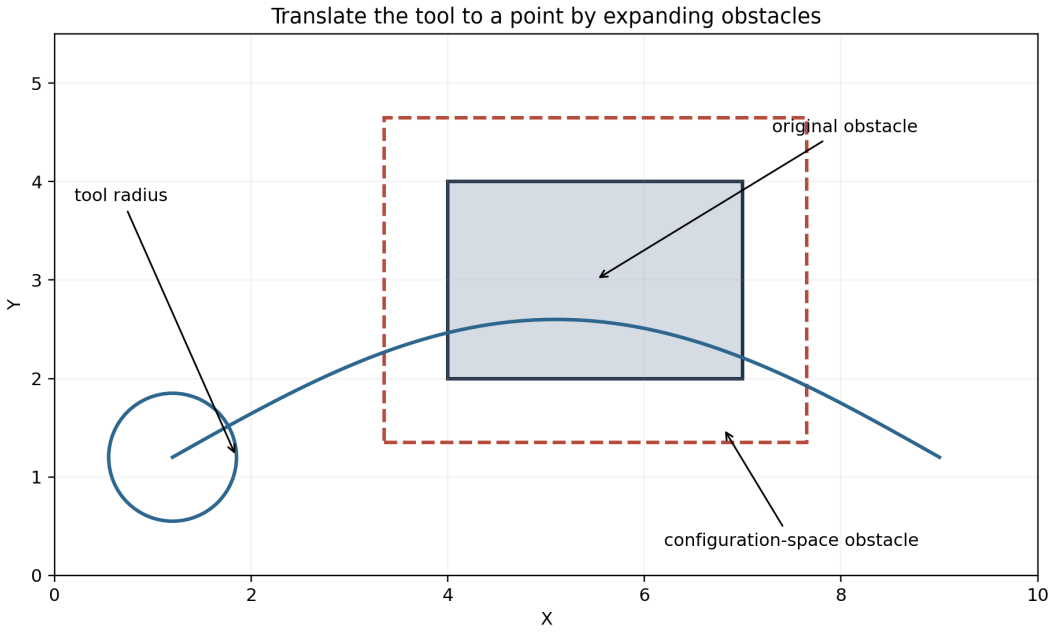

Figure 3.4: Configuration-space expansion turns a finite-radius tool into a point path.

3.7.5 Shortest-path linking

More efficient linkers can search a graph or field in free configuration space:

- visibility graph around polygonal obstacles;
- A* on a conservative grid;
- navigation mesh;
- fast marching on a clearance-weighted cost field;
- stock-aware local retract and stay-down moves.

The objective might be path length or estimated time, but feasibility remains independently checked.

3.7.6 Counterexample: link checked against final stock

A link between two early roughing paths is tested against the final cleared pocket. It passes. During actual execution, a central island of stock still exists and the low link crosses it. Every link must reference the stock state that precedes it, not a globally convenient final simulation.

3.8 Stock representations and swept-volume updates

3.8.1 Motivation

Planning, simulation, and verification need a representation of remaining material. No single representation is best for all jobs.

3.8.2 Height field

A **height field** stores one Z value per XY location. It is efficient for three-axis top-down machining without undercuts. It cannot represent multiple vertical intervals, caves, or arbitrary side entry.

3.8.3 Dexel model

A **dexel** stores one or more material intervals along a ray. A vertical dexel grid can represent multiple layers. A triple-dexel model uses three orthogonal ray families to improve surface reconstruction and collision detail.

3.8.4 Voxel model

A **voxel** model stores occupancy in three dimensions. It is general and simple to update but can be memory-intensive and produces staircase geometry unless refined.

3.8.5 Boundary or exact solid model

A B-rep or exact constructive solid geometry model can represent boundaries precisely, but repeated swept-volume Boolean operations are computationally difficult and numerically delicate.

3.8.6 Inner and outer approximations

Let X be an unknown exact set. An **inner approximation** X^- is guaranteed to lie inside X; an **outer approximation** X^+ is guaranteed to contain X. Maintain:

$$X^- \subseteq X \subseteq X^+.$$

The direction depends on the claim.

> **Fundamental idea — soundness has a direction.** To prove that nothing collides, pretend moving and forbidden objects are at least as large as they may really be. To prove that material was definitely removed, count only the portion certainly swept. Using an approximation in the wrong direction can make a convincing picture and an invalid theorem.

For collision absence:

$$Sweep^+ \cap O^+ = \emptyset \Rightarrow Sweep \cap O = \emptyset.$$

For guaranteed removal:

$$R^- \subseteq R_{true}.$$

For conservative remaining stock, if $S \subseteq S^+$ and $R^- \subseteq R$, then:

$$S' = S \setminus R \subseteq S^+ \setminus R^-.$$

Thus $S'^+ = S^+ \setminus R^-$ is a sound outer bound on residual stock.

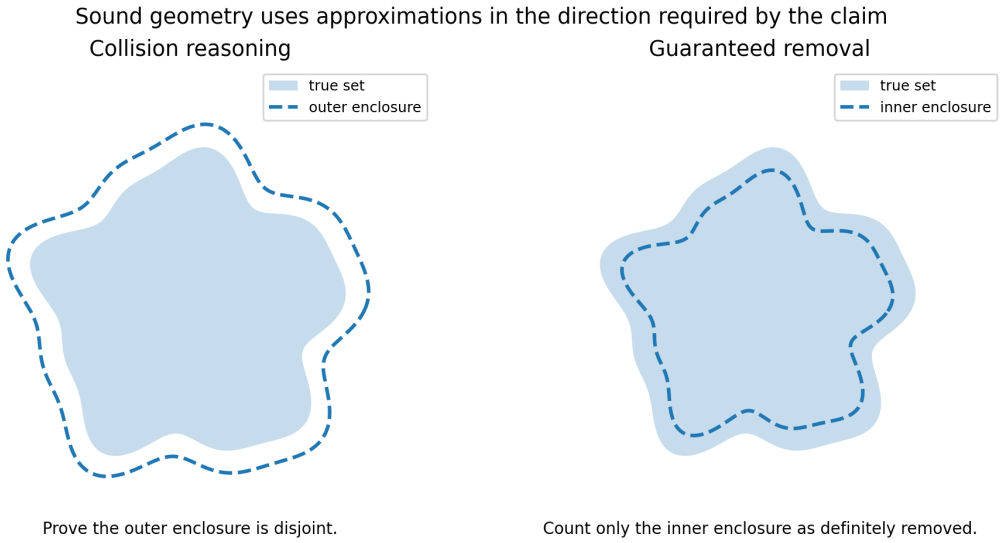

Figure 3.5: Collision proofs use outer enclosures; guaranteed-removal proofs use inner enclosures.

3.8.7 Worked example: no-gouge and coverage need opposite sweeps

To prove no gouge, over-approximate the cutting sweep and show it avoids protected material. To prove required removal, under-approximate the sweep and show the required region lies inside it. One nominal sampled sweep cannot automatically support both claims.

3.8.8 Counterexample: one dexel result promoted to two claims

A simulation sweeps a nominal tool through a dexel stock model and reports material removed. It receives no target part. It can detect sampled rapid-through-stock and spoilboard penetration. It cannot establish "maximum gouge into target" because the target proposition is not even defined in its inputs.

3.9 Robust computational geometry

3.9.1 Motivation

Many geometry algorithms fail not because their distance estimates are slightly wrong, but because a floating-point sign changes a discrete decision. A segment is considered to intersect instead of miss; a polygon orientation flips; two contours connect.

3.9.2 Definition: predicate and construction

A **geometric predicate** returns a discrete result: orientation, sidedness, intersection, containment, or ordering. A **geometric construction** computes coordinates or shapes.

Predicates deserve special numerical treatment because their errors change topology discontinuously.

3.9.3 Orientation predicate

For 2D points a, b, c:

$$orient2d(a, b, c) = (b_x - a_x)(c_y - a_y) - (b_y - a_y)(c_x - a_x).$$

Its sign tells whether c lies left or right of directed line ab. Near collinearity, floating-point cancellation can produce the wrong sign. Adaptive exact predicates are a standard response to floating-point sign failures in computational geometry [R17]. Adaptive exact predicates evaluate quickly in normal cases and increase precision near degeneracy.

3.9.4 Worked example: a nearly collinear turn

Let $a = (0, 0)$, $b = (10^8, 10^8)$, and $c = (2 \cdot 10^8, 2 \cdot 10^8 + 10^{-4})$. The exact orientation is positive but the determinant subtracts two numbers near $2 \cdot 10^{16}$. Ordinary double precision may lose the tiny residual. A filtered predicate first evaluates in hardware precision, estimates its error bound, and falls back to expansion or exact arithmetic only when the sign is uncertain.

3.9.5 Interval arithmetic

Interval analysis supplies outward-rounded enclosures for real-valued computations [R18]. An interval $[a, b]$ represents every real value between its endpoints. Arithmetic uses outward rounding so the exact result remains enclosed.

If:

$$x \in [a, b], \quad y \in [c, d],$$

then:

$$x + y \in [a + c, b + d].$$

Intervals can bound transforms, curve coordinates, distances, and residuals over a parameter region.

3.9.6 Dependency problem

For $x \in [0, 1]$, naive interval arithmetic gives:

$$x - x \in [-1, 1]$$

although the exact value is zero. Subdivision, symbolic simplification, affine arithmetic, or Taylor models can tighten bounds. A wide interval is inconclusive, not incorrect.

3.9.7 Continuous collision by branch and bound

```
function checkInterval(path, parameterInterval I):
    P = outerBoundOfPath(path, I)
    W = outerSweep(P, toolAssembly, uncertainty)
```

```
    if W is provably disjoint from obstacles:
        return safe(I, separationBound)

    if an inner intersection is provable:
        return collision(I, counterexampleRegion)

    if I cannot be subdivided further:
        return inconclusive(I)

    split I and recurse
```

This algorithm covers the continuous domain because every parameter interval is either proved, refuted, or reported unresolved.

3.9.8 Counterexample: exact arithmetic solves physical uncertainty

Exact predicates can determine the topology of the nominal mesh. They do not prove that the mesh matches the clamped workpiece, that the tool diameter is exact, or that the machine follows the commanded trajectory. Numerical robustness and model validity are separate obligations.

3.10 Operation ordering as operations research

3.10.1 Motivation

After paths are generated, the compiler must choose an order. Nearest-neighbor ordering can reduce rapid distance but violate rough-before-finish, probe dependencies, support constraints, or tool grouping.

Operations research is the mathematical study of decision-making under constraints, using optimization, graph algorithms, scheduling, probability, and control. In CAM it addresses not only shortest travel but also tool assignment, working-step order, setup choice, resource limits, and time parameterization.

3.10.2 Definition: precedence graph

Precedence-constrained routing models have been applied directly to CNC toolpath ordering [R22]. A **precedence graph** is a directed acyclic graph $G = (V, E)$ whose vertices are operations and whose edge (a, b) means a must occur before b.

Examples:

- rough pocket before finish pocket;
- probe top surface before frame-dependent cuts;
- machine internal features before releasing surrounding stock;
- drill pilot before using a larger drill;
- finish a fragile wall only after bulk removal strategy is complete.

3.10.3 State-dependent transition cost

Let $c(i, j, S)$ be the cost of moving from operation i to j under stock state S. The cost may include:

- retract and rapid time;
- tool change;
- spindle acceleration;
- probe or accessory transitions;
- risk or engagement constraints.

Because S changes, this is not an ordinary static traveling-salesman problem.

3.10.4 Path orientation

Each open path may have two candidate orientations. A scheduling node can be `(path, orientation, entryChoice)`. Directional process constraints may remove one orientation. The optimizer chooses among the legal states.

3.10.5 Mixed-integer formulation sketch

Let binary variable x_{ij} indicate that operation state j follows i. Minimize:

$$\sum_{i,j} c_{ij} x_{ij}$$

subject to:

- one predecessor and successor per selected operation state;
- subtour elimination;
- one orientation per path;
- precedence constraints;
- tool and setup compatibility;
- stock-dependent feasibility.

For large jobs, exact mixed-integer optimization may be too expensive. Heuristics are acceptable if a checker verifies feasibility and recomputes objective cost.

3.10.6 Counterexample: set subtraction commutes, operations commute

At the ideal stock level:

$$(S \setminus R_1) \setminus R_2 = (S \setminus R_2) \setminus R_1.$$

It does not follow that physical operations commute. Removing R_1 may eliminate support needed during R_2, expose a safe entry, or change tool engagement. Effect summaries must include reads and dependencies, not only removed volume.

```
interface OperationEffects {
  readsStock: RegionSet;
  removesStock: RegionSet;
  readsMeasurements: readonly BindingId[];
  writesMeasurements: readonly BindingId[];
```

```
  requiresTool: ToolId;
  precedenceTags: readonly string[];
}
```

3.11 Feed scheduling and time parameterization

3.11.1 Motivation

A geometric path with feed labels is not yet a physically feasible trajectory. Axis limits and curvature can require slowing down. Controller look-ahead and jerk limits affect following error and cycle time.

3.11.2 Path parameterization

Given path $q(s)$ and progress $s(t)$:

$$\dot{q} = q'(s)\dot{s},$$

$$\ddot{q} = q''(s)\dot{s}^2 + q'(s)\ddot{s}.$$

Axis velocity limits impose:

$$|q_i'(s)\dot{s}| \le v_i^{max}.$$

Acceleration limits impose constraints on $\dot{s}$ and $\ddot{s}$.

3.11.3 Curvature limit

For planar speed v along curvature κ, normal acceleration is:

$$a_n = v^2\kappa.$$

If $a_n \le a_n^{max}$:

$$v \le \sqrt{\frac{a_n^{max}}{\kappa}}.$$

Sharp corners have very high or undefined curvature and require blending, stopping, or controller-specific corner handling.

3.11.4 Worked example: speed on a small arc

A path contains an arc of radius 2 mm, so $\kappa = 0.5\ \text{mm}^{-1}$. If allowed normal acceleration is 500 mm/s²:

$$v \le \sqrt{\frac{500}{0.5}} = \sqrt{1000} \approx 31.6\ \text{mm/s} = 1897\ \text{mm/min}.$$

A commanded feed of 2500 mm/min cannot be maintained through the arc under this simplified constraint.

3.11.5 Time-optimal path parameterization

The optimization problem is:

$$\min T$$

subject to velocity, acceleration, jerk, torque, tracking-error, spindle, and process-force constraints. Reachability-based time-optimal path parameterization provides one principled family of methods [R24]. Reachability-based algorithms propagate feasible velocity intervals along the path. The planner may produce a candidate time law; an independent checker verifies all constraints on the represented intervals.

3.11.6 Process constraints

Maximum machine feed is only one bound. Cutting feed also depends on:

- chip load per tooth;
- spindle RPM and flute count;
- radial and axial engagement;
- tool material and stickout;
- work material;
- machine rigidity;
- desired finish.

Many of these are empirical models or library assumptions. They should be named as such rather than presented as formal geometric guarantees.

3.12 Error budgets and quantitative refinement

3.12.1 Motivation

The final surface differs from the nominal target for many reasons: mesh tessellation, field sampling, path approximation, coordinate rounding, frame uncertainty, tool runout, and servo following. A tolerance is meaningful only if these contributions are connected to a bound.

3.12.2 Definition: error budget

An **error budget** records component bounds, metrics, frames, assumptions, and composition rules used to derive a final quantitative claim.

```
type ErrorBound =
  | { metric: "hausdorff-position"; frame: FrameId; value: Mm }
  | { metric: "normal-surface"; surface: Hash; value: Mm }
  | { metric: "max-gouge-depth"; target: Hash; value: Mm }
  | { metric: "transform-translation"; transform: Hash; value: Mm }
  | { metric: "transform-rotation"; transform: Hash; value: Radians }
  | { metric: "axis-following"; axis: AxisId; value: Mm };
```

Different metrics cannot be added without a conversion theorem.

3.12.3 Pass sensitivity

If pass f has Lipschitz or sensitivity bound L_f and local approximation error ε_f:

$$\varepsilon_{out} \leq L_f \varepsilon_{in} + \varepsilon_f.$$

A simple sum assumes compatible metrics and $L_f \leq 1$ or includes amplification separately.

3.12.4 Worked example: surface budget

Suppose the pocket finish uses:

Source	Bound
target tessellation	0.008 mm
cutter-location approximation	0.012 mm
arc fitting	0.004 mm
G-code rounding	0.001 mm
work-frame translation	0.010 mm
following error	0.010 mm

A conservative additive bound is:

$$0.008 + 0.012 + 0.004 + 0.001 + 0.010 + 0.010 = 0.045 \text{ mm}.$$

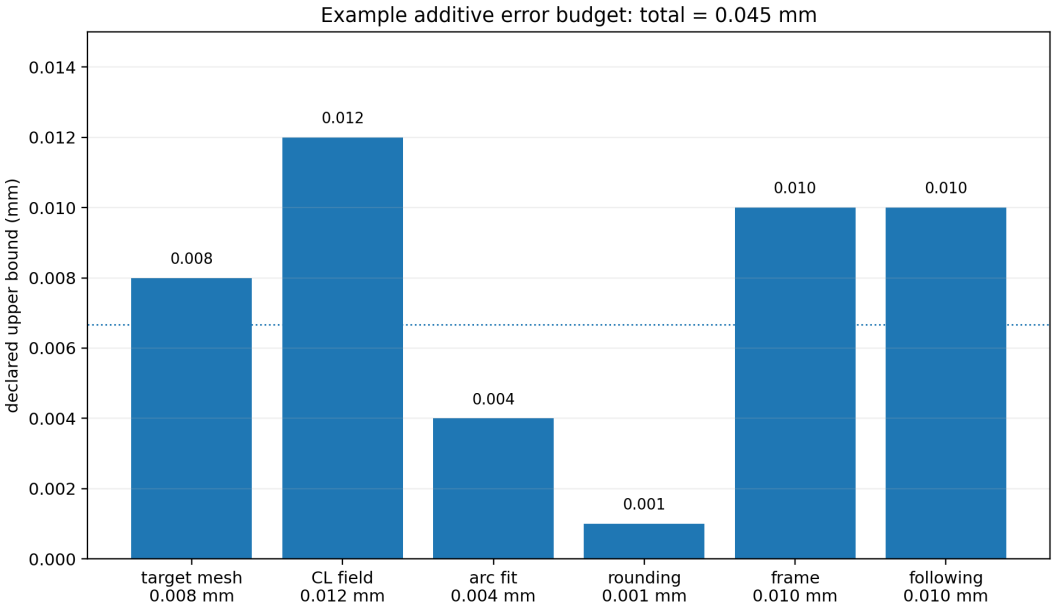

Figure 3.6: An example additive error budget.

If the feature tolerance is 0.05 mm, only 0.005 mm remains as reserve under this simplified compatible-metric model. If angular frame uncertainty contributes another 0.01 mm at the feature radius, the budget fails. Every local pass may meet its default while the composed program misses the feature tolerance.

3.12.5 Worst-case versus statistical composition

Root-sum-square composition assumes an appropriate probabilistic model and independence. It is not a deterministic maximum. A certificate must identify whether a bound is:

- deterministic worst-case;
- probabilistic with a confidence level;
- empirical from calibration;
- nominal or heuristic.

3.12.6 Counterexample: one scalar `totalGeometric`

Adding chord tolerance, simulation cell size, transform rotation, and scallop height into one number erases metrics and propagation. The sum may be useful as a warning dashboard but cannot support a precise theorem until conversions are justified.

3.13 Complete worked planning example

We now plan the running pocket with a 6 mm flat end mill for roughing and the same tool for finishing.

3.13.1 Step 1: normalize intent

- Stock: 60 mm by 40 mm by 8 mm.
- Pocket footprint: $[15, 45] \times [10, 30]$ mm.
- Final floor: $z = -4$ mm.
- Roughing allowance: 0.2 mm radial and axial.
- Final tolerance: 0.05 mm.
- Roughing stepdown: 1.5 mm.
- Roughing stepover: 40% of diameter = 2.4 mm.

3.13.2 Step 2: compute center regions

The roughing wall is left 0.2 mm heavy, so the cutter center stays at least:

$$r + 0.2 = 3.2 \text{ mm}$$

inside the final boundary. The first roughing loop uses a 23.6 mm by 13.6 mm centerline rectangle. Inward loops are spaced 2.4 mm.

3.13.3 Step 3: choose depth levels

Leave 0.2 mm on the floor. Roughing depth is 3.8 mm:

$$-1.5, \quad -3.0, \quad -3.8.$$

3.13.4 Step 4: choose entry

A 3-degree ramp requires about 28.6 mm per 1.5 mm drop, which is possible along the long dimension only with a carefully placed zig-zag. A helix of centerline radius 1.5 mm needs a clear circular region plus tool radius. The planner proposes a helix near the pocket center and supplies a continuous-clearance witness. If the checker cannot prove clearance, it tries a certified ramp or rejects the automatic entry.

3.13.5 Step 5: generate roughing loops

At each depth, generate inward offsets until a coverage checker shows the remaining center region lies within the tool sweep. Preserve climb-milling orientation. Record which center-region band each loop covers.

3.13.6 Step 6: plan links

Within one cleared layer, short stay-down links may be safe. Between depth levels, a descent is cutting motion and must stay inside already cleared XY space. Between roughing and finishing, link safety is checked against the roughing residual stock, not the final pocket.

3.13.7 Step 7: finish

The wall finish follows the final centerline offset 3.0 mm from the pocket boundary at full depth. The floor finish covers the floor at $z = -4.0$ with a small stepover chosen from flat-tool coverage and finish requirements. The finish paths reference the final target and tolerance claims.

3.13.8 Step 8: schedule

Precedence:

```
rough level 1
  -> rough level 2
  -> rough level 3
  -> wall finish
  -> floor finish
  -> retract and spindle stop
```

Alternative wall/floor order can be considered if process effects prove independence. A nearest-neighbor heuristic cannot violate the graph.

3.13.9 Step 9: parameterize

Start with material-library feed assumptions, then limit by machine feed, acceleration, curvature, and entry constraints. Attach empirical assumptions separately from machine-feasibility proofs.

3.13.10 Step 10: emit witness bundle

The planner returns:

- path artifacts;
- offset and coverage correspondences;
- entry and link clearance evidence requests;
- depth and stepover records;
- precedence graph;
- stock-state sequence;
- error contributions;
- provenance.

Chapter 4 explains how these objects become checked claims rather than optimistic metadata.

3.13.11 Check your understanding

For each geometric quantity in the witness bundle, ask three questions: Is it nominal, an inner bound, or an outer bound? In which frame and metric is its error stated? Which later claim consumes it? If those questions have no answer, the quantity is suitable for preview but not yet for certification.

3.14 Exercises

3.14.1 Cutter-location geometry

1. Derive the flat-tool cutter-location formula for a height field.
2. For a ball radius of 4 mm and target point 2 mm from the axis at height 5 mm, compute the legal tip height caused by that point.
3. Explain why vertex-only triangle contact is insufficient.
4. Design a spatial-index node bound for a flat end mill.
5. State what extra geometry is needed to model a bull-nose cutter.

3.14.2 Sampling and topology

6. Compute the number and memory size of field samples for a 100 mm by 80 mm region at 0.2, 0.05, and 0.01 mm spacing.
7. Construct a narrow feature missed by a regular grid and explain what additional bound would make sampling sound.
8. Draw all 16 marching-squares sign cases and identify the two saddle cases.
9. Explain why exact grid-edge IDs are superior to coordinate quantization for chaining.
10. Find two X coordinates in a 150 mm work envelope that collide under a modulo period of 67.108864 mm.

3.14.3 Pocket and finishing calculations

11. For a 4 mm cutter and 35% stepover, compute the lateral spacing.
12. Plan depth levels for a 7.2 mm pocket, 2 mm maximum stepdown, and 0.15 mm floor allowance.
13. Compute ramp length for 2 mm depth at 2.5 degrees.
14. Derive the exact ball-tool scallop-spacing formula from a circle cross-section.
15. Compute spacing for $R = 2$ mm and $h = 0.005$ mm.
16. Explain why a planar scallop formula is not sufficient on a sharply curved surface.

3.14.4 Clearance and stock

17. State separate set predicates for tool gouge, holder collision, rapid-through-stock, and guaranteed removal.
18. Compare height fields, dexels, triple dexels, and voxels for a three-axis pocket and for an undercut part.
19. Design a stock-state identifier scheme for scheduled links.
20. Give a case where a final-stock link check passes but the execution-time link collides.
21. Write pseudocode for an adaptive continuous collision checker.

3.14.5 Operations research and dynamics

22. Construct a precedence DAG for roughing, drilling, probing, and finishing a part.
23. Formulate a small path-orientation scheduling problem with binary variables.
24. Give two operations whose removed volumes are disjoint but whose physical order still matters.
25. Compute curvature-limited speed for radius 5 mm and normal acceleration 300 mm/s².
26. Explain the difference between a feasible schedule, an optimal schedule, and a schedule with a certified optimality gap.

3.14.6 Error reasoning

27. Build a typed error budget for the pocket wall. Identify which bounds are compile-time, calibration-time, and runtime.
28. Give an example where a pass amplifies input error with $L > 1$.
29. Explain why root-sum-square composition does not prove a worst-case tolerance.
30. Design an adaptive refinement policy that returns `inconclusive` rather than silently accepting an unresolved cell.

Chapter 4

Certificates, Validation, and Runtime Assurance

Chapter orientation

A CAM compiler can generate excellent-looking paths and still have no defensible answer to the question, "What exactly was checked?" Testing shows that selected examples behaved as expected. Simulation shows what happened in a model under selected parameters. Neither automatically establishes a universal property of the exact deployed bytes under all relevant executions.

This chapter turns the semantic and geometric material into an assurance architecture. We will define claims precisely, use abstract interpretation to analyze controller state, validate transformations independently, package evidence as a dependency graph, minimize the trusted checker, model the Z1 protocol as a transition system, and bind compile-time results to live machine state.

The chapter ends with a detailed reading of the supplied Dropcut and controller design. The purpose is not to grade the implementation. It is to show how theoretical distinctions reveal both strong design choices and hidden gaps.

4.0.1 Learning objectives

After this chapter, you should be able to:

- distinguish testing, simulation, verification, certification, and attestation;
- write structured claims with explicit subjects, assumptions, evidence, methods, and bounds;
- classify representation, semantic, geometric, compiler, and temporal invariants;
- construct a simple abstract interpreter for modal machine programs;
- explain proof-producing analysis and proof-carrying code;
- design target-gouge, holder-clearance, rapid-clearance, and required-removal claims separately;
- build a certificate dependency DAG and an operating-policy gate;
- validate final controller bytes by independent parsing and interpretation;
- model upload, start, hold, resume, abort, alarm, and ambiguous timeout as protocol states;
- design hash-bound runtime authorization and a small assurance monitor;
- map the architecture onto the current Dropcut/Z1 codebase and prioritize repairs.

4.1 Testing, simulation, proof, and certification

4.1.1 Motivation

A test may run the pocket planner on one rectangle and assert that it returns three loops. A simulation may sweep those loops through a dexel stock and show a cavity. A screenshot may look correct. These are valuable observations, but each supports a different kind of conclusion.

4.1.2 Definition: test

A **test** executes selected cases and compares observed results with expectations. Tests find regressions and counterexamples. Passing tests do not by themselves prove behavior for untested inputs.

Property-based testing expands coverage by generating many cases and checking algebraic laws or invariants. It remains finite empirical evidence unless combined with exhaustive finite enumeration.

4.1.3 Definition: simulation

A **simulation** executes one or more modeled traces. Its conclusion is conditional on the simulator, model, initial state, discretization, and selected inputs. Simulation is excellent for preview, debugging, time estimates, and finding collisions. Absence of a detected collision is not automatically a continuous-domain proof.

4.1.4 Definition: verification

Verification establishes a precise proposition under explicit assumptions by a sound argument or checker. The argument may be exact, bounded, exhaustive over a finite model, or based on a conservative abstraction.

4.1.5 Definition: certificate

A **certificate** is a machine-checkable package that binds:

- a proposition;
- a precise subject artifact;
- assumptions;
- evidence;
- a method and checker identity;
- a result;
- quantitative bounds and coverage when relevant;
- dependencies on other claims.

4.1.6 Definition: attestation

An **attestation** provides authenticated evidence about origin, identity, configuration, or execution. A signature can establish that a machine reported a hash. It does not prove that the path represented by that hash is collision-free.

4.1.7 Worked comparison

Consider four statements:

1. “The preview showed no collision.”

2. “A 0.1 mm dexel simulation found no removed stock during rapids.”
3. “A conservative continuous swept-volume checker proved every rapid disjoint from outer stock and fixture enclosures by at least 0.08 mm.”
4. “The controller signed a report that it stored bytes with hash H.”

Statement 1 is visual inspection. Statement 2 is sampled simulation. Statement 3 is a bounded verification claim. Statement 4 is an attestation. None should be renamed to another category.

4.1.8 Counterexample: `safe: true`

A boolean can hide unexamined dimensions. Did the system check tool-center travel but not holder collision? Did it compare against stock but not target? Was the simulation sampled? Did it validate final bytes or pre-postprocessor IR? A useful interface says “target gouge: not checked; fixture clearance: proved with 0.08 mm minimum; holder: assumption missing,” not `safe: true`.

> **Design consequence — every green mark needs a sentence.** A user-facing status should be generated from a structured proposition and result, not from a generic confidence flag.

4.2 Assertions, assumptions, guarantees, and invariants

4.2.1 Motivation

The words “assertion,” “check,” “invariant,” and “certificate” are often used interchangeably. This makes it impossible to tell whether a condition was measured, assumed, or proved.

4.2.2 Definitions

An **assertion** is a proposition at one point or about one artifact.

A **precondition** must hold before an operation.

A **postcondition** is guaranteed after normal completion if the precondition and assumptions hold.

An **invariant** is initialized and preserved across relevant transitions.

An **assumption** is required but established outside the current proof.

A **guarantee** is the proposition established under the assumptions.

A **witness** is producer-supplied data that helps a checker establish a proposition.

Evidence is the data a named checker actually validates.

4.2.3 Taxonomy of invariants

4.2.3.1 Representation invariants

- all numbers finite;
- array lengths match schema;
- a path endpoint matches its final segment;
- every ID is unique;
- transforms have valid rotation matrices.

4.2.3.2 Path invariants

- segments connect under the declared identity policy;
- arc radii and sweep are coherent;
- parameter domains are valid;
- approximation bounds are attached and composable.

4.2.3.3 Machine-state invariants

- cutting implies a selected tool;
- cutting implies a valid spindle state;
- motion implies homing and a known frame transform;
- tool change implies spindle stopped;
- stop commands remain admissible in every state.

4.2.3.4 Geometric invariants

- stock only decreases under subtractive commands;
- protected material remains outside the cutting sweep;
- fixtures remain unchanged;
- holder and machine structures remain disjoint.

4.2.3.5 Compiler invariants

- provenance is total;
- every artifact has a stable identity;
- unsupported operations do not survive full lowering;
- every approximation carries a metric and bound;
- no raw unparsed controller block appears in a production-certifiable bundle.

4.2.3.6 Temporal invariants

- a running execution hash equals the authorized stored hash;
- a failed or stale preflight cannot authorize later motion;
- an abort acknowledgement is not reported before a terminal stopped or fault state;
- a session with ambiguous command outcome is quarantined.

4.2.4 Worked invariant proof: spindle state around cuts

Let invariant I be:

$$I(\sigma) \equiv \text{NextCommandIsCut}(\sigma) \Rightarrow \text{SpindleRunning}(\sigma).$$

A simplistic proof by scanning the previous command is unsound because branches, raw blocks, or modal state may intervene. A state-token IR or abstract interpreter can establish that every control-flow predecessor of a cut has spindle state `on[rpm interval]`.

At a merge, if one branch has spindle on and another off, the abstract state is `{on,off}` and the cut obligation fails. The analysis refuses rather than guessing.

4.2.5 Counterexample: a comment as an assumption discharge

A tool record contains `diameter: 3.175` and comment "measured." The compiler has not verified when, how, with what uncertainty, or for which physical tool the measurement occurred. Convert the claim into a calibration artifact or leave it as an operator assumption.

4.3 Abstract interpretation of modal machine programs

Abstract interpretation gives a general theory for computing sound approximations of all program behaviors represented by an abstract state [R30].

4.3.1 Motivation

The final G-code interpreter may encounter unknown initial modes, branches, subprograms, probe results, and raw commands. Running one concrete simulation does not cover every state. We need a sound approximation of sets of possible states.

4.3.2 Definition: concrete and abstract domains

Let C be a concrete state domain. An **abstract domain** A represents sets of concrete states. A concretization function:

$$\gamma : A \to \mathcal{P}(C)$$

maps each abstract value to the concrete states it contains.

An abstract transfer function $\widehat{F}$ is sound when:

$$F(\gamma(a)) \subseteq \gamma(\widehat{F}(a)).$$

It may over-approximate. It must not omit a possible concrete successor.

4.3.3 Example abstract state

```
interface AbstractMachineState {
  position: Box3 | "unknown";
  homing: "homed" | "unhomed" | "maybe";
  tool: ToolRef | Set<ToolRef> | "unknown";
  spindle: "off" | RpmInterval | "unknown";
  wcs: TransformInterval | "unknown";
  units: Set<"mm" | "inch">;
  distanceMode: Set<"absolute" | "incremental">;
  motionMode: Set<"rapid" | "linear" | "cwArc" | "ccwArc">;
  feed: Interval<number> | "unknown";
  alarm: "yes" | "no" | "maybe";
}
```

4.3.4 Transfer functions

For `G21`, units become `{mm}`. For `G90`, distance mode becomes `{absolute}`. For an absolute X word x with rounding interval $[x-\rho, x+\rho]$, the abstract X position becomes that interval after applying the current WCS enclosure. In incremental mode, the interval is added to the current X interval.

If distance mode is `{absolute,incremental}`, the successor joins both interpretations. The result is less precise but sound.

4.3.5 Definition: join

A **join** $a \sqcup b$ is an abstract value representing every state represented by either a or b. Intervals join by taking the smallest enclosing interval. Finite sets join by union.

At control-flow merges, join combines branch information.

4.3.6 Worked example: branch around spindle stop

```
if optionalFinish:
    cut finishPath
    M5
else:
    M5
merge:
    program end
```

Both branches establish spindle off, so the join is `off`. If the first branch omitted `M5`, the join would be `{on,off}` and the final-spindle-off claim would fail.

4.3.7 Worked example: abstractly interpreting a short program

Consider:

```
G21 G90
M3 S12000
G1 X15 Y10 Z2 F450
G1 Z-1.5
M5
```

From an initial state with unknown units, distance mode, position, and spindle, the first block establishes millimetres and absolute mode. The second establishes spindle-on with RPM interval $[12000, 12000]$. The first motion sets an exact commanded work-coordinate point, expanded by transform and rounding uncertainty. Before the plunge block, the analyzer can prove units, absolute mode, feed, and spindle state. After `M5`, spindle is definitely off. If the first block omitted `G90`, the position after `X15` would remain a join of absolute and incremental interpretations unless initial mode were an assumption.

4.3.8 Fixed points and loops

A **fixed point** of a function F is a value x satisfying $F(x) = x$. Loop analysis seeks an abstract state stable under another iteration. Starting from the entry state and repeatedly applying transfer and join may produce

an ascending chain. A **widening** operator forces that chain to stabilize by replacing slowly growing detail with a coarser safe bound. It trades precision for guaranteed analysis termination.

A loop requires computing such an invariant. Iteration may not terminate if intervals grow one step at a time. Widening can jump to a coarser bound, possibly `unknown` or unbounded.

For production certification, an easier policy may reject unbounded controller macros and require statically bounded repetition.

4.3.9 Proof-producing abstract interpretation

The analyzer can attach an abstract state before and after every block:

```
interface BlockInvariant {
  blockIndex: number;
  before: AbstractMachineState;
  after: AbstractMachineState;
}
```

A small checker verifies:

1. the initial concrete state set is included in the first abstract state;
2. each block transfer is sound;
3. successor states connect across control flow;
4. each local safety predicate follows from the `before` state;
5. the final state satisfies policy.

The complex analyzer is untrusted. The checker replays a simple proof.

4.3.10 Counterexample: trusting declared raw effects

A raw block declares `effects: ["comment"]` but contains `G0 X100`. Unless a trusted parser validates the block, the declaration is only an assumption from the same producer that wants acceptance. A sound transfer sets every possibly affected component to unknown or rejects the program.

4.4 Translation validation and proof-producing passes

4.4.1 Motivation

Proving a complete TypeScript geometry compiler correct is a major research project. We can obtain substantial assurance sooner by validating each actual transformation result.

4.4.2 Definition: proof-producing pass

A **proof-producing pass** emits output plus evidence designed for a smaller checker. The pass may use heuristics, caches, floating-point acceleration, and external solvers. Its evidence must be sufficient for the checker's theorem.

4.4.3 Worked example: coordinate rounding

Input machine coordinate:

$$x = 12.34542 \text{ mm}.$$

Three-decimal output:

$$\hat{x} = 12.345 \text{ mm}.$$

The checker computes:

$$|x - \hat{x}| = 0.00042 \text{ mm} \leq 0.0005 \text{ mm}.$$

For a sequence, it verifies every coordinate and accumulates or propagates bounds according to the relevant metric. It also normalizes negative zero and rejects non-finite output.

4.4.4 Worked example: path reorder

An optimizer returns a new operation order. The checker verifies:

- each required operation appears exactly once;
- precedence edges are respected;
- chosen orientations are legal;
- every link is certified against the correct stock state;
- tool and frame requirements hold;
- objective cost is recomputed.

The optimizer's search strategy is irrelevant to feasibility.

4.4.5 Worked example: solver optimality gap

An optimizer reports feasible cost $J = 102$ seconds and a valid lower bound $L = 98$ seconds. The checker establishes:

$$98 \leq J^* \leq 102.$$

The relative gap is:

$$\frac{102 - 98}{98} \approx 4.08\%.$$

It is honest to say "feasible schedule within 4.1% of the certified lower bound." Without L, say "feasible schedule with estimated cost 102 seconds," not "optimal."

4.4.6 Composition of pass claims

Suppose pass 1 establishes relation $R_1(A, B)$ and pass 2 establishes $R_2(B, C)$. A composition theorem derives $R(A, C)$:

$$R_1(A, B) \wedge R_2(B, C) \Rightarrow R(A, C).$$

For bounded geometry, composition also propagates error. The certificate checker, not stage-name matching, should apply the theorem.

4.5 Geometric certificate claims

4.5.1 Motivation

"Collision-free" is not one property. The cutter, holder, stock, target, fixture, and machine envelope participate differently. A claim schema should prevent evidence from being applied to the wrong proposition.

4.5.2 Target no-gouge

Let T_c be cutting geometry and $P_{protected}$ target material that must remain. A conservative claim is:

$$\text{Sweep}^{+}(T_c, x) \cap (P_{protected} \ominus B_\delta) = \emptyset.$$

The tolerance layer B_δ permits the declared maximum penetration.

4.5.3 Holder and fixture clearance

Let T_a be the complete assembly and O fixtures and machine obstacles:

$$\text{Sweep}^{+}(T_a, x) \cap O^{+} = \emptyset.$$

This claim requires holder geometry and stickout assumptions. Tool-center travel alone is insufficient.

4.5.4 Rapid-through-stock

For each rapid r_i executed at stock state S_i:

$$\text{Sweep}^{+}(T_a, r_i) \cap (S_i^{+} \cup O^{+}) = \emptyset.$$

A low rapid can be safe after material is cleared and unsafe before. The stock-state dependency belongs in the proposition.

4.5.5 Required removal

Let V_{req} be required-absent material and R^{-} a guaranteed inner removed volume:

$$V_{req} \subseteq R^{-}.$$

No-gouge and required-removal claims use opposite approximation directions.

4.5.6 Machine travel

For trajectory $q(t)$ and admissible machine configuration set Q_{adm}:

$$\forall t, \quad q(t) \in Q_{adm}.$$

For lines and simple arcs, coordinate extrema can often be bounded analytically. Sampling a curve and observing in-range points is not an exact travel proof.

4.5.7 Worked continuous checker result

A branch-and-bound checker divides one arc into parameter intervals. For each interval it computes an outer swept-volume box expanded by tool and transform uncertainty. It proves 31 intervals disjoint from fixture enclosures. One interval remains unresolved at the minimum subdivision width. The correct result is:

```
fixture clearance: inconclusive
unresolved path interval: s in [0.4412, 0.4420]
minimum possible separation interval: [-0.003, 0.012] mm
```

The planner can re-route or request finer geometry. It must not round the interval to positive and declare success.

4.6 Certificate schema and dependency graph

4.6.1 Motivation

A list of statuses does not explain which artifact or assumption each row refers to. Certificates must survive caching, distribution, and independent checking. This requires content binding and explicit dependencies.

4.6.2 Structured claim

```
interface Claim {
  id: ClaimId;
  subject: ArtifactRef;
  proposition: StructuredPredicate;
  result:
    | "proved-exact"
    | "proved-bounded"
    | "translation-validated"
    | "exhaustive-finite-check"
    | "simulation-only"
    | "assumed"
    | "unknown"
    | "refuted";
  method: MethodRef;
  assumptions: readonly AssumptionRef[];
  evidence: readonly EvidenceRef[];
```

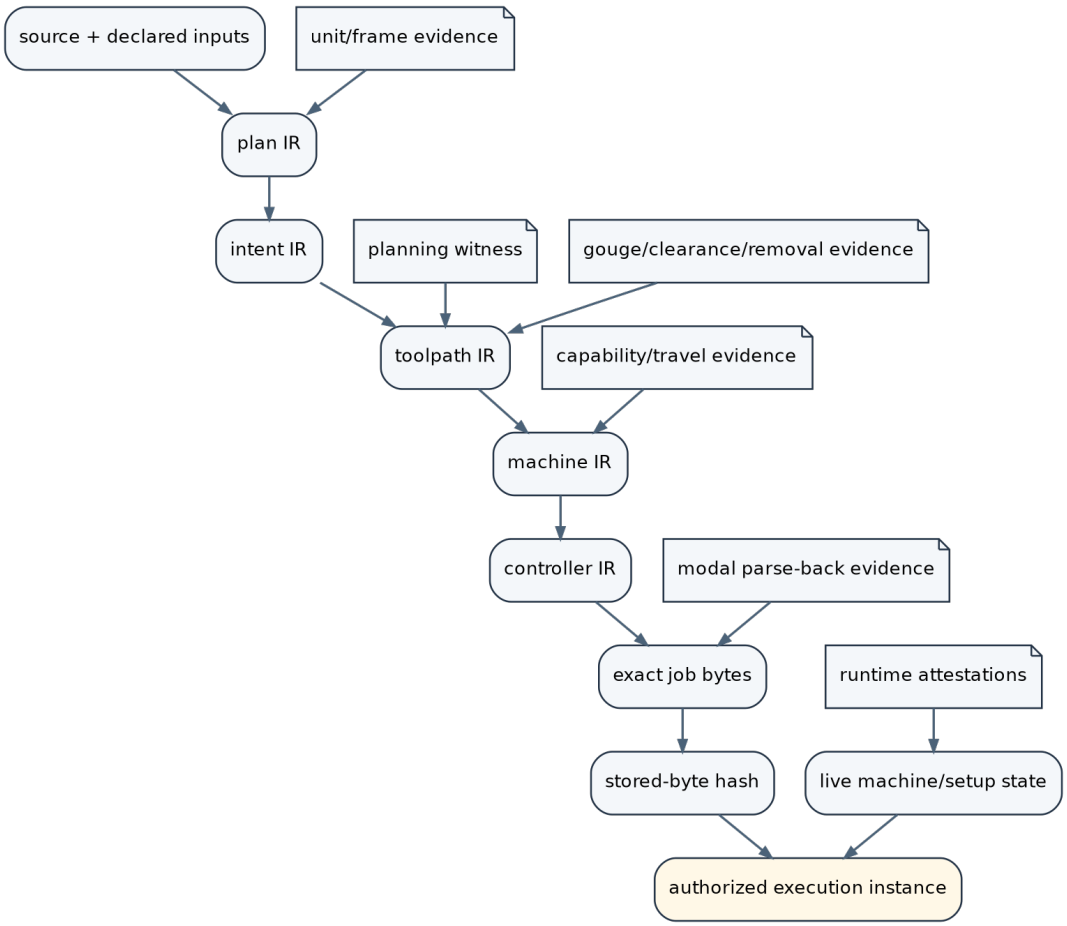

Figure 4.1: A certificate graph connects source, IRs, final bytes, runtime state, and evidence.

```
  dependencies: readonly ClaimId[];
  bound?: ErrorBound;
  checker: CheckerIdentity;
}
```

4.6.3 Worked example: one complete claim

```
const fixtureClaim: Claim = {
  id: "C-fixture-17",
  subject: scheduledProgramRef,
  proposition: {
    kind: "minimum-separation",
    movingAssembly: toolAssemblyRef.hash,
    obstacles: fixtureSetRef.hash,
    stockStateSequence: stockSequenceRef.hash,
    minimum: mm(0.08),
  },
  result: "proved-bounded",
  method: intervalSweepCheckerRef,
  assumptions: [toolMeasurementAssumption, frameBoundAssumption],
  evidence: [subdivisionTreeRef],
  dependencies: [pathGeometryClaim, transformClaim],
  bound: { metric: "minimum-clearance", value: mm(0.08) },
  checker: checkerIdentity,
};
```

Every field answers a student question: what is being claimed, about which bytes or IR, relative to which geometry and stock states, under which assumptions, checked by what implementation, and with what quantitative result?

4.6.4 Example proposition

```
{
  kind: "maximum-penetration",
  sweptArtifact: "sha256:...",
  protectedTarget: "sha256:...",
  metric: "signed-normal-depth",
  maximum: mm(0.02),
}
```

The target hash is mandatory. Evidence without that target cannot satisfy the schema.

4.6.5 Assumption record

```
interface Assumption {
  id: AssumptionId;
```

```
  proposition: StructuredPredicate;
  source: "operator" | "calibration" | "machine-attestation" | "library";
  evidence?: ArtifactRef;
  validFrom?: string;
  validUntil?: string;
  runtimeCheck?: RuntimeCheckSpec;
}
```

Examples include tool diameter interval, fixture identity, firmware semantics profile, and WCS transform set.

4.6.6 Certificate policy

A **policy** defines which claims and result strengths are required for a use class.

Use class	Minimum examples
Preview	schema validity, finite values, parse success
Attended air cut	machine travel, final-byte parse-back, exact upload hash, runtime identity
Attended material cut	plus target, stock, fixture, holder, tool, and WCS claims or explicit assumptions
Unattended production	plus protocol liveness, calibrated dynamics, runtime monitor, recovery and interlock claims

A function named `isFullyVerified` should evaluate a policy, not merely check for absence of errors.

4.6.7 Invalidation

Changing any dependency invalidates downstream claims:

- tool or holder geometry;
- machine profile;
- firmware profile;
- target, stock, or fixture;
- frame transform;
- postprocessor precision;
- final bytes;
- runtime state epoch, meaning the changing version identifier of safety-relevant live state.

Content-addressing makes invalidation mechanical.

4.7 The trusted computing base

4.7.1 Motivation

A CAM application contains too much code to trust wholesale: UI components, editors, mesh loaders, heuristic planners, solvers, spatial indices, caches, visualization, protocol clients, and backends. Assurance improves when the part whose correctness must be trusted is small and explicit.

4.7.2 Definition: trusted computing base

The **trusted computing base**, or TCB, is the code, formal definitions, keys, and assumptions whose failure can invalidate the assurance claim.

A practical TCB may include:

- structured claim semantics;
- canonical serialization and hashing;
- independent parsers and reference interpreters;
- robust interval and predicate kernel used by checkers;
- certificate-DAG validation;
- runtime identity and hash handshake;
- a small command/state monitor;
- accepted physical assumptions.

The following should ideally remain untrusted producers:

- JavaScript script host;
- strategy implementations;
- contour stitching;
- optimizers;
- preview renderer;
- main compiler orchestration;
- UI.

4.7.3 Definition: proof-carrying CAM

By analogy with proof-carrying code, **proof-carrying CAM** means that a complex producer delivers a job plus evidence that a small consumer-side checker validates before execution.

Proof-carrying code introduced the producer/consumer pattern for executable code and safety policies [R15]. The important adaptation here is that CAM evidence spans discrete program state, continuous geometry, quantitative uncertainty, and live physical assumptions.

```
planner / optimizer / postprocessor
              |
              v
       artifacts + evidence
              |
              v
        independent checker
              |
       accept, reject, or inconclusive
```

4.7.4 Checker independence

A checker that calls the same contour-key function as the planner can reproduce the same collision bug. Independence can be increased with:

- separate implementations;
- different representations;
- simpler algorithms;
- strict schemas;
- reduced feature set;
- differential and property testing;
- optional mechanized proofs of core lemmas.

Independence is a spectrum. Even a second algorithm in the same language is better than reusing every helper.

4.7.5 Checker resource safety

Evidence may be malformed or enormous. A checker needs size, recursion, integer, and time limits. It must never execute producer-supplied code. “Proof checking” is not safe if the proof object contains a JavaScript callback.

4.8 Final-byte validation

4.8.1 Motivation

The machine executes serialized bytes, not the pre-postprocessor IR. Formatting, modal compression, arc conventions, line insertion, comments, coordinate rounding, or encoding can change behavior.

4.8.2 Parse-back procedure

```
validateFinalBytes(controllerIR C, bytes B, dialect D, initialModalState M0):
    parsed = independentParse(B, D)
    reject on syntax or unsupported construct

    sourceTrace = interpret(C, M0)
    byteTrace = interpret(parsed, M0)

    compare event order, tool and spindle state,
            path geometry, probe bindings, and final state
    check numeric deviation against output budget
    check required preamble and epilogue
    return equivalence or bounded-refinement evidence
```

4.8.3 Explicit initial state

A controller program should establish required units, plane, distance mode, feed mode, and work offset or declare them as runtime assumptions. Parse-back equivalence depends on the same initial modal state used by the actual controller.

4.8.4 Worked example: modal omission

The postprocessor omits `G90` because its internal default is absolute. The machine retains `G91` from a prior manual operation. The parsed file interpreted under unknown initial state has two possible traces. The final-

byte checker either requires an explicit `G90` preamble or records absolute mode as a runtime assumption that must be checked—if the controller can report it reliably.

4.8.5 Hash exact bytes

Hash the exact encoded byte sequence, including line endings if the controller distinguishes them. A preview generated from Controller IR should display the hash of the corresponding serialized artifact. Upload verification compares the controller's stored content to this hash or to a cryptographically equivalent digest relation.

4.9 Controller protocols as state machines

4.9.1 Motivation

The Z1 client does more than send G-code. It discovers a machine, queries state, uploads files, verifies digests, starts jobs, manages hold and resume, and reacts to alarms. The correctness boundary includes this protocol.

> **Side route — safety and security meet here.** A network endpoint that can start or resume motion is both a machine-safety boundary and a security boundary. Authentication, origin checks, credential handling, and encrypted transport do not prove geometric safety, but without them an unauthorized party may bypass every carefully designed operator workflow.

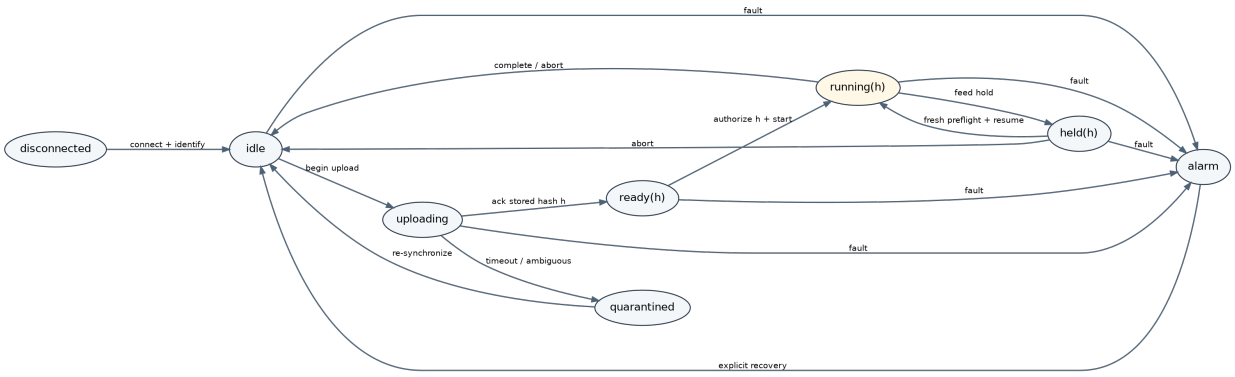

Figure 4.2: A simplified controller lifecycle state machine.

4.9.2 States

A useful model includes:

- disconnected;
- connected but unidentified;
- idle;
- uploading;
- ready with stored hash h;
- running execution h;
- held execution h;
- alarm;
- quarantined after ambiguous outcome.

4.9.3 Risk classes

Classify operations by effect rather than syntax:

1. read-only;
2. stop-only;
3. accessory output;
4. data mutation;
5. state-enabling;
6. motion or unbounded execution.

The ordering supports "maximum risk across a compound request." Stop operations deserve a special rule: a stop that can be refused by a failing motion preflight is not a dependable stop.

4.9.4 Definition: fail-closed classification

A classifier is **fail-closed** when unknown or ambiguous input is assigned a restrictive class or rejected. It must parse the complete payload, not only the first token.

4.9.5 Counterexample: first-token classification

A generic text path receives:

```
status
G0 X100
```

If classification splits whitespace and examines only the first verb, the payload may be marked read-only while the controller interprets two commands. The safe options are:

- prohibit embedded command separators;
- parse the complete supported grammar and classify every block;
- expose only typed routes for mutating operations.

An unknown bare verb should not default to read-only merely because it does not look like `G`, `M`, or `T` code. Firmware command languages often contain motion-enabling textual verbs.

4.9.6 Atomic admission and execution

A preflight reads live state, evaluates conditions, and then an action is sent. If another command can change state between these steps, the authorization is stale.

A **state epoch** is a monotonically increasing identifier changed whenever safety-relevant controller state changes. Authorization records the epoch it checked. If the live epoch differs when execution begins, the authorization is stale and must be recomputed.

A robust sequence is:

```
acquire session authority
read fresh state and state epoch
check command-specific preconditions
bind authorization to command/job hash and epoch
send action atomically with respect to other mutating actions
```

```
observe acknowledgement or enter quarantine
release authority
```

4.9.7 Resume and cycle start

Both resume a possibility of motion. A dedicated `resume` route may perform preflight while a raw realtime cycle-start route only asks for user confirmation. Semantically they belong to the same state-enabling family and should share the required authorization relation, adjusted for their distinct controller states.

4.9.8 Ambiguous timeout and retry

After a mutating command times out, the host cannot assume failure. The client should mark the session ambiguous, avoid automatic retry, and re-synchronize with trusted status framing. Idempotent reads can be retried under different rules.

4.9.9 Durable versus lossy events

UI telemetry may be lossy. Safety-relevant completion, alarm, upload sentinel, and state-transition events should use a durable or backpressured channel. Dropping a protocol boundary can cause the host to assign a reply to the wrong command.

4.10 Runtime assurance and physical assumptions

4.10.1 Motivation

Compile-time evidence is conditional. Before cutting, the system must connect artifact assumptions to the actual machine and setup.

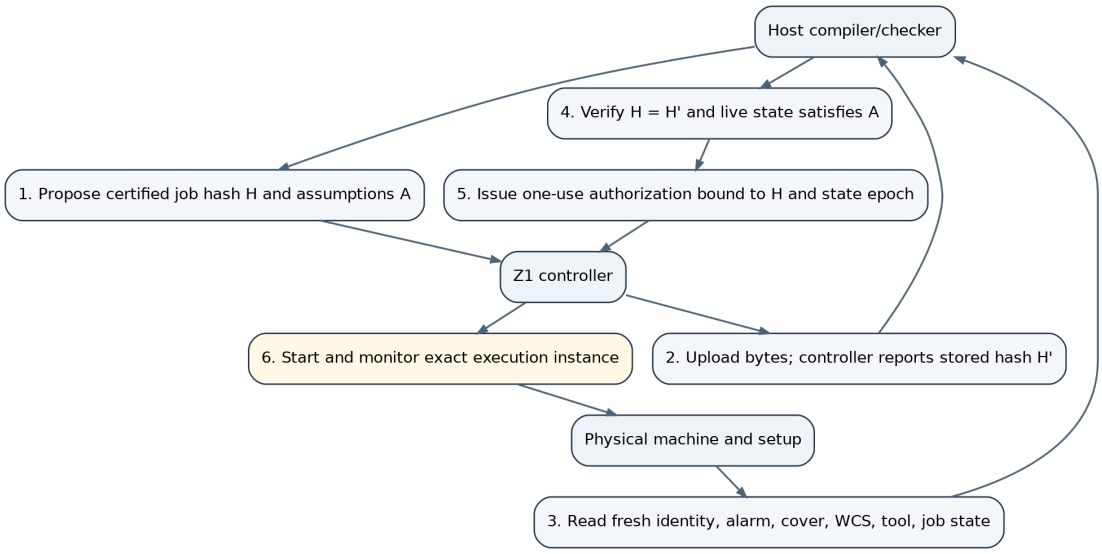

Figure 4.3: A runtime handshake binds exact job bytes to fresh machine state.

4.10.2 Runtime handshake

1. The host proposes certified job hash H and assumption set A.
2. The controller stores the exact bytes and reports stored hash H'.
3. The host reads fresh machine identity, firmware, alarm, cover, homing, WCS, tool, and job state.
4. The host checks $H = H'$ and verifies that live state satisfies A.
5. The host issues a one-use authorization bound to H and the current state epoch.
6. The controller starts that execution instance and reports durable lifecycle events.

4.10.3 Preflight limits

A software preflight can verify reported cover state, alarm state, current job, and some axis or homing information. It may not know:

- whether the workpiece is clamped correctly;
- whether the physical tool matches its ID;
- whether a clamp model matches reality;
- whether stock dimensions are correct;
- whether a sensor is mechanically reliable;
- whether firmware reports homing unambiguously.

The correct design records these gaps. A physical emergency stop and guarded enclosure remain primary safety mechanisms.

4.10.4 Definition: runtime assurance monitor

Runtime-assurance and Simplex-style architectures place a small trusted safety mechanism around a more capable component [R25]. A **runtime assurance monitor** is a small trusted component that observes or intercepts commands and enforces a safety envelope around a more complex controller or planner. A Simplex-style architecture can switch to a trusted safe action when the advanced component approaches an unsafe state.

For a Z1-class system, the monitor may enforce:

- allowed controller-state transitions;
- command risk classes;
- exact job-hash binding;
- axis and feed envelopes;
- spindle/tool consistency;
- watchdog policy;
- communication-loss behavior;
- explicit stop confirmation.

4.10.5 Stopping distance

A stop request is not an instantaneous stop. With speed v and guaranteed deceleration a:

$$d_{stop} \geq \frac{v^2}{2a}.$$

Add distance traveled during detection, network, controller, and actuator latency. A safety envelope must reserve this distance. Runtime assurance therefore connects discrete protocol logic to continuous dynamics.

4.10.6 Degradation policy

When an assumption cannot be established, choose explicitly:

- refuse execution;
- downgrade to preview or air cut;
- require recorded operator override;
- reduce feed and restrict envelope;
- re-probe or re-measure;
- recompile with wider uncertainty;
- quarantine after ambiguous state.

Silent continuation is not an assurance strategy.

4.11 The Dropcut and Z1 implementation as a case study

This section applies the chapter's vocabulary to the supplied snapshot. The observations are about that snapshot, not about every later revision.

4.11.1 Strong foundations

4.11.1.1 Unit and frame types

The units package uses branded `Mm`, `Rpm`, and `MmPerMin` values and converts inches at construction. The frame package tags points and implements rigid transforms, composition, and inverse. These are high-value distinctions because they prevent ordinary mistakes before geometry algorithms run.

The pedagogical caution is that brands and generic frame parameters are not runtime proofs. Deserialization and explicit casts remain trust boundaries.

4.11.1.2 Geometric path IR

The path package distinguishes lines, geometric arcs, and bulk polylines. Arcs carry center, axis, and sweep rather than G-code `I/J/K` offsets and ambient plane. This is exactly the right abstraction for target-independent planning and capability-driven lowering.

The path builder maintains an endpoint representation invariant. Its approximate join tolerance should be treated as a practical check, not exact categorical equality.

4.11.1.3 Non-modal canonical commands

The compiler separates cuts, traverses, spindle actions, tools, and other operations. Keeping feeds and paths explicit makes analysis local. Modal compression belongs in the backend.

4.11.1.4 Capability-driven lowering

The machine profile determines whether arcs are preserved or linearized and whether feeds need adjustment. This is preferable to scattering machine-name conditionals through strategies.

4.11.1.5 Explicit risk classes and fresh preflight

The controller client distinguishes read, stop, accessory, data, state-enabling, and motion classes. It explicitly states that stops are never gated and that preflight reads fresh status rather than UI cache. Those are sound control-design principles.

4.11.1.6 Honest intent in certificate statuses

The analysis layer rejects a single `safe` boolean and distinguishes exact, resolution-bounded, unchecked, and unverifiable statuses. That vocabulary is a strong step toward honest assurance.

4.11.2 Gaps revealed by the theory

4.11.2.1 Script isolation is not an enforced sandbox

The script host uses same-realm `new Function`, shadows obvious globals, and documents the limitation. In a Node or CLI context, the timeout option is not enforced by a separately terminable process. The correct architecture is staged evaluation in a worker or process with memory and time limits, followed by immutable AST transfer.

4.11.2.2 Path construction validates less than comments may suggest

An arc can be appended without proving radius consistency, axis normalization, or endpoint relation. A polysegment does not prove its first point joins the cursor. These are representation and geometry legality checks to add before higher claims.

4.11.2.3 Contour endpoint packing aliases the workspace

The endpoint key reduces quantized coordinates modulo 2^{26}. At 10^{-6} mm quantization, keys repeat every 67.108864 mm. This turns an optimization intended to avoid strings into a topology risk within a desktop mill's work envelope. Exact grid-edge identities or full tuple equality should replace it.

4.11.2.4 Sampled stock evidence is promoted too broadly

The sampled analysis can detect low rapids and spoilboard penetration against a stock model. The certificate construction has no protected target artifact in its inputs, yet a generic sampled status can be used for a gouge field. A claim-specific schema should make that type mismatch impossible.

4.11.2.5 "Exact" travel and interlock labels need proof methods

A validator that scans commands and samples paths may be very useful, but the result is exact only if the continuous path, transforms, and machine envelope are covered analytically or conservatively. Interlock claims also depend on runtime protocol behavior rather than canonical commands alone.

4.11.2.6 Scalar error budgets lose metrics

Summing chord tolerance and output rounding is conservative only under compatible metrics and propagation assumptions. The current scalar is a useful warning indicator; a certificate needs typed bounds and sensitivity rules.

4.11.2.7 Raw effects are producer declarations

A raw block marked with declared effects remains untrusted until independently parsed. The safe abstract transfer is unknown or policy rejection.

4.11.2.8 Generic controller classification is incomplete

The classifier's main branch reasons from the first whitespace-separated verb. Compound payloads can contain later commands. Unknown textual commands can fall through as read-only even though the commentary aims to be fail-closed. A complete grammar or closed typed API is needed for the generic path.

4.11.2.9 State-enabling routes should share authorization semantics

The job-resume path invokes preflight, while a cycle-start route may require confirmation but call the realtime action directly. Both can release motion. They should be bound to fresh state, job identity, and authorization epoch.

4.11.2.10 Final-byte and execution binding remain separate obligations

Post-emission simulation is useful, but final bytes should be independently parsed and compared to Controller IR. Upload verification should bind the stored digest and start action to the same certified artifact and live state snapshot.

4.11.3 A constructive interpretation

These gaps do not imply that every package must be rewritten. They identify semantic boundaries where small changes yield disproportionate assurance:

- replace topology keys;
- isolate script execution;
- make claim inputs type-specific;
- add final-byte parse-back;
- close the generic command grammar;
- unify state-enabling authorization;
- separate nominal simulation from conservative checking.

4.12 A staged implementation roadmap

4.12.1 Stage 1: make claims impossible to overstate

- Replace generic certificate rows with structured propositions.
- Require target artifacts for gouge claims and holder artifacts for holder claims.
- Label ordinary dexel and point sampling `simulation-only` unless continuous coverage is proved.

- Bind every claim to artifact and configuration hashes.
- Treat raw blocks as unknown in production policy.

4.12.2 Stage 2: create reference semantics

Implement small pure interpreters for:

- canonical commands;
- Machine IR;
- Controller IR;
- supported G-code dialect;
- controller lifecycle transitions.

Use them for tests and pass comparison.

4.12.3 Stage 3: validate discrete passes

Begin with comparatively tractable transformations:

- unit and frame elaboration;
- modal compression;
- coordinate rounding;
- serialization and parse-back;
- arc linearization;
- traverse expansion;
- feed clamping;
- preamble and epilogue insertion.

4.12.4 Stage 4: build a geometric checker kernel

- robust predicates;
- interval transforms;
- exact or conservative line/arc bounds;
- tool and holder solids;
- target and fixture artifacts;
- continuous swept-volume subdivision;
- separate inner and outer stock approximations;
- typed error metrics.

Keep strategies untrusted.

4.12.5 Stage 5: formalize the protocol

Specify upload, start, hold, resume, abort, timeout, disconnect, and alarm as a transition system. Model-check finite safety properties. Introduce a state epoch and one-use hash-bound authorization.

4.12.6 Stage 6: mechanize the small core

Good candidates for theorem-prover or proof-assistant work are:

- canonical command semantics;
- modal G-code interpreter;
- state-token well-formedness;
- certificate graph validity;
- claim-composition lemmas;
- risk-class protocol invariants;
- numeric lemmas for lines, arcs, and rounding.

Do not begin by proving every mesh planner correct. Validate their outputs instead.

4.13 Complete worked certificate for the running pocket

We now assemble a plausible certificate graph. This is a specification of the desired architecture, not a claim that the supplied snapshot already produces it.

4.13.1 Artifact chain

```
A0 source and declared inputs
A1 elaborated Plan IR
A2 pocket Intent IR
A3 rough and finish Toolpath IR
A4 Scheduled Program IR
A5 Z1 Machine IR
A6 Makera Controller IR
A7 exact .nc bytes
A8 uploaded/stored byte digest
A9 live execution-state snapshot
```

Every artifact has a schema and hash.

4.13.2 Core assumptions

- T1 diameter is in $[5.990, 6.010]$ mm; holder geometry and stickout match artifact H1.
- Fixture artifact F1 matches the physical clamp setup.
- Work transform lies in interval set $\mathcal{T}$ derived from probing record P1.
- Machine profile Z1-M and firmware semantics profile Z1-F match live identity.
- Following error remains below the calibrated bound under the certified feed schedule.

4.13.3 Planning claims

1. **Intent satisfaction candidate**: A3 paths correspond to all required pocket operations.
2. **Coverage**: inner cutting sweeps cover required roughing and finishing regions within residual allowance.
3. **Target no-gouge**: outer cutting sweeps penetrate protected target by at most 0.02 mm in the named metric.
4. **Entry feasibility**: each helical or ramp entry is continuously clear relative to its stock state.
5. **Link clearance**: every traverse assembly sweep is disjoint from current stock and fixtures by a positive bound.

4.13.4 Scheduling claims

- all precedence edges respected;
- each operation appears exactly once;
- tool and spindle states valid;
- stock-state references form a consistent chain;
- estimated objective cost recomputed;
- directionality constraints respected.

4.13.5 Machine claims

- all configurations lie inside Z1 travel envelope under transform set $\mathcal{T}$;
- feeds, spindle rates, and parameterized dynamics lie inside profile limits;
- unsupported arcs are linearized within the allocated Hausdorff bound;
- final spindle state is off.

4.13.6 Controller and byte claims

- parsing A7 under declared initial state produces a trace that bounded-refines A6;
- preamble establishes units, distance mode, plane, and work offset;
- numeric output error stays within 0.001 mm allocation;
- A8 equals the digest of A7;
- execution authorization names A8 and live state epoch from A9.

4.13.7 Policy result

For an attended material cut, the checker accepts only if all required claims are proved or bounded and every runtime-checkable assumption is discharged. A missing fixture identity yields refusal or explicit downgrade to air cut. A sampled preview may remain attached as diagnostic evidence but does not substitute for the required geometric claim.

4.13.8 Operator presentation

The UI should summarize rather than obscure:

```
JOB HASH: 7d...a2
Machine/profile: matched
Stored bytes: matched
Work transform: inside certified interval
Tool T1: operator-confirmed; measurement record valid
Target gouge: proved <= 0.020 mm
Fixture clearance: proved >= 0.080 mm
Holder clearance: proved >= 0.120 mm
Rapid through stock: proved absent for stock states S0-S17
Required removal: proved within 0.030 mm residual bound
Final spindle state: proved off
Unmodeled physical assumptions: clamp torque, stock material homogeneity
Operating policy: attended material cut - ACCEPTED
```

The last line is a policy conclusion generated from specific claims. It is not an unexplained green shield.

4.14 Chapter synthesis

The compiler is trustworthy not because every algorithm is formally proved, but because the architecture keeps difficult producers separate from small, precise checkers. High-level intent survives long enough to state manufacturing claims. Each pass declares a semantic relation. Geometry uses approximation in the direction required by the proposition. Controller bytes are interpreted, not visually trusted. Runtime authorization binds the certified artifact to fresh machine state.

The key mental model is:

$$\text{intent} + \text{assumptions} + \text{checked refinements} + \text{runtime binding} \Longrightarrow \text{conditional physical guarantee.}$$

Every term in that expression matters. Remove intent and the system cannot define gouge. Remove assumptions and the claim overstates reality. Remove checked refinements and a postprocessor can introduce unaccounted motion. Remove runtime binding and the machine may execute different bytes or a different setup.

4.14.1 Check your understanding

Choose one green status in a hypothetical UI. Rewrite it as a complete sentence naming the subject artifact, proposition, result strength, method, assumptions, checker, and bound. If the sentence becomes awkward or impossible, the underlying certificate field is probably too vague.

4.15 Exercises

4.15.1 Vocabulary and claims

1. Classify each as test, simulation, verification, or attestation: a unit test of arc sampling; a rendered stock preview; an interval proof of travel; a signed controller hash.
2. Write separate structured propositions for tool gouge, holder collision, rapid-through-stock, and required removal.
3. Give three assumptions that cannot be established from a CAM source program.
4. Explain why a claim result and a claim method must be separate fields.
5. Design a policy for an attended air cut.

4.15.2 Invariants and abstract interpretation

6. State initialization and preservation obligations for “running job hash equals authorized hash.”
7. Define an abstract domain for active tool and give its join operation.
8. Write transfer functions for `G20`, `G21`, `G90`, `G91`, and `M5`.
9. Analyze a branch in which only one arm selects T1 before a cut.
10. Explain how a raw command should affect the abstract state.
11. Design a proof object containing block invariants and a checker for it.

4.15.3 Translation validation

12. Design a coordinate-rounding validator for XYZ and feed values.
13. Write a witness schema for arc linearization.
14. Explain how to validate a path-reordering optimizer without trusting its search.
15. Given candidate cost 250 and certified lower bound 235, compute the relative gap.
16. State a composition theorem for two bounded geometry passes.

4.15.4 Certificates and TCB

17. Draw a certificate DAG for a program containing a probe-derived work offset.
18. List the minimum TCB for final-byte modal equivalence.
19. Give a case where a checker shares too much code with the producer.
20. Design resource limits for an interval subdivision proof.
21. Explain why a signature on a `.nc` file does not prove no gouge.

4.15.5 Protocol and runtime

22. Write a finite-state model for idle, uploading, ready, running, held, alarm, and quarantine.
23. State three safety and two liveness properties for that model.
24. Construct a time-of-check/time-of-use race around preflight.
25. Explain why state-enabling commands need authorization even when they contain no axis coordinates.
26. Design a fail-closed grammar for a generic read-only command endpoint.
27. Describe the correct response to an ambiguous start timeout.
28. Compute ideal stopping distance for 20 mm/s and 400 mm/s^2, then add 100 ms of latency distance.

4.15.6 Codebase review

29. Replace the modulo endpoint key with a collision-free topology scheme.
30. Redesign the sampled certificate API so target gouge evidence cannot be constructed without a target artifact.
31. Propose a worker/process protocol for deterministic JavaScript staging.
32. Unify cycle-start and resume under one state-enabling authorization function.
33. Design a final-byte parse-back checker for the Makera dialect subset used by the project.

4.15.7 Capstone

34. Specify a complete certificate-carrying compilation and execution flow for the running pocket. Your answer must include the IR artifacts, pass relations, geometric claims, assumptions, checker identities, final-byte hash, controller state machine, and operator-facing policy result. Identify at least three points where the correct result may be `inconclusive` rather than accepted or refuted.

Appendix A

Mathematical Toolkit for the Main Text

The four chapters use a small collection of mathematical ideas from logic, geometry, analysis, and optimization. They were introduced where the machining problem first required them. This appendix puts them in one place for reference and supplies additional examples. It is not a prerequisite: return here when a symbol or proof pattern feels unfamiliar.

A.1 Sets, predicates, and specifications

A.1.1 Motivation

A manufacturing request usually permits many implementations. We therefore need a way to describe a family of acceptable outcomes without enumerating it.

A.1.2 Definition: set and membership

A **set** is a collection of objects. The notation $x \in A$ means that object x belongs to set A. The notation $A \subseteq B$ means that every member of A also belongs to B.

For the running pocket, let Σ be the set of all possible complete machine-and-workpiece states. The intent denotes a subset:

$$[\![I_{pocket}]\!] \subseteq \Sigma.$$

A final state belongs to this subset exactly when the pocket dimensions, protected material, process constraints, and terminal machine state meet the specification.

A.1.3 Definition: predicate

A **predicate** is a function that returns true or false. A predicate can define a set implicitly:

$$A = \{x \mid P(x)\}.$$

The vertical bar is read “such that.” The compiler does not need to list every acceptable workpiece. It can implement or reason about a predicate such as:

```
function acceptsPocketResult(
  result: FinalState,
  intent: PocketIntent,
): boolean;
```

A.1.4 Worked example: required and protected material

Let R be required-removal material and P protected material. If S_f is final stock, an idealized predicate might require:

$$R \cap S_f = \emptyset \qquad \text{and} \qquad P \subseteq S_f.$$

The first clause says no required-removal material remains. The second says all protected material remains. Real specifications replace exact equality with allowance and tolerance zones.

A.1.5 Counterexample: one point as a specification

A list containing one approved final mesh is not an adequate intent if many tolerance-equivalent meshes are acceptable. It confuses one witness with the set of allowed outcomes.

A.2 Functions, partial functions, and relations

A.2.1 Definition: function

A function $f : A \to B$ associates every input in A with exactly one output in B. A deterministic unit conversion is a function:

$$\text{inchToMm}(x) = 25.4x.$$

A.2.2 Definition: partial function

A **partial function** is undefined for some inputs. Arc construction is partial if the supplied radius and endpoints do not determine a valid arc. In software, represent partiality explicitly:

```
type Result<T, E> =
  | { ok: true; value: T }
  | { ok: false; error: E };
```

Returning `NaN`, an empty path, or a silently repaired arc hides the reason the operation is undefined.

A.2.3 Definition: relation

A relation R between A and B is a set of permitted pairs:

$$R \subseteq A \times B.$$

A command semantics is naturally relational because one starting state may lead to contact, no contact, alarm, or an uncertain measurement. A compiler-pass relation may permit several concrete implementations of the same abstract operation.

A.2.4 Definition: powerset

The **powerset** $\mathcal{P}(A)$ is the set of all subsets of A. A nondeterministic semantics can be written:

$$[\![c]\!] : \Sigma \to \mathcal{P}(\Sigma \times Trace \times Outcome).$$

It maps one input state to a set of possible successors and observations.

A.2.5 Worked example: probing

A downward probe from nominal $z = 5$ mm toward a nominal surface at $z = 0$ mm may contact anywhere in $[-0.01, 0.01]$ mm under sensor and transform uncertainty. The semantics returns a family of measurement outcomes, not one exact scalar. A later work-frame claim must cover that family.

A.3 Logic and proof shape

The symbols used most often are:

Notation	Reading
$\neg P$	not P
$P \wedge Q$	P and Q
$P \vee Q$	P or Q
$P \Rightarrow Q$	if P, then Q
$P \Leftrightarrow Q$	P exactly when Q
$\forall x\, P(x)$	for every x, $P(x)$
$\exists x\, P(x)$	there exists an x for which $P(x)$

A universal safety claim has the form:

$$\forall t \in [0, T], \quad \text{AssemblyPose}(t) \cap O = \emptyset.$$

One collision time refutes it. Testing a thousand sample times can find a counterexample, but no finite sample count establishes the universal statement without a coverage theorem.

A.3.1 Induction for invariants

To prove that invariant I holds throughout a transition system, establish:

1. **Initialization**: $I(\sigma_0)$.
2. **Preservation**: for every transition, $I(\sigma) \wedge T(\sigma, \sigma') \Rightarrow I(\sigma')$.

For stock monotonicity, initialization is immediate. Preservation requires showing that `cut` replaces S by $S \setminus R$ and every non-cutting command leaves S unchanged.

A.4 Metrics and neighborhoods

A.4.1 Definition: metric

A metric $d(x, y)$ measures distance and satisfies:

1. $d(x, y) \geq 0$;
2. $d(x, y) = 0$ exactly when $x = y$;
3. $d(x, y) = d(y, x)$;
4. $d(x, z) \leq d(x, y) + d(y, z)$.

Euclidean distance between points is a metric. Maximum normal penetration and minimum clearance are useful quantities, but they must be defined carefully before being treated as metrics.

A.4.2 Definition: distance to a set

$$d(x, A) = \inf_{a \in A} d(x, a).$$

The symbol inf means the greatest lower bound. For a closed geometric set, this is normally the distance to the nearest point.

A.4.3 Definition: neighborhood

The ε-neighborhood of set A is:

$$N_\varepsilon(A) = \{x \mid d(x, A) \leq \varepsilon\}.$$

A bounded-refinement theorem says the output lies in an allowed neighborhood of the exact specification under a named metric.

A.4.4 Definition: Hausdorff distance

For compact sets A and B:

$$d_H(A, B) = \max\left(\sup_{a \in A} d(a, B), \sup_{b \in B} d(b, A)\right).$$

The first directed term asks how far A strays from B; the second asks whether B contains parts not approached by A. Both are needed. A tiny subset of an arc can lie on the exact arc and have zero one-way deviation while failing to cover almost all of it.

A.4.5 Worked example: arc linearization

For a circular arc of radius r replaced by chords with maximum angular span θ, the maximum sagitta is:

$$e = r\left(1 - \cos\frac{\theta}{2}\right).$$

For $r = 10$ mm and $\theta = 10^\circ$:

$$e \approx 10(1 - \cos 5^\circ) \approx 0.0381 \text{ mm}.$$

A 0.02 mm allocation therefore requires smaller segments.

A.5 Frames, rigid transforms, and uncertainty

A rigid transform from frame A to frame B consists of a rotation $R \in SO(3)$ and translation $t \in \mathbb{R}^3$:

$$p^B = Rp^A + t.$$

Composition is:

$$(R_2, t_2) \circ (R_1, t_1) = (R_2R_1, R_2t_1 + t_2).$$

The inverse is:

$$(R, t)^{-1} = (R^T, -R^Tt).$$

A.5.1 Worked example: angular uncertainty

For small angle $\delta\theta$, a point at distance r from the datum can move by approximately:

$$\delta p \le r\delta\theta.$$

At $r = 80$ mm with $\delta\theta = 0.0003$ rad:

$$\delta p \le 80 \times 0.0003 = 0.024 \text{ mm}.$$

This already consumes almost half of a 0.05 mm feature tolerance before planning or machine-following error is counted.

A.6 Curves, parameterization, and curvature

A geometric path is a function:

$$\gamma : [0, 1] \to \mathbb{R}^3.$$

A monotone time law $s : [0, T] \to [0, 1]$ creates a trajectory:

$$x(t) = \gamma(s(t)).$$

For an arc-length parameter s, curvature is:

$$\kappa(s) = \left\| \frac{d^2\gamma}{ds^2} \right\|.$$

For a circle of radius r, $\kappa = 1/r$. If allowable normal acceleration is a_n^{max}, then:

$$v \leq \sqrt{\frac{a_n^{max}}{\kappa}} = \sqrt{a_n^{max} r}.$$

A path with a sharp corner has unbounded ideal curvature at the corner. The controller must stop, round, or deviate; a timing model that assumes continuous constant feed through the corner is physically false.

A.7 Minkowski sums, erosion, and configuration space

A.7.1 Definition: Minkowski sum

For sets $A, B \subseteq \mathbb{R}^n$:

$$A \oplus B = \{a + b \mid a \in A,\ b \in B\}.$$

A.7.2 Definition: erosion

$$A \ominus B = \{x \mid x + B \subseteq A\}.$$

For a pocket region A and cutter footprint B, $A \ominus B$ is the set of legal cutter-center locations that keep the footprint inside the pocket.

For obstacle O and translating assembly T, the configuration-space obstacle is:

$$O_C = O \oplus (-T).$$

A tool-reference point collides exactly when it enters O_C, under the fixed-orientation rigid model.

A.7.3 Worked example: rectangular pocket

A 30 mm by 20 mm rectangular pocket machined by a 6 mm diameter flat end mill has a center-feasible finishing rectangle inset by 3 mm from every wall. Its nominal dimensions are therefore 24 mm by 14 mm. Corner reachability depends on the desired internal radius; a 3 mm cutter radius cannot produce a smaller internal corner radius.

A.8 Intervals and conservative enclosures

An interval $[a, b]$ represents every real number between a and b. Arithmetic is outward-rounded so the exact result remains enclosed.

If $x \in [a, b]$ and $y \in [c, d]$:

$$x + y \in [a + c, b + d].$$

Multiplication takes the minimum and maximum of the four endpoint products.

A.8.1 Inner and outer geometry

For true geometry X:

$$X^- \subseteq X \subseteq X^+.$$

To prove collision absence, use outer moving and obstacle sets:

$$M^+ \cap O^+ = \emptyset \Rightarrow M \cap O = \emptyset.$$

To prove guaranteed removal, use an inner cutting sweep R^-:

$$R_{required} \subseteq R^- \Rightarrow R_{required} \subseteq R_{true}.$$

A.8.2 Counterexample: the wrong direction

An inner approximation of a holder may miss a protruding collet and falsely report clearance. An outer approximation of removed material may include cells the tool never certainly touched and falsely report coverage.

A.9 Graphs, partial orders, and schedules

A directed graph contains vertices and directed edges. A **precedence graph** has an edge $a \to b$ when operation a must occur before b. A legal schedule is a topological ordering of an acyclic precedence graph.

A partial order need not compare every pair. Roughing must precede finishing, but two independent drilling operations may be incomparable until the optimizer chooses an order.

A.9.1 Worked example

Operations are:

```
P = probe stock top
R = rough pocket
F = finish pocket
I = inspect floor
```

Required edges are:

```
P -> R
R -> F
F -> I
```

There is only one legal order. Add four independent corner holes after probing and before inspection, and many legal orders appear. The optimizer may group tools or reduce travel while respecting the partial order.

A.10 Optimization and optimality claims

A constrained optimization problem has the form:

$$\min_{x} J(x) \quad \text{subject to} \quad x \in \mathcal{F},$$

where $\mathcal{F}$ is the feasible set. Safety properties define feasibility, not weighted penalties.

If a checker establishes candidate cost J and a valid lower bound L:

$$L \le J^* \le J.$$

The relative optimality gap is:

$$\frac{J - L}{L}.$$

A heuristic without a lower bound can produce a useful feasible solution, but “optimal” is not established.

A.11 Deterministic and probabilistic bounds

A deterministic statement is:

$$|x - \hat{x}| \le \varepsilon.$$

A probabilistic statement is:

$$\Pr(|X - \hat{x}| \le \varepsilon) \ge 1 - \alpha.$$

Root-sum-square error combination normally assumes statistical structure such as independence. It does not produce a worst-case bound merely because the arithmetic is smaller than a sum. A certificate must label empirical, probabilistic, nominal, and deterministic quantities distinctly.

A.12 Toolkit exercises

1. Express “every cutting move has spindle on” with a universal quantifier.

2. Give a relation that is not a function using probe outcomes.
3. Compute the Hausdorff distance between point sets $\{0, 2\}$ and $\{0, 1, 2\}$ on the real line.
4. Erode a 20 mm by 10 mm rectangle by a disc of radius 2 mm and state the straight-edge extents of the center region.
5. Explain why a collision proof uses outer enclosures for both moving assembly and obstacle.
6. Construct a precedence graph with two different legal topological orders.
7. Given $J = 125$ and $L = 118$, compute the relative gap.

Appendix B

Reference APIs and Checker Algorithms

This appendix gathers the principal interfaces from the chapters into one coherent reference design. The signatures are deliberately explicit. They are not presented as a drop-in library; they are a specification that an implementation can split into packages and codecs.

B.1 Artifact identity

```ts
type Hash = string & { readonly __brand: "sha256" };
type SchemaId = string & { readonly __brand: "schema" };
type FrameId = string & { readonly __brand: "frame" };
type ClaimId = string & { readonly __brand: "claim" };
type AssumptionId = string & { readonly __brand: "assumption" };

type Mm = number & { readonly __unit: "mm" };
type Radians = number & { readonly __unit: "rad" };
type MmPerMin = number & { readonly __unit: "mm/min" };
type Rpm = number & { readonly __unit: "rpm" };

interface ArtifactRef<T = unknown> {
  readonly hash: Hash;
  readonly schema: SchemaId;
  readonly mediaType: string;
  readonly byteLength: number;
  readonly _type?: T;
}
```

The hash covers canonical bytes, not a mutable JavaScript object. Schema identity is separate because identical bytes interpreted under different schemas may have different meaning.

B.2 Frames and paths

```
interface Point3<F extends FrameId> {
  readonly x: Mm;
  readonly y: Mm;
  readonly z: Mm;
  readonly frame: F;
}

interface Transform<From extends FrameId, To extends FrameId> {
  readonly from: From;
  readonly to: To;
  readonly rotation: readonly [number, number, number, number];
  readonly translation: readonly [Mm, Mm, Mm];
  readonly uncertainty: readonly ErrorBound[];
}

type Segment<F extends FrameId> =
  | { readonly kind: "line"; readonly to: Point3<F> }
  | {
      readonly kind: "arc";
      readonly to: Point3<F>;
      readonly center: Point3<F>;
      readonly axis: readonly [number, number, number];
      readonly sweep: Radians;
    }
  | {
      readonly kind: "polyline";
      readonly points: readonly Point3<F>[];
      readonly approximation: ErrorBound;
    };

interface Path<F extends FrameId> {
  readonly frame: F;
  readonly start: Point3<F>;
  readonly end: Point3<F>;
  readonly segments: readonly Segment<F>[];
  readonly provenance: Provenance;
}
```

B.3 Manufacturing intent

```
interface PocketIntent {
  readonly kind: "pocket";
```

```
  readonly id: string;
  readonly frame: FrameId;
  readonly boundary: ArtifactRef<Region2>;
  readonly top: Mm;
  readonly floor: Mm;
  readonly wallTolerance: Mm;
  readonly floorTolerance: Mm;
  readonly roughingAllowance: {
    readonly radial: Mm;
    readonly axial: Mm;
  };
  readonly requiredRemoval: ArtifactRef<Solid>;
  readonly protectedMaterial: ArtifactRef<Solid>;
  readonly predecessors: readonly string[];
  readonly toolConstraints: readonly ToolConstraint[];
  readonly provenance: Provenance;
}
```

A target-gouge proposition can now require `protectedMaterial.hash`. The type structure prevents the checker from inventing it from unrelated simulation output.

B.4 Scheduled effectful program

```
interface StateToken {
  readonly id: string;
}

interface StockStateRef extends ArtifactRef<StockModel> {}

interface ScheduledStep {
  readonly id: string;
  readonly before: StateToken;
  readonly after: StateToken;
  readonly stockBefore: StockStateRef;
  readonly stockAfter: StockStateRef;
  readonly command: CanonicalCommand;
  readonly dependencies: readonly string[];
  readonly provenance: Provenance;
}

type CanonicalCommand =
  | { readonly kind: "selectTool"; readonly tool: string }
  | { readonly kind: "startSpindle"; readonly rpm: Rpm }
  | { readonly kind: "stopSpindle" }
  | {
```

```
      readonly kind: "cut";
      readonly path: ArtifactRef<Path<FrameId>>;
      readonly tool: string;
      readonly feed: MmPerMin;
      readonly operation: string;
    }
  | {
      readonly kind: "traverse";
      readonly path: ArtifactRef<Path<FrameId>>;
      readonly clearanceClaim: ClaimId;
    }
  | {
      readonly kind: "probe";
      readonly path: ArtifactRef<Path<FrameId>>;
      readonly outputBinding: string;
    };
```

B.5 Pass contract

```
interface PassManifest {
  readonly id: string;
  readonly version: string;
  readonly inputSchema: SchemaId;
  readonly outputSchema: SchemaId;
  readonly relation: RelationId;
  readonly configurationHash: Hash;
}

interface CertifyingPass<I, O, W> {
  readonly manifest: PassManifest;

  transform(
    input: ArtifactRef<I>,
    config: ArtifactRef,
  ): Result<{
    output: ArtifactRef<O>;
    witness: ArtifactRef<W>;
    diagnostics: readonly Diagnostic[];
  }, readonly Diagnostic[]>;
}

interface PassChecker<I, O, W> {
  check(
    input: ArtifactRef<I>,
```

```
    output: ArtifactRef<O>,
    witness: ArtifactRef<W>,
    manifest: PassManifest,
  ): CheckResult;
}
```

The checker is separate from the producer. The pass manifest names the semantic relation rather than merely a stage label.

B.6 Claims, assumptions, and evidence

```
type ClaimResult =
  | "proved-exact"
  | "proved-bounded"
  | "translation-validated"
  | "exhaustive-finite-check"
  | "simulation-only"
  | "assumed"
  | "unknown"
  | "refuted";

interface Claim {
  readonly id: ClaimId;
  readonly subject: ArtifactRef;
  readonly proposition: StructuredPredicate;
  readonly result: ClaimResult;
  readonly method: ArtifactRef<CheckerMethod>;
  readonly assumptions: readonly AssumptionId[];
  readonly evidence: readonly ArtifactRef[];
  readonly dependencies: readonly ClaimId[];
  readonly bound?: ErrorBound;
  readonly checker: CheckerIdentity;
}

interface Assumption {
  readonly id: AssumptionId;
  readonly proposition: StructuredPredicate;
  readonly source:
    | "operator"
    | "measurement"
    | "calibration"
    | "machine-attestation"
    | "library";
  readonly evidence?: ArtifactRef;
  readonly runtimeCheck?: RuntimeCheckSpec;
```

```
}
```

B.7 Typed error bounds

```
type ErrorBound =
  | {
      readonly metric: "hausdorff-position";
      readonly frame: FrameId;
      readonly value: Mm;
      readonly interpretation: "deterministic";
    }
  | {
      readonly metric: "normal-surface";
      readonly surface: Hash;
      readonly value: Mm;
      readonly interpretation: "deterministic" | "empirical";
    }
  | {
      readonly metric: "maximum-gouge-depth";
      readonly target: Hash;
      readonly value: Mm;
      readonly interpretation: "deterministic";
    }
  | {
      readonly metric: "transform-angle";
      readonly transform: Hash;
      readonly value: Radians;
      readonly interpretation: "deterministic" | "probabilistic";
    };
```

A generic `number` field named `totalGeometric` is insufficient because incompatible metrics cannot be added without a conversion theorem.

B.8 Algorithm: certificate-DAG validation

```
checkCertificate(graph, policy):
  validate graph schema and version
  verify every referenced artifact hash
  require unique claim and assumption identifiers
  require every dependency and evidence reference to exist
  require the claim-dependency graph to be acyclic

  for claim in topological order:
    require dependency results accepted by policy
```

```
    choose checker from an allow-listed checker identity
    actual = checker.verify(claim.proposition,
                            claim.subject,
                            claim.evidence,
                            claim.assumptions)
    reject if producer result is stronger than actual
    record actual result and quantitative bound

  for required proposition in policy:
    require a matching accepted claim of sufficient strength

  return accepted summary or structured rejection
```

A producer cannot upgrade `simulation-only` evidence to `proved-bounded`; the checker returns the result strength.

B.9 Algorithm: abstract modal-state checker

```
checkAbstractTrace(program, proof):
  state = abstractInitialState(program.declaredInitialState)

  for each block i:
    require state is included in proof[i].before
    successor = transfer(program.block[i], proof[i].before)
    require successor is included in proof[i].after
    require local safety predicates hold in proof[i].before
    state = proof[i].after

  require terminal policy holds in state
  return accepted or counterexample block
```

The inclusion direction matters. A producer may supply a coarser state, but not one that excludes a possible concrete behavior.

B.10 Algorithm: continuous clearance by subdivision

```
checkClearance(path, assembly, obstacles, interval I, limits):
  pathBox = outerBound(path, I)
  sweepBox = expandForAssemblyAndUncertainty(pathBox, assembly)

  if separated(sweepBox, outer(obstacles)) by delta > 0:
    return proved-safe(I, delta)

  if definitelyIntersects(innerSweep(path, I), inner(obstacles)):
    return refuted(I, counterexampleRegion)
```

```
  if resolutionLimitReached(I, limits):
    return inconclusive(I, sweepBox)

  split I into I1, I2
  return combine(
    checkClearance(path, assembly, obstacles, I1, limits),
    checkClearance(path, assembly, obstacles, I2, limits)
  )
```

The algorithm has three honest outcomes. Reaching the subdivision limit is not success.

B.11 Algorithm: final-byte validation

```
validateFinalBytes(controllerIR, bytes, dialect, initialState):
  parsed = independentParse(bytes, dialect)
  if parsed has syntax or unsupported-operation errors:
    return refuted

  sourceTrace = interpret(controllerIR, initialState)
  byteTrace = interpret(parsed, initialState)

  compare:
    command effects and order
    motion geometry and direction
    tool, spindle, feed, plane, units, and work-offset state
    probe bindings and failure behavior
    final modal and machine state
    numeric deviation under formatting budget

  verify required preamble and epilogue predicates
  return exact-equivalence or bounded-refinement evidence
```

B.12 Algorithm: state-epoch authorization

```
authorize(jobBundle, liveController):
  verify bundle certificate under requested operating policy
  state = liveController.readFreshState()
  storedHash = liveController.readStoredJobHash()

  require storedHash == jobBundle.byteHash
  require state.machineIdentity matches bundle.machineProfile
  require state.firmware matches bundle.controllerSemantics
  require state satisfies every runtime-checkable assumption
```

```
  require state is idle or in an explicitly authorized held state

  token = oneUseAuthorization(
    jobHash = storedHash,
    stateEpoch = state.epoch,
    assumptionSnapshotHash = hash(state.relevantFields),
    expiry = shortDeadline,
  )

  return token
```

The start or resume action consumes the token only if the epoch remains unchanged.

Appendix C

Selected Exercise Solutions

The solutions below are not a complete answer key. They model the style of reasoning expected: name the proposition, expose assumptions, distinguish exact from bounded claims, and identify what remains unproved.

C.1 Chapter 1, Exercise 6: constant-feed duration

The path length is 40 mm and the commanded feed is 1,200 mm/min. Convert feed to mm/s:

$$1200 \div 60 = 20 \text{ mm/s}.$$

The constant-speed duration is therefore:

$$40 \div 20 = 2 \text{ s}.$$

This is a lower-level kinematic estimate, not an actual machine-time theorem. The axis must accelerate and decelerate; corner blending, controller look-ahead, feed overrides, axis limits, and following constraints may reduce speed. A valid time estimate needs a time law consistent with machine dynamics.

C.2 Chapter 1, Exercise 7: frame-angle uncertainty

Using the small-angle bound $\delta p \leq r\delta\theta$:

$$\delta p \leq 80 \text{ mm} \times 0.0003 = 0.024 \text{ mm}.$$

The result is a maximum transverse position contribution under the stated angular interval and radius. It should not be added to unrelated surface or stochastic errors without a compatible metric and propagation rule.

C.3 Chapter 1, Exercise 8: spindle-start contract

One possible Hoare triple is:

$$\{\text{Connected} \wedge \text{NoAlarm} \wedge \text{SelectedTool} = T \wedge 12000 \leq \text{MaxRpm}(T) \wedge 12000 \leq \text{MaxRpm}(M)\}$$

$$\text{startSpindle}(12000)$$

$$\{\text{SpindleCommandedOn} \wedge \text{CommandedRpm} = 12000\}.$$

A stronger postcondition such as "spindle physically rotating at exactly 12,000 RPM" requires feedback, settling time, sensor accuracy, and controller assumptions.

C.4 Chapter 1, Exercise 11: non-transitive proximity

Choose $\varepsilon = 1$, $a = 0$, $b = 0.75$, and $c = 1.5$. Then:

$$|a - b| = 0.75 < 1, \qquad |b - c| = 0.75 < 1,$$

but:

$$|a - c| = 1.5 \geq 1.$$

Therefore "within epsilon" is not an equivalence relation and cannot act as exact endpoint identity.

C.5 Chapter 1, Exercise 12: clear endpoints, colliding sweep

Let a line move a cylindrical holder from $(-10, 0, 0)$ to $(10, 0, 0)$. Put a thin vertical clamp at $x = 0$. Both endpoints are clear, but the segment crosses the clamp. Endpoint tests establish only endpoint clearance. Continuous segment-versus-expanded-obstacle checking is required.

C.6 Chapter 2, Exercise 2: why a brand is not evidence

A TypeScript brand can prove that a value passed through one code path in one process. It does not state which proposition was checked, which artifacts were inputs, which checker ran, what assumptions were used, or whether later transformations invalidated the result. The correct model is an unchanged artifact plus claim objects bound to its hash.

C.7 Chapter 2, Exercise 8: indexed probe

```ts
type Cmd<S0, S1, A> = {
  run(state: S0): Promise<Result<{ state: S1; value: A }, Fault>>;
};

type Probe = Cmd<ProbeReady, ProbeReady, Measurement>;
```

```
function probeToward(
  direction: UnitVec3,
  maximum: Mm,
  feed: MmPerMin,
): Probe;
```

The pre-state requires a selected probe, valid frame, homing, and controller readiness. The result carries contact status and a bounded measurement. “Preserves `ProbeReady`” does not imply that all fault outcomes return that state; the error branch must describe recovery state explicitly.

C.8 Chapter 2, Exercise 13: modal-compression relation

Let P be explicit Controller IR and B the compressed block sequence. With declared initial modal state m_0, the relation is:

$$\operatorname{interpret}(P, m_0) = \operatorname{interpret}(B, m_0).$$

Equality should cover canonical events and terminal modal state. Text equality is irrelevant. Without m_0, omitted words have no unique interpretation.

C.9 Chapter 2, Exercise 14: sagitta derivation

For a circle of radius r and chord subtending angle θ, bisect the isosceles triangle. The distance from center to chord midpoint is $r\cos(\theta/2)$. The radial difference from the arc is:

$$e = r - r\cos\frac{\theta}{2} = r\left(1 - \cos\frac{\theta}{2}\right).$$

Solving for maximum segment angle given error e:

$$\theta \le 2\arccos\left(1 - \frac{e}{r}\right).$$

C.10 Chapter 2, Exercise 19: async scope failure

```
await withTool(T1, async () => {
  await loadGeometry();
  cut(path); // may run after withTool restored the previous tool
});
```

A synchronous `try/finally` wrapper restores context as soon as the promise is returned, not when it resolves. Fix by prohibiting promises, explicitly awaiting the callback inside an async combinator, or eliminating ambient dynamic context during elaboration.

C.11 Chapter 3: flat-end cutter-location height

For a flat cutter of radius r and vertical axis, legal tool-tip height at XY center c is the maximum mesh height under the cutter footprint, adjusted for the tool reference convention:

$$z_{CL}(c) = \max_{q \in \text{surface},\ \|q_{xy} - c\| \le r} q_z.$$

This nominal formula assumes a height-like surface under the footprint. Vertical walls, overhangs, mesh defects, and holder geometry require additional treatment.

C.12 Chapter 3: ball-tool scallop stepover

For a ball radius R and desired planar scallop height h, half-step a satisfies:

$$(R - h)^2 + a^2 = R^2.$$

Therefore:

$$a = \sqrt{2Rh - h^2}, \qquad s = 2\sqrt{2Rh - h^2}.$$

For $R = 3$ mm and $h = 0.01$ mm:

$$s = 2\sqrt{0.06 - 0.0001} \approx 0.4895 \text{ mm}.$$

This is a planar cross-section result. Surface slope, effective radius, path curvature, deflection, and following error require further reduction or proof.

C.13 Chapter 3: modulo endpoint-key collision

If a quantized coordinate is reduced modulo $2^{26} = 67{,}108{,}864$ integer units and one unit is 10^{-6} mm, keys repeat every:

$$67{,}108{,}864 \times 10^{-6} = 67.108864 \text{ mm}.$$

Two unrelated endpoints separated by that amount can receive the same key. The repair is not a larger modulus. Use exact grid-edge/topology IDs, an injective tuple encoding over a proven range, or a map keyed by full integer tuples with equality checks.

C.14 Chapter 3: link checked against wrong stock state

Suppose roughing removes a wall that blocks a low link. A checker using final stock declares the link clear, but the schedule places the link before roughing. The program collides with material that still exists. Every

link claim must reference the stock state immediately before that link, and scheduling validation must ensure the state chain is consistent.

C.15 Chapter 3: curvature speed limit

For radius $r = 2$ mm and allowable normal acceleration $a_n = 400$ mm/s²:

$$v \le \sqrt{a_n r} = \sqrt{800} \approx 28.28 \text{ mm/s}.$$

In mm/min:

$$28.28 \times 60 \approx 1697 \text{ mm/min}.$$

Axis velocity, tangential acceleration, jerk, and contour-error constraints may impose a lower bound.

C.16 Chapter 3: error budget

Assume compatible deterministic position bounds:

```
mesh enclosure        0.008 mm
planning              0.012 mm
arc fitting           0.004 mm
serialization         0.001 mm
frame translation     0.010 mm
frame angular effect  0.010 mm
```

A worst-case additive bound is:

$$0.008 + 0.012 + 0.004 + 0.001 + 0.010 + 0.010 = 0.045 \text{ mm}.$$

A requested 0.04 mm maximum is not certified. The proper response is refinement, better calibration, a looser declared tolerance, or `inconclusive`; not rounding the total down.

C.17 Chapter 4, Exercise 1: classify evidence

- Unit test of arc sampling: **test**.
- Rendered stock preview: **simulation/visualization**.
- Interval proof of travel for every segment and transform enclosure: **verification**, with the exact proposition and assumptions named.
- Signed controller hash: **attestation** of bytes or origin.

The last item says nothing about no-gouge unless a separate verified claim is bound to the same hash.

C.18 Chapter 4, Exercise 8: abstract transfers

A simple modal domain can define:

```
G20: units := {inch}
G21: units := {mm}
G90: distanceMode := {absolute}
G91: distanceMode := {incremental}
M5:  spindle := {off}
```

Unknown raw commands conservatively set every potentially affected component to `unknown`, unless a trusted parser derives narrower effects.

C.19 Chapter 4, Exercise 15: optimality gap

Given $J = 250$ and $L = 235$:

$$\frac{250 - 235}{235} = \frac{15}{235} \approx 0.06383.$$

The candidate is within 6.4% of the certified lower bound. This does not mean it is exactly 6.4% above the true optimum; the optimum may be higher than the lower bound.

C.20 Chapter 4, Exercise 21: signature versus gouge proof

A signature proves that exact bytes were signed by a holder of the corresponding key under the cryptographic assumptions. It does not interpret the bytes, connect them to a target model, account for tool and frame uncertainty, or analyze swept volume. A no-gouge claim needs those semantic inputs and a geometric checker. The signature can bind that claim to the bytes after the claim exists.

C.21 Chapter 4, Exercise 24: preflight race

1. Thread A reads "idle, homed, no alarm" and passes preflight for start.
2. Thread B changes WCS or selects another stored job.
3. Thread A sends start using its stale decision.

A session lock alone is sufficient only if it covers state refresh, precondition evaluation, and command admission. A state epoch and one-use authorization make the dependency explicit and detectable across process or controller boundaries.

C.22 Chapter 4, Exercise 28: stopping distance

With $v = 20$ mm/s and $a = 400$ mm/s²:

$$d_{brake} = \frac{v^2}{2a} = \frac{400}{800} = 0.5 \text{ mm}.$$

At 100 ms latency, the machine travels another:

$$d_{latency} = v\Delta t = 20 \times 0.1 = 2 \text{ mm}.$$

A simple bound is therefore at least 2.5 mm, before controller, servo, and uncertainty margins. “Stop requested” cannot mean “already stationary.”

C.23 Capstone evaluation rubric

A strong capstone answer should include:

- a source and declared-input artifact;
- elaborated intent with units, frames, tools, target, stock, fixtures, and tolerances;
- proposed paths plus planning witnesses;
- a schedule with stock-state and state-token chains;
- Machine IR and Controller IR;
- final-byte parse-back evidence;
- separate no-gouge, holder, fixture, rapid, coverage, travel, process, and terminal-state claims;
- typed quantitative bounds and assumptions;
- controller identity, upload hash, state epoch, and runtime authorization;
- explicit `inconclusive` branches for unresolved geometry, stale assumptions, or protocol ambiguity.

The answer should not replace these objects with one `safe` boolean.

Appendix D

Glossary

Abstraction function. A mapping that forgets lower-level details while retaining observations relevant to a higher-level specification. It is used when comparing controller traces with manufacturing intent.

Abstract domain. A set of values that conservatively represents families of concrete states, such as intervals, bounding boxes, or sets of modal modes.

Abstract interpretation. Sound execution of a program over an abstract domain to cover many or all concrete behaviors at once [R30].

Admissible set. The configurations or candidate programs satisfying all hard constraints.

Artifact. An immutable compiler input or output with canonical serialization, schema identity, and content hash.

Assertion. A proposition intended to hold at one point in a program, proof, or execution.

Assumption. A fact required by a claim but not established by that claim, such as the physical identity of a tool or fixture.

Attestation. Authenticated evidence about origin, identity, configuration, or bytes. It is not automatically semantic proof.

Behavior. An observable execution or outcome. The chosen observation level may include final stock, machine state, events, timing, or faults.

Bounded refinement. Refinement permitting a quantified deviation under an explicit metric.

Canonical action. A controller-independent machining command with explicit physical meaning, forming a semantic waist between planning and target syntax.

Certificate. A machine-checkable package of a proposition, subject artifact, result strength, method, assumptions, evidence, dependencies, quantitative bounds, and checker identity.

Certificate DAG. The acyclic dependency graph connecting artifacts and property-specific claims.

Checker. A comparatively small implementation that validates evidence and returns the strongest result it can establish.

Claim. A precise proposition about a precise artifact or execution instance.

Clearance. Separation between the moving tool assembly and stock, target, fixture, or machine obstacle under a named model and uncertainty set.

Closed command language. A command interface that enumerates allowed operations and rejects unknown syntax instead of inferring that it is harmless.

Compiler pass. A transformation or analysis over an IR, ideally specified by input/output legality and a semantic relation.

Configuration space. A space whose points encode complete configurations. Collision becomes membership in a forbidden region.

Conservative approximation. An approximation biased so that acceptance cannot hide the failure being checked. The required direction depends on the proposition.

Content address. An identity derived from canonical artifact bytes, normally a cryptographic hash.

Controller IR. Structured target-controller operations before final textual serialization.

Controller semantics profile. A versioned description of how a firmware dialect interprets commands, modes, file transfer, acknowledgements, and lifecycle actions.

Counterexample. A concrete state, path interval, payload, or execution showing that a universal claim is false.

Cutter-location surface. The locus of legal tool-reference positions tangent to or clear of a target surface for a given cutter model.

Cyber-physical system. A system in which software interacts with continuous physical state through sensors and actuators.

Denotational semantics. A mapping from syntax or IR to mathematical meaning, often a set of acceptable states or traces.

Dexel. A depth element representing occupied intervals along a ray; useful for stock modeling.

Diagnostic. Structured information about failure, uncertainty, degradation, source provenance, and possible remediation.

Dialect. The syntax and operational behavior of a particular controller language and firmware.

Dimensional type. A type carrying a physical dimension such as length, speed, angle, or spindle rate.

Effect. An observable change or interaction such as machine-state mutation, failure, measurement, stock removal, or controller communication.

Elaboration. Resolution of names, units, frames, defaults, tools, and implicit context into explicit IR.

Enclosure. A set guaranteed to contain a true but uncertain value or geometry.

Error budget. A structured accounting of quantitative approximation and uncertainty contributions under explicit propagation rules.

Evidence. Data checked to establish a claim, such as an interval subdivision tree, abstract invariant, parse-back trace, separating witness, or solver certificate.

Fail closed. Reject or classify as unsafe when meaning is unknown, rather than assuming an unknown command or state is harmless.

Feasible set. The set of solutions satisfying every hard constraint before an objective is optimized.

Final-byte validation. Independent parsing and interpretation of the exact serialized bytes that will be transferred to the controller.

Fixed point. A value x for which $F(x) = x$. Static analysis computes fixed points to represent loop invariants.

Frame. A named coordinate system. A coordinate without a frame lacks complete geometric meaning.

Groupoid. A category in which every arrow has an inverse. Rigid coordinate frames and invertible transforms form a useful example.

Gouge. Removal of protected target material beyond the allowed tolerance or allowance region.

Hausdorff distance. The maximum nearest-set deviation in both directions between two compact point sets.

Hoare triple. A contract $\{P\}\, c\, \{Q\}$ relating a command's precondition and postcondition.

Inner approximation. A set guaranteed to lie inside the true set. It is useful for guaranteed-removal claims.

Intent IR. An intermediate language of features, tolerances, protected and required regions, resources, and precedence before exact toolpaths are chosen.

Invariant. A property true initially and preserved by every relevant transition.

IR legality. The well-formedness, typing, capability, and semantic conditions required at one representation level.

Job bundle. Exact controller bytes plus all profiles, geometry, assumptions, evidence, and hashes required to interpret and authorize them.

Kleisli composition. Composition of effectful functions using a monad or indexed computation abstraction.

Legality predicate. A check that an IR contains only valid operations and fully resolved types, frames, units, and capabilities for its level.

Lipschitz bound. A bound L on how much output distance can grow relative to input distance.

Liveness property. A temporal requirement that a desired event eventually occurs, such as an abort eventually reaching a terminal state.

Lowering. A refinement from an abstract IR to a more concrete one by choosing geometry, machine, controller, or serialization detail.

Machine IR. Machine-specific trajectories and actions after capability, frame, and limit resolution but before controller serialization.

Machine profile. A content-addressed description of kinematics, limits, speeds, dynamics, spindle, tool, probe, and controller capabilities.

Macro language. A compile-time language used to construct an inert program artifact.

Metric. A distance function satisfying non-negativity, identity, symmetry, and the triangle inequality.

Modal state. Controller state retained across blocks, such as units, distance mode, plane, feed, spindle speed, and work offset.

Monad. An abstraction with `pure` and `bind` for composing computations with effects under identity and associativity laws.

Operations research. Mathematical optimization and decision analysis under constraints, including scheduling, routing, resource allocation, and optimal control.

Operational semantics. Rules describing program execution as state transitions.

Outer approximation. A set guaranteed to contain the true set. It is useful for collision and no-gouge checks.

Parse-back validation. Parsing serialized output into structured operations and comparing interpreted behavior with pre-serialization IR.

Partial correctness. A guarantee that the postcondition holds when a command terminates; it does not establish termination.

Path. A geometric curve independent of time and process classification.

Path parameterization. A monotone map from time to progress along a path, determining velocity and acceleration.

Planner. A producer that searches for a path, schedule, or process witness satisfying intent.

Postcondition. A proposition guaranteed after successful execution from a state satisfying the precondition.

Postprocessor. The target-specific compiler backend that lowers machine or controller IR to exact controller bytes.

Precondition. A proposition that must hold before an operation is valid or a theorem applies.

Predicate. A true-or-false function, often used to define a set of acceptable states.

Proof obligation. A proposition that must be discharged before an artifact or action is accepted.

Proof-producing pass. A pass that emits a witness intended for an independent checker.

Provenance. Structured links from generated artifacts and diagnostics to source, passes, parameters, tools, and parent artifacts.

Rapid safety. A claim that a non-cutting motion's complete assembly sweep avoids the stock and obstacles present at that step.

Raw command. Target syntax whose effects have not been derived by a trusted parser; it normally makes affected analyses unknown.

Refinement. Addition of implementation detail or removal of choices without introducing behavior forbidden by the abstract specification.

Relation. A set of allowed input/output pairs, more general than a deterministic function.

Representation invariant. Internal coherence of a data structure, such as path endpoint consistency; not necessarily a physical-safety property.

Residual stock. Material remaining after modeled cutting. Conservative residual stock uses outer stock and inner removal approximations.

Robust predicate. A geometric decision procedure whose sign or Boolean result remains mathematically correct near floating-point degeneracy.

Runtime assurance. A small trusted monitor and policy that constrains execution of a more complex component.

Safety property. A temporal property stating that a bad event never occurs.

Scalar field. A function assigning one scalar to every point in a domain, such as legal tool-tip height over XY.

Semantic preservation. Equality, refinement, or bounded correspondence between input and output meanings.

Semantic waist. A compact intermediate language separating high-level producers from low-level targets while retaining essential meaning.

Simulation. Execution of a model for selected inputs or traces. Simulation can find defects but needs a coverage theorem to establish universal properties.

Soundness. The property that everything an analysis reports as established is true of all represented concrete cases under the assumptions.

SSA state token. A single-assignment value consumed and produced by effectful IR operations to expose ordering and data dependencies.

Staged authoring. Running a user language in one phase to construct an inert AST for later trusted compilation.

State epoch. A changing identifier that binds authorization to the precise safety-relevant controller state that was checked.

Stock monotonicity. The subtractive-process invariant $S_{i+1} \subseteq S_i$.

Swept volume. The union of all positions occupied by a solid along a trajectory.

Target. Desired or protected part geometry against which allowance, residual, and gouge claims are stated.

Temporal semantics. Meaning defined over complete event and state traces, including safety and liveness.

Time law. A monotone function assigning time to progress along a geometric path.

Topology. Connectivity, adjacency, boundaries, components, and holes independent of small metric deformation.

Trace. A sequence of states and observable events from one execution.

Translation validation. Checking one actual compiler transformation result against its input rather than trusting the transformer for all inputs.

Trajectory. A time-indexed path or machine configuration.

Trusted computing base. The code, definitions, keys, numeric kernels, and assumptions whose correctness is necessary for an assurance claim.

Typestate. Encoding protocol state in types so available operations depend on the current state.

Uncertainty set. A set of possible values for a measurement, transform, tool dimension, or physical behavior.

Weakest precondition. The least restrictive condition sufficient before a command to ensure a desired post-condition afterward.

Widening. An abstract-interpretation operator that forces fixed-point iteration to terminate by moving to a coarser safe abstraction.

Witness. Producer-supplied data intended to demonstrate a construction, correspondence, feasibility result, or proof step.

Work coordinate system. A frame relating part-program coordinates to the machine frame; it is a runtime-critical geometric assumption.

References

The references below are selected for the ideas used directly in the text. They emphasize primary papers, standards, and institutional reports.

[R1] Thomas R. Kramer, Frederick M. Proctor, and Elena R. Messina. *The NIST RS274/NGC Interpreter, Version 3*. NISTIR 6556, National Institute of Standards and Technology, 2000. https://www.nist.gov/publications/nist-rs274ngc-interpreter-version-3.

[R2] Frederick M. Proctor, Thomas R. Kramer, and John L. Michaloski. *Canonical Machining Commands*. NISTIR 5970, National Institute of Standards and Technology, 1997. https://doi.org/10.6028/NIST.IR.5970.

[R3] C. A. R. Hoare. “An Axiomatic Basis for Computer Programming.” *Communications of the ACM* 12, no. 10 (1969): 576–580, 583. https://doi.org/10.1145/363235.363259.

[R4] Edsger W. Dijkstra. “Guarded Commands, Nondeterminacy and Formal Derivation of Programs.” *Communications of the ACM* 18, no. 8 (1975): 453–457. https://doi.org/10.1145/360933.360975.

[R5] Gordon D. Plotkin. “A Structural Approach to Operational Semantics.” *Journal of Logic and Algebraic Programming* 60–61 (2004): 17–139; originally DAIMI FN-19, 1981. https://doi.org/10.1016/j.jlap.2004.03.009.

[R6] Leslie Lamport. “The Temporal Logic of Actions.” *ACM Transactions on Programming Languages and Systems* 16, no. 3 (1994): 872–923. https://doi.org/10.1145/177492.177726.

[R7] Eugenio Moggi. “Notions of Computation and Monads.” *Information and Computation* 93, no. 1 (1991): 55–92. https://doi.org/10.1016/0890-5401(91)90052-4.

[R8] Philip Wadler. “The Essence of Functional Programming.” In *Proceedings of POPL 1992*, 1–14. https://doi.org/10.1145/143165.143169.

[R9] Robert E. Strom and Shaula Yemini. “Typestate: A Programming Language Concept for Enhancing Software Reliability.” *IEEE Transactions on Software Engineering* SE-12, no. 1 (1986): 157–171. https://doi.org/10.1109/TSE.1986.6312929.

[R10] Robert Atkey. “Parameterised Notions of Computation.” *Journal of Functional Programming* 19, nos. 3–4 (2009): 335–376. https://doi.org/10.1017/S095679680900728X.

[R11] Ron Cytron, Jeanne Ferrante, Barry K. Rosen, Mark N. Wegman, and F. Kenneth Zadeck. “Efficiently Computing Static Single Assignment Form and the Control Dependence Graph.” *ACM Transactions on Programming Languages and Systems* 13, no. 4 (1991): 451–490. https://doi.org/10.1145/115372.115320.

[R12] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. “MLIR: Scaling Compiler Infrastructure

for Domain Specific Computation." In *Proceedings of CGO 2021*, 2–14. https://doi.org/10.1109/CGO51591.2021.9370308.

[R13] Xavier Leroy. "Formal Verification of a Realistic Compiler." *Communications of the ACM* 52, no. 7 (2009): 107–115. https://doi.org/10.1145/1538788.1538814.

[R14] Amir Pnueli, Michael Siegel, and Eli Singerman. "Translation Validation." In *TACAS 1998*, 151–166. https://doi.org/10.1007/BFb0054170.

[R15] George C. Necula. "Proof-Carrying Code." In *Proceedings of POPL 1997*, 106–119. https://doi.org/10.1145/263699.263712.

[R16] Walid Taha and Tim Sheard. "MetaML and Multi-Stage Programming with Explicit Annotations." *Theoretical Computer Science* 248, nos. 1–2 (2000): 211–242. https://doi.org/10.1016/S0304-3975(00)00053-0.

[R17] Jonathan Richard Shewchuk. "Adaptive Precision Floating-Point Arithmetic and Fast Robust Geometric Predicates." *Discrete & Computational Geometry* 18 (1997): 305–363. https://doi.org/10.1007/PL00009321.

[R18] Ramon E. Moore, R. Baker Kearfott, and Michael J. Cloud. *Introduction to Interval Analysis.* SIAM, 2009. https://doi.org/10.1137/1.9780898717716.

[R19] Tomás Lozano-Pérez. "Spatial Planning: A Configuration Space Approach." *IEEE Transactions on Computers* C-32, no. 2 (1983): 108–120. https://doi.org/10.1109/TC.1983.1676196.

[R20] J. A. Sethian. "A Fast Marching Level Set Method for Monotonically Advancing Fronts." *Proceedings of the National Academy of Sciences* 93, no. 4 (1996): 1591–1595. https://doi.org/10.1073/pnas.93.4.1591.

[R21] Ron Kimmel and J. A. Sethian. "Computing Geodesic Paths on Manifolds." *Proceedings of the National Academy of Sciences* 95, no. 15 (1998): 8431–8435. https://doi.org/10.1073/pnas.95.15.8431.

[R22] Ilker Kucukoglu, Tulin Gunduz, Fatma Balkancioglu, Emine Chousein Topal, and Oznur Sayim. "Application of Precedence Constrained Travelling Salesman Problem Model for Tool Path Optimization in CNC Milling Machines." *International Journal of Optimization and Control: Theories & Applications* 9, no. 3 (2019): 59–68. https://doi.org/10.11121/ijocta.01.2019.00662.

[R23] Qiang Zhang, Shurong Li, and Jianxin Guo. "Minimum Time Trajectory Optimization of CNC Machining with Tracking Error Constraints." *Abstract and Applied Analysis* 2014, Article 835098. https://doi.org/10.1155/2014/835098.

[R24] Hung Pham and Quang-Cuong Pham. "A New Approach to Time-Optimal Path Parameterization Based on Reachability Analysis." *IEEE Transactions on Robotics* 34, no. 3 (2018): 645–659. https://doi.org/10.1109/TRO.2018.2819195.

[R25] J. Tanner Slagel, Lauren M. White, Aaron Dutle, César A. Muñoz, and Nicolas Crespo. "A Formal Verification Framework for Runtime Assurance." In *NASA Formal Methods 2024*, 322–328. https://doi.org/10.1007/978-3-031-60698-4_19.

[R26] ISO 14649-1. *Industrial Automation Systems and Integration — Physical Device Control — Data Model for Computerized Numerical Controllers — Part 1: Overview and Fundamental Principles.* International Organization for Standardization.

[R27] Masatomo Inui, Takashi Sakurai, and Nobuyuki Umezu. "Data Conversion Technology between Triple Dexel Model and Polygonal Model." *Journal of the Japan Society for Precision Engineering* 76, no. 2 (2010):

226–231. https://doi.org/10.2493/jjspe.76.226.

[R28] Weihan Zhang and Ming-Chuan Leu. "Surface Reconstruction Using Dexel Data from Three Sets of Orthogonal Rays." *Journal of Computing and Information Science in Engineering* 9, no. 1 (2009): 011008. https://doi.org/10.1115/1.3086034.

[R29] Andrew W. Appel. "Foundational Proof-Carrying Code." In *Proceedings of the 16th Annual IEEE Symposium on Logic in Computer Science*, 247–256, 2001. https://doi.org/10.1109/LICS.2001.932501.

[R30] Patrick Cousot and Radhia Cousot. "Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints." In *Proceedings of POPL 1977*, 238–252. https://doi.org/10.1145/512950.512973.

Source Snapshot and Reproducibility Note

The implementation-specific observations in this edition are based on the supplied `dropcut-studio.zip` snapshot inspected in August 2026. They describe that snapshot, not later revisions. The text distinguishes nominal algorithms, sampled simulations, and sound certificates deliberately; a worked specification is not a claim that the supplied implementation already establishes it.

The Markdown file is the canonical source of this edition. Diagrams are generated from Graphviz or Matplotlib source included in the source bundle. The PDF is rendered from the same Markdown through Pandoc and XeLaTeX. The TypeScript signatures and pseudocode are pedagogical reference designs. They require implementation, independent validation, machine-specific profiles, and live assumption checks before they can support material cutting.